



ELEC 5690 - Advanced Topics in Artificial Intelligence for Medical Image Analysis

Lecture 02: Basic Vision Models (Classification)

Sep 15, 2026



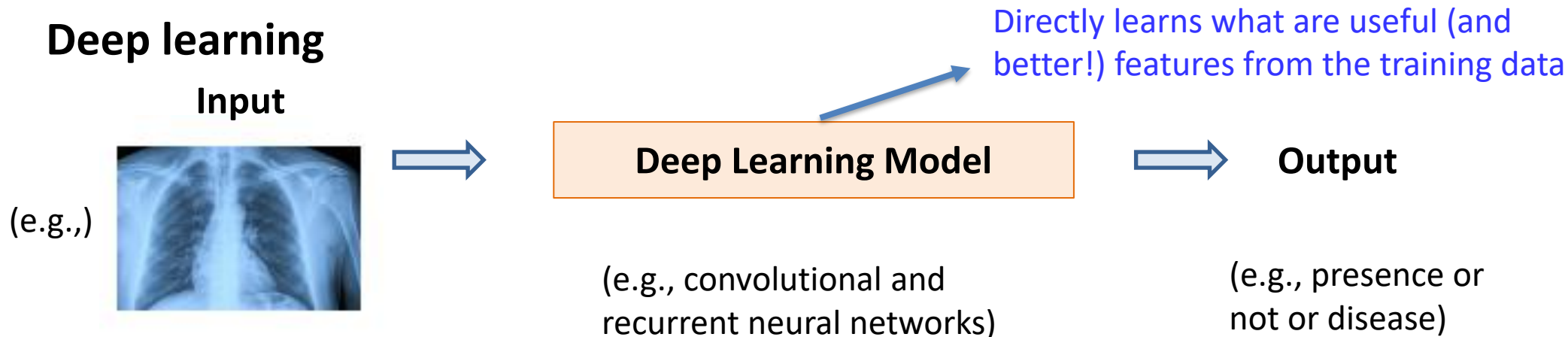
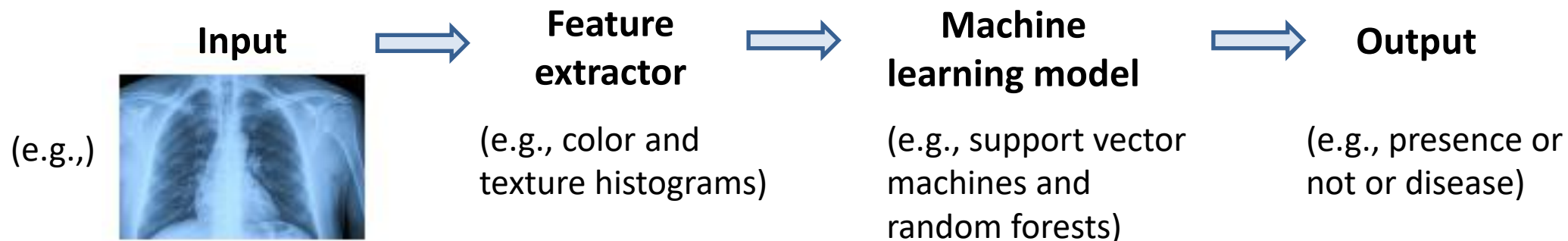
- Review what we have learned last time
- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies of CNNs for medical image classification

1. Last time



- Deep learning (a type of machine learning)

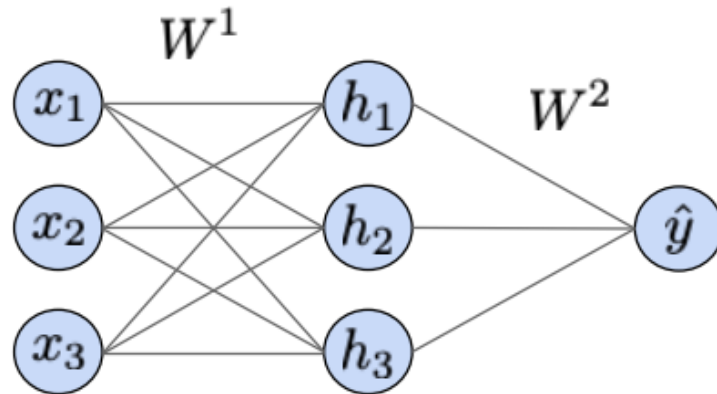
Traditional machine learning approaches



1. Last time



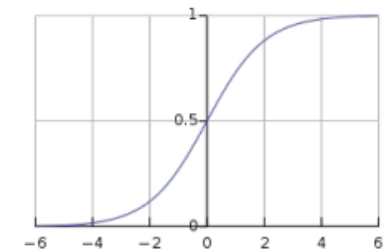
- Define a two-layer fully-connected neural network



Output: $h = \sigma(W^1x + b^1)$
 $\hat{y} = W^2h + b^2$

Full function expression:

$$\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$$



Sigmoid "activation function"

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

Activation functions

Introduce non-linearity into the model --
allowing it to represent highly complex functions.

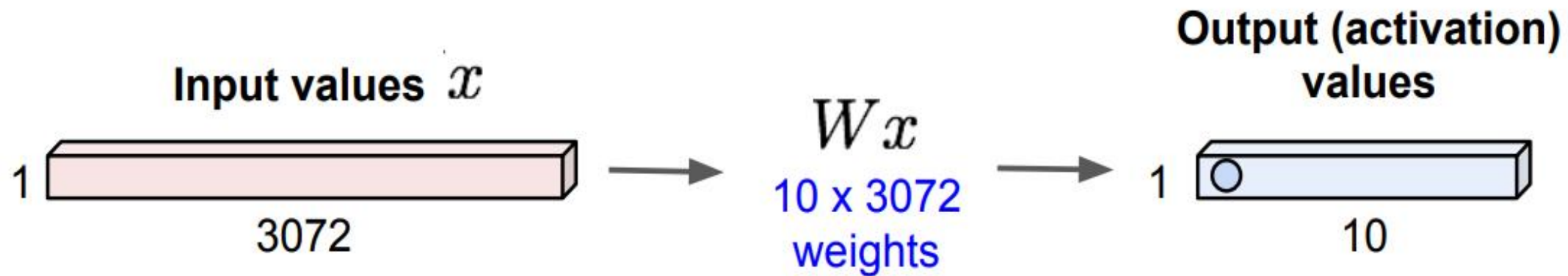
Dimensions of the weight matrix for each fully
connected layer is [output dim. x input dim.]

Dimensions of the bias vector is [output dim x 1]

1. Last time



Fully-connected layers: in graphical form



$$\hat{y} = Wx$$

Diagram illustrating the equation $\hat{y} = Wx$ with labels for the components:

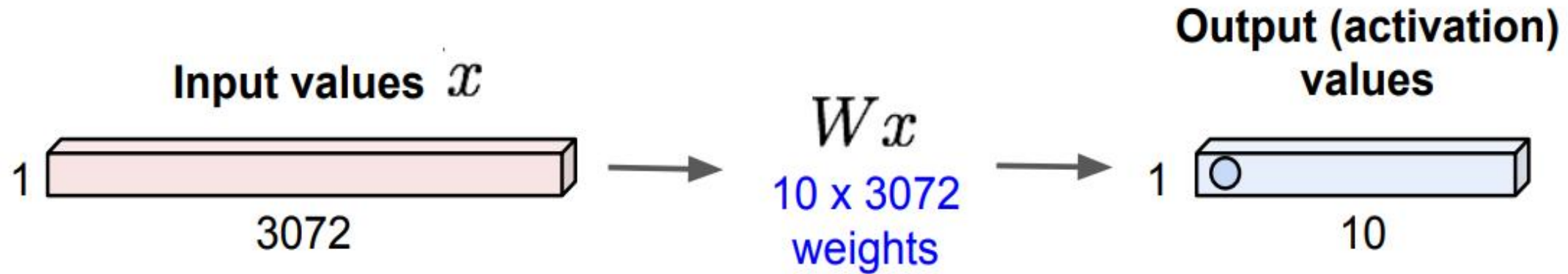
- $\hat{y}$ is labeled Output (10×1)
- W is labeled Weights (10×3072)
- x is labeled Input (3072×1)

Slide credit: CS231n

1. Last time



Fully-connected layers: in graphical form



In Keras:

```
keras_model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=1, use_bias=True)
])
keras_model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                    loss='mse')
keras_model.fit(dataset, epochs=1000)
```

Activation function and bias configurations included!

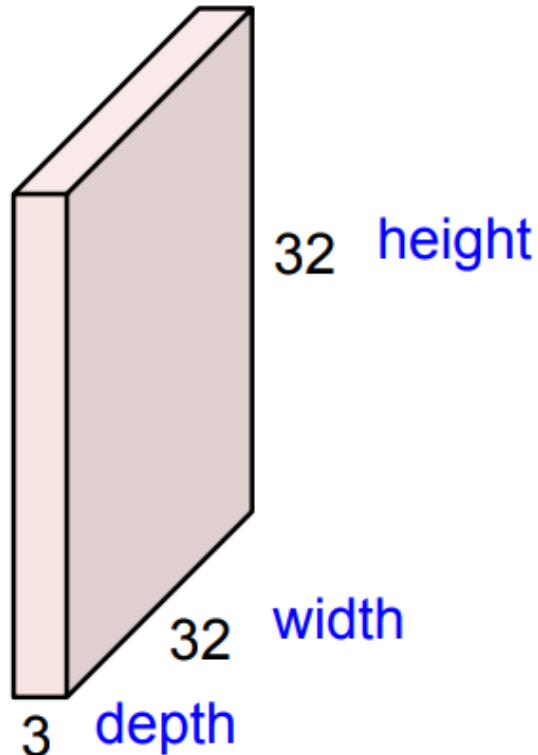
Slide credit: CS231n

1. Last time



Convolutional layer

32x32x3 image -> preserve spatial structure



Input now has spatial height and width dimensions!

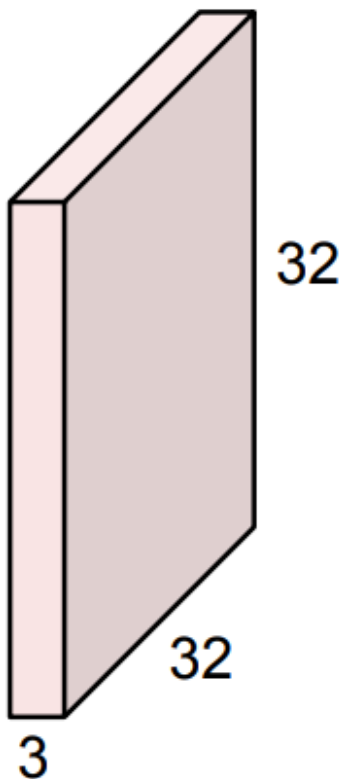
In contrast to fully-connected layers, we want to preserve spatial structure when processing with a convolutional layer

1. Last time



Convolutional layer

32x32x3 image



Filters always extend the full depth of the input volume

5x5x3 filter (weights)

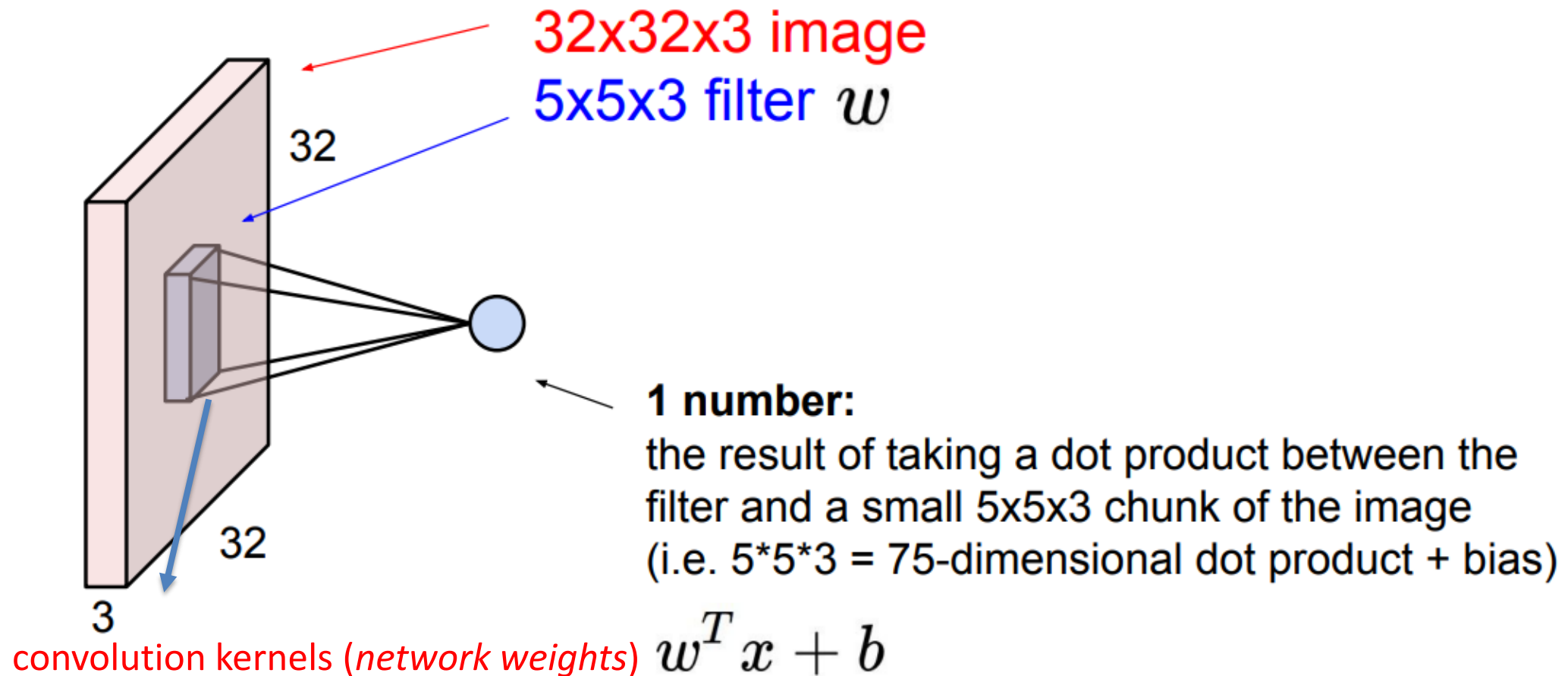


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

1. Last time



Convolutional layer



1. Last time



Convolutional layer

$\times 1$	$\times 0$	$\times 1$
$\times 0$	$\times 1$	$\times 0$
$\times 1$	$\times 0$	$\times 1$

Filter



convolution kernels (*network weights*)

1 _{$\times 1$}	1 _{$\times 0$}	1 _{$\times 1$}	0	0
0 _{$\times 0$}	1 _{$\times 1$}	1 _{$\times 0$}	1	0
0 _{$\times 1$}	0 _{$\times 0$}	1 _{$\times 1$}	1	1
0	0	1	1	0
0	1	1	0	0

Image

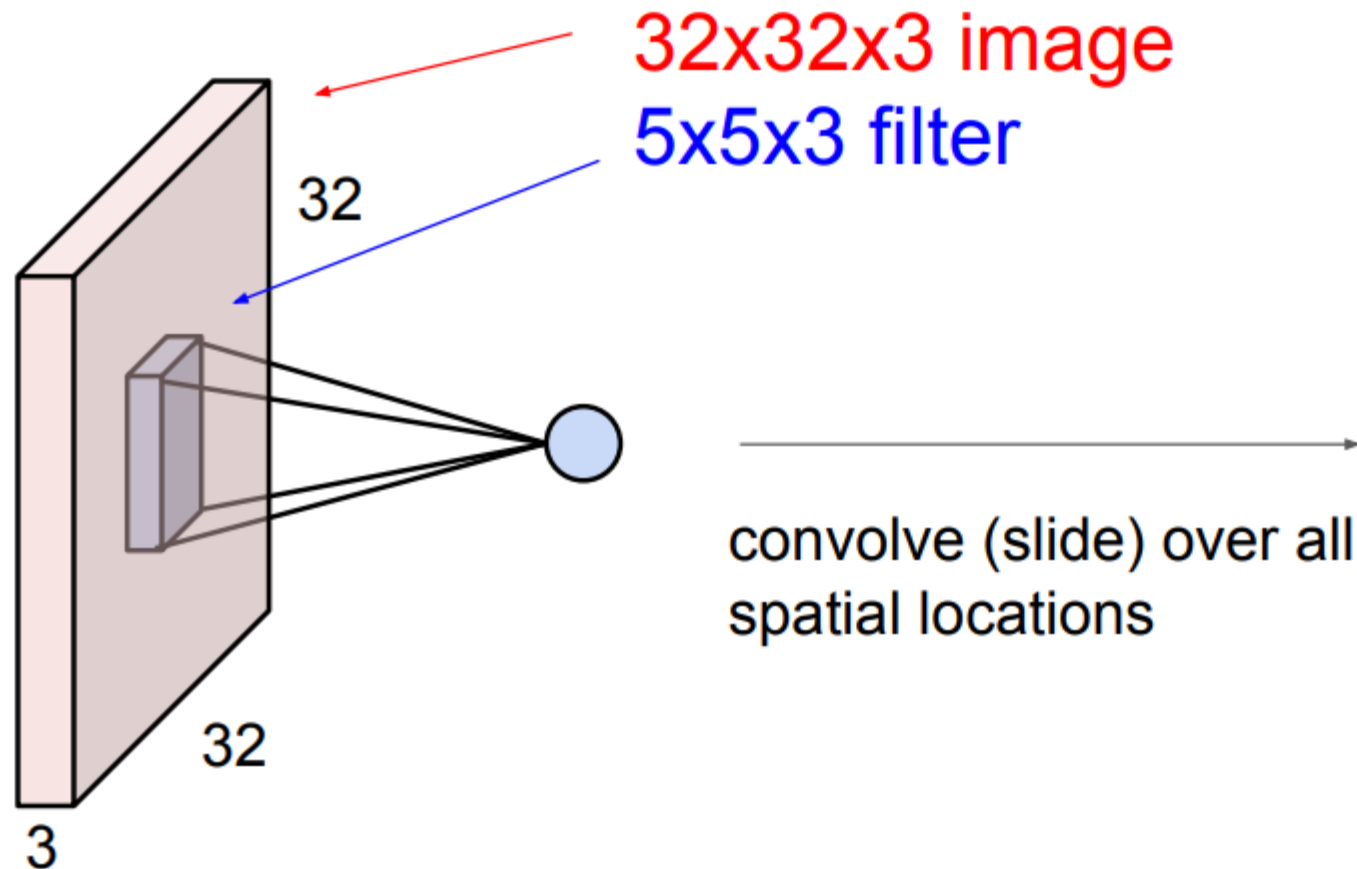
4		

Convolved
Feature

1. Last time



Convolutional layer

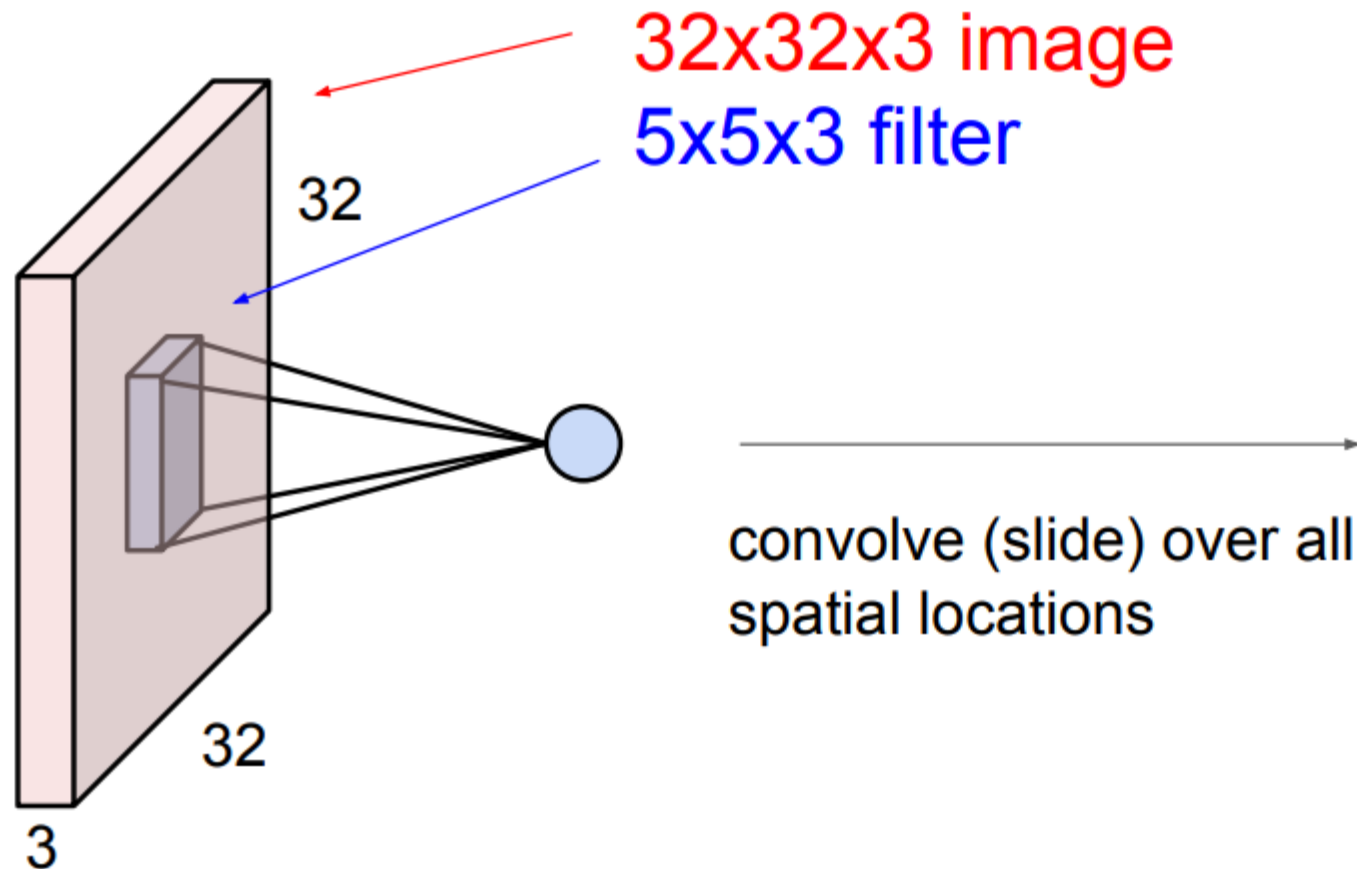


Activation map

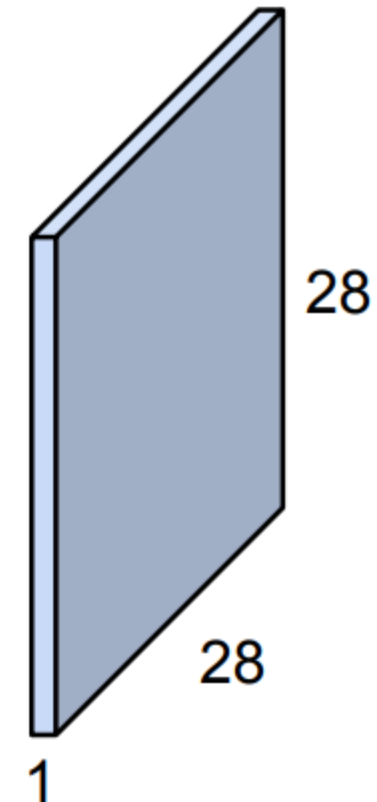
1. Last time



Convolutional layer



Activation map

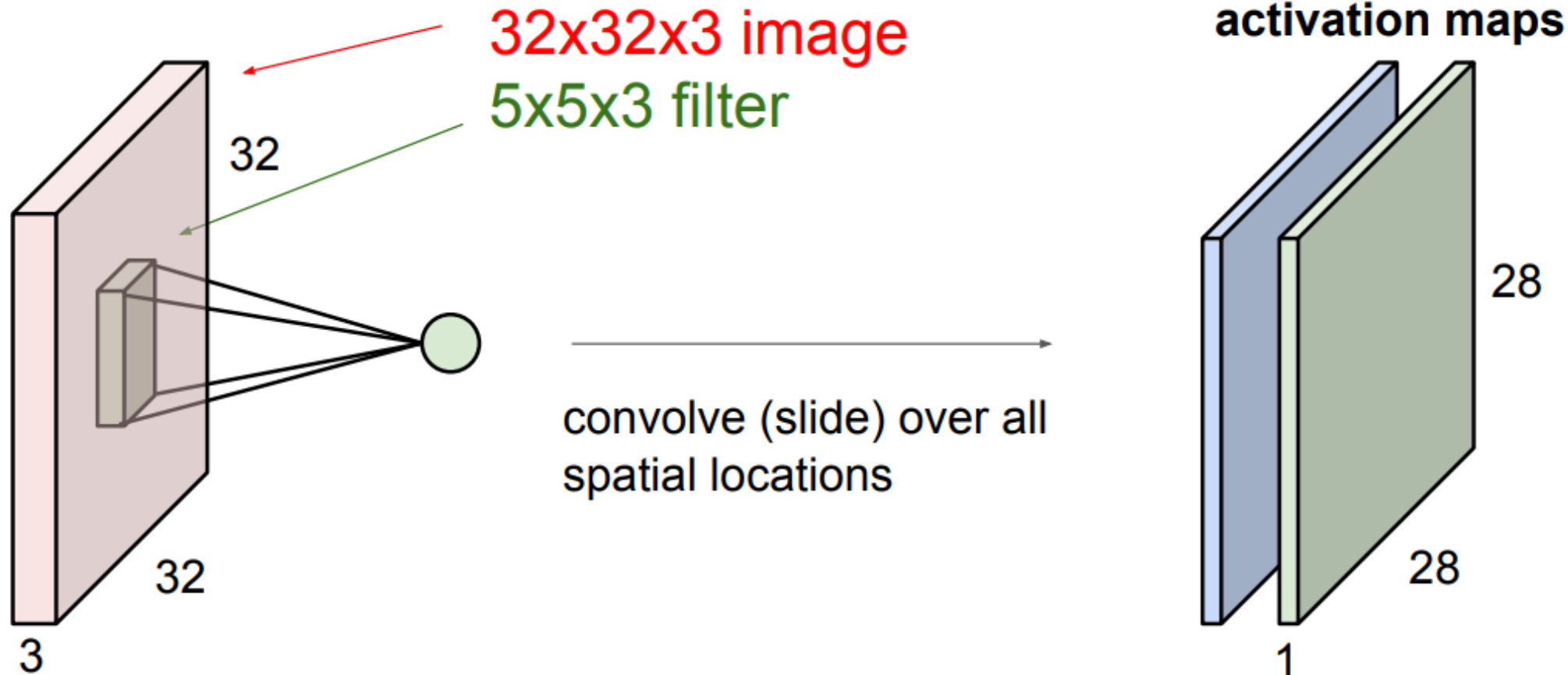


1. Last time



Convolutional layer

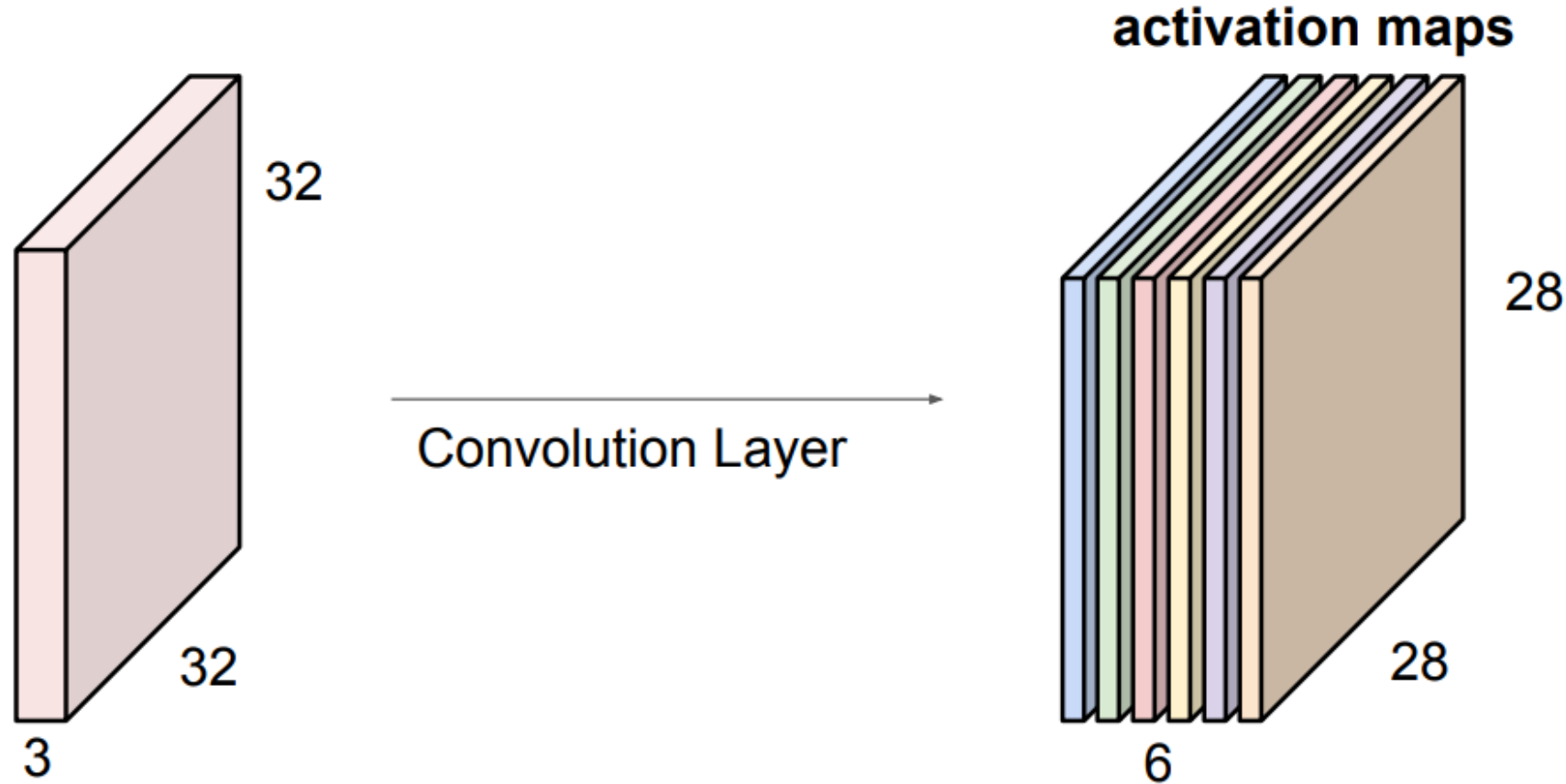
consider a second, **green** filter



1. Last time



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

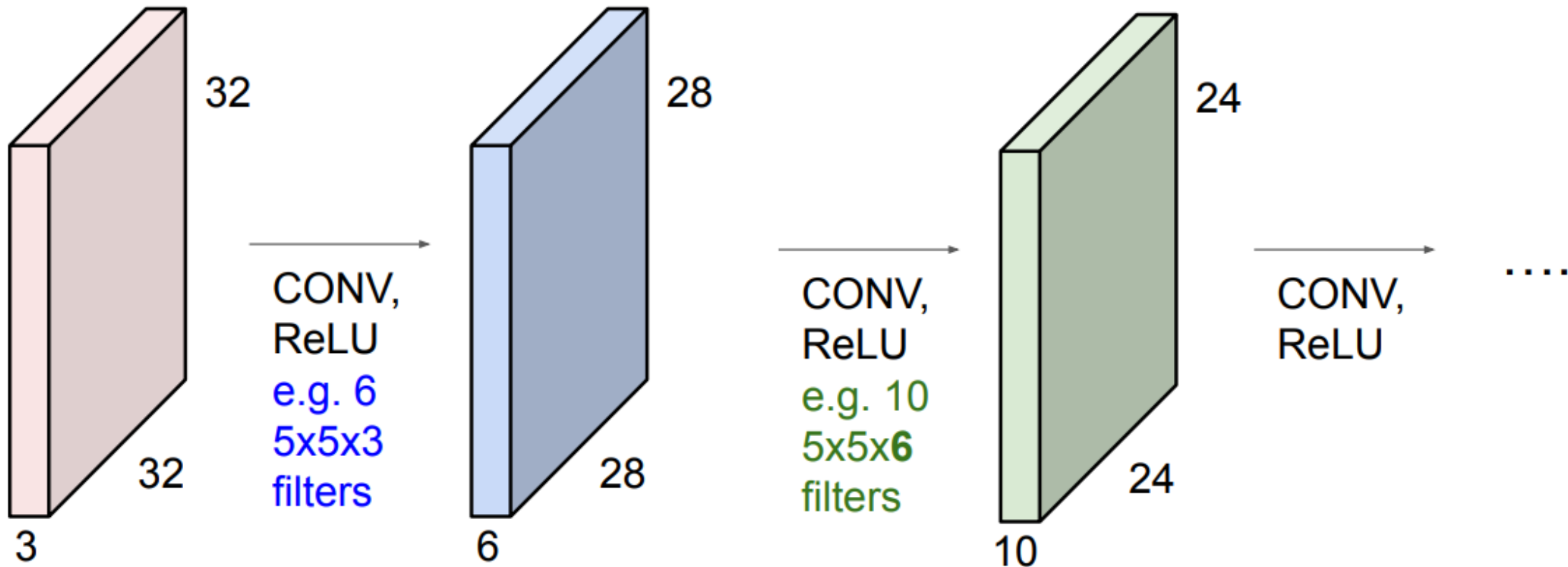


We stack these up to get a “new image” of size 28x28x6!

1. Last time



Preview: ConvNet (or CNN) is a sequence of Convolution Layers, interspersed with activation functions



1. Last time



Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

Common settings:

$K =$ (powers of 2, e.g. 32, 64, 128, 512)

- $F = 3, S = 1, P = 1$
- $F = 5, S = 1, P = 2$
- $F = 5, S = 2, P = ?$ (whatever fits)
- $F = 1, S = 1, P = 0$

$W_2 = (W_1 - 3 + 2)/1 + 1$

$W_2 = (W_1 - 5 + 2P)/2 + 1$

1. Last time



In Keras

Padding options: 'valid' does not pad, use 'same' to pad such that input and output spatial dimensions are the same size

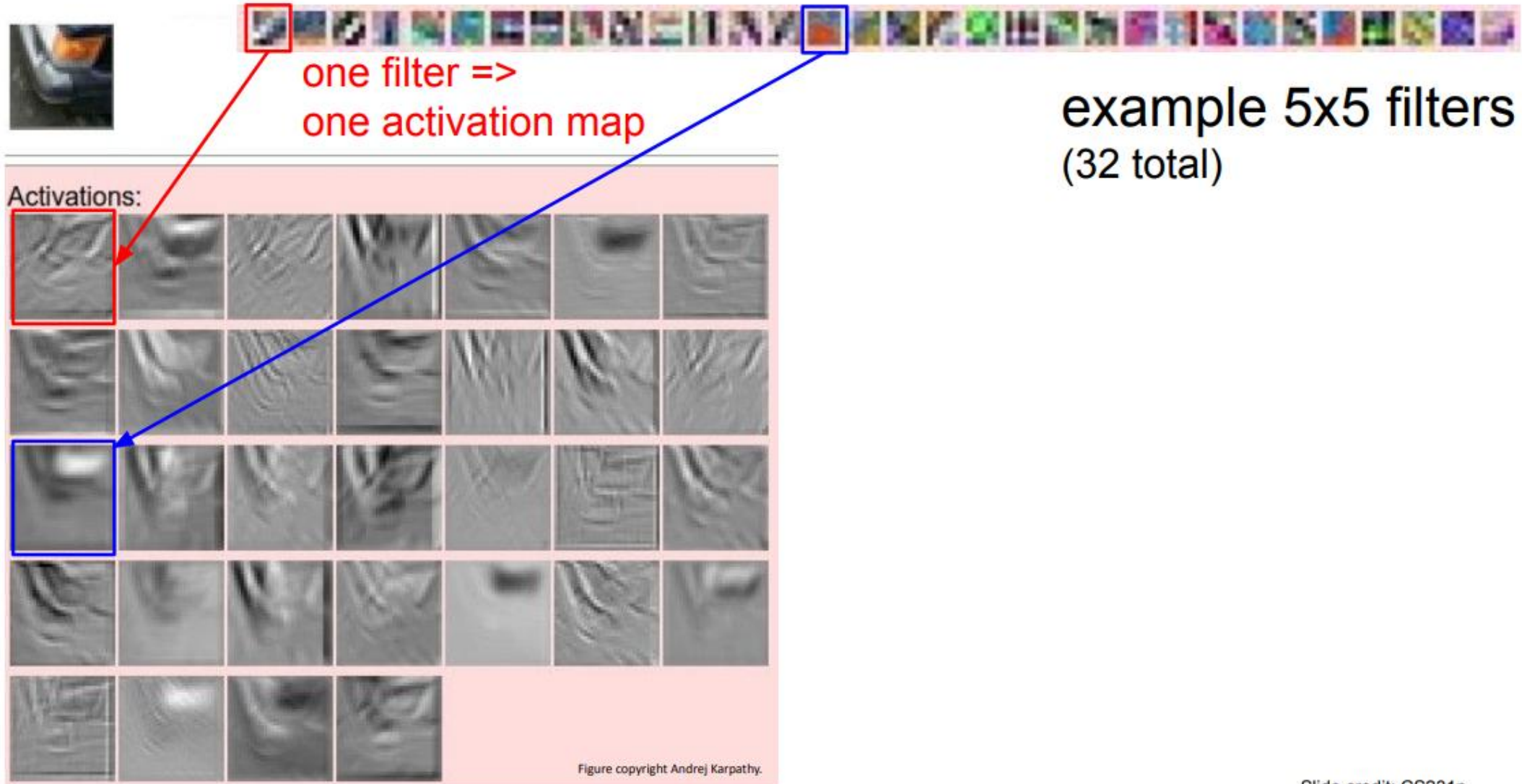
Conv2D

[\[source\]](#)

```
keras.layers.Conv2D(filters, kernel_size, strides=(1, 1), padding='valid', data_format=None, d:
```

2D convolution layer (e.g. spatial convolution over images).

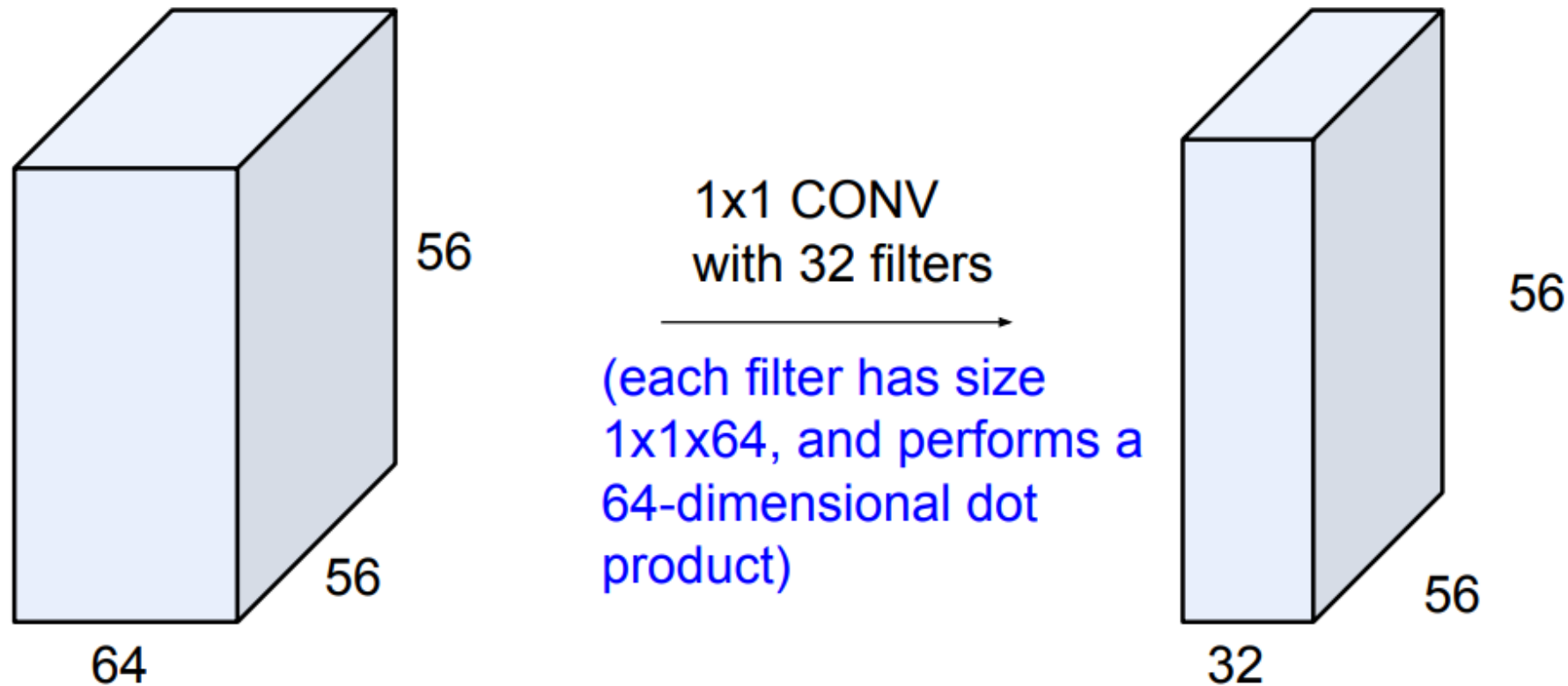
1. Last time



1. Last time



(btw, 1x1 convolution layers make perfect sense -> performs **dimensionality reduction** in the depth dimension)

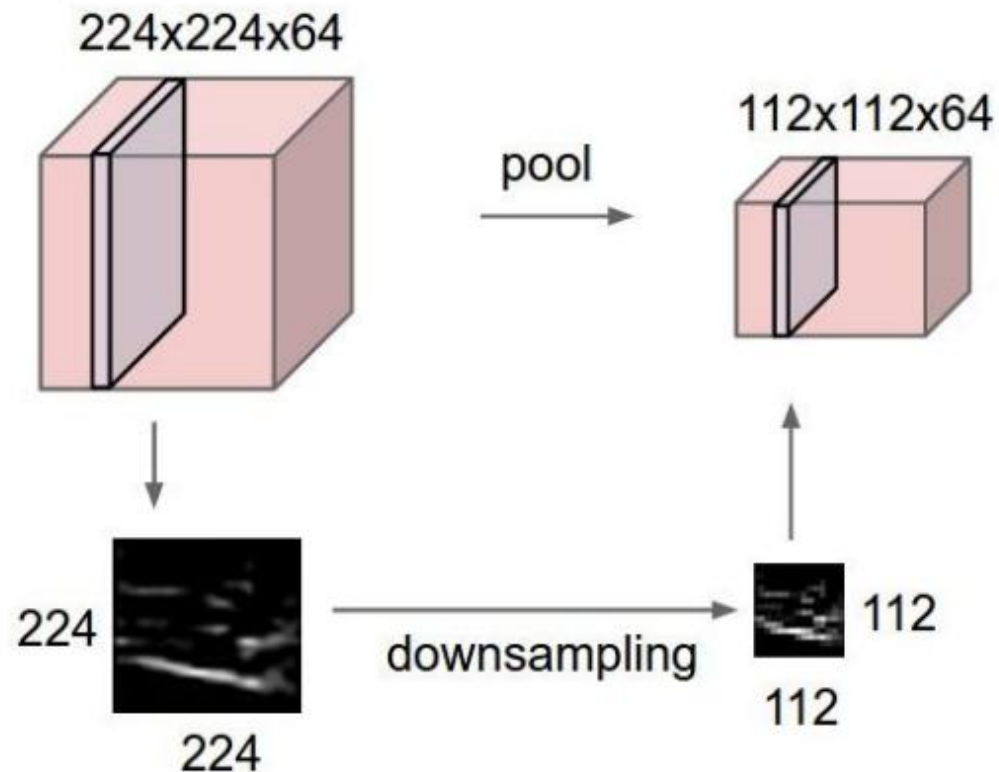


1. Last time



Pooling layer

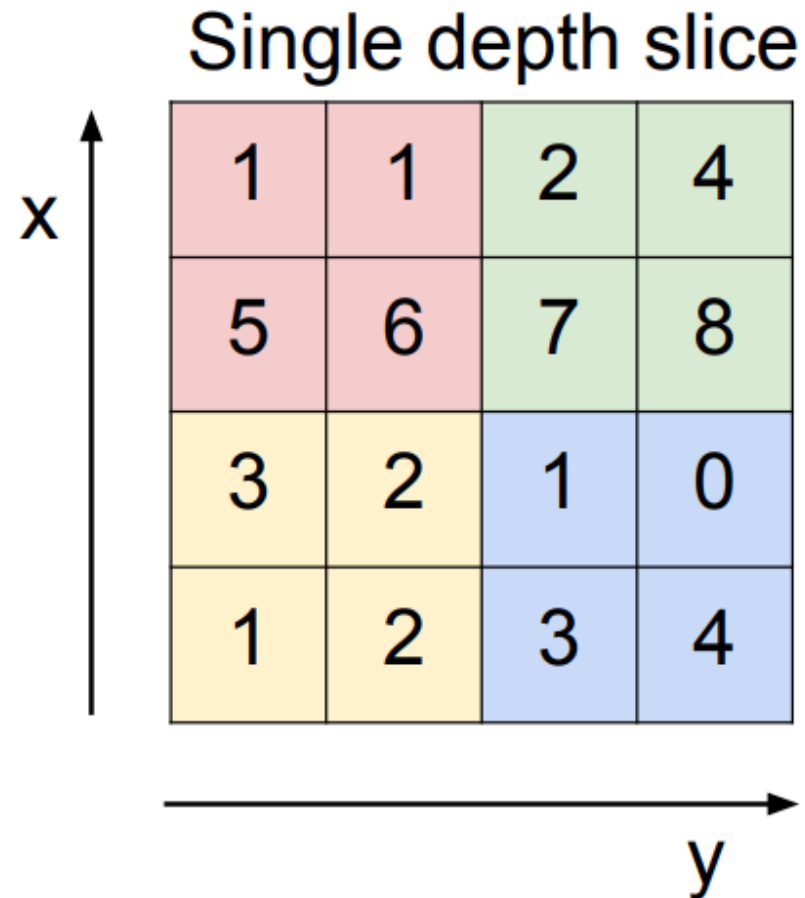
- makes the representations smaller and more manageable
- operates over each activation map independently:



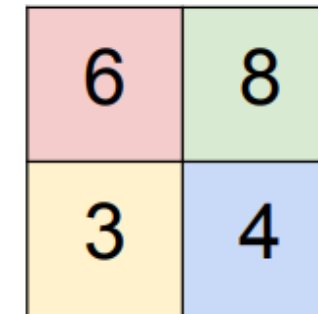
1. Last time



Max pooling



max pool with 2x2 filters
and stride 2



1. Last time



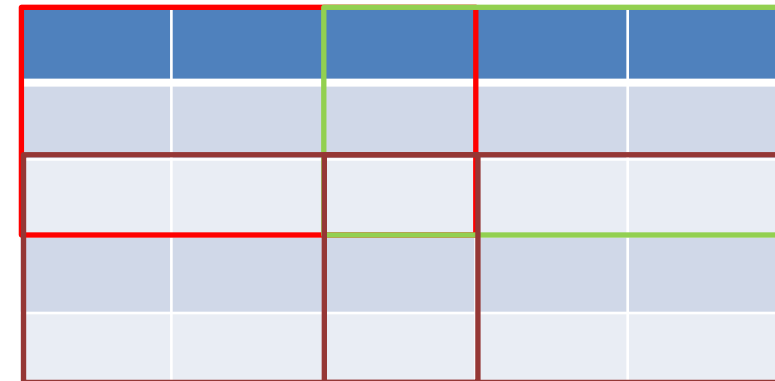
Pooling layer: practical implementation

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
 - their spatial extent F ,
 - the stride S ,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F)/S + 1$
 - $H_2 = (H_1 - F)/S + 1$
 - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

Common settings:

$F = 2, S = 2$

$F = 3, S = 2$



In Keras:

MaxPooling2D

```
keras.layers.MaxPooling2D(pool_size=(2, 2), strides=None, padding='valid', data_format=None)
```

[source]

1. Last time



Optimization methods: SGD, SGD with momentum, RMSProp, Adam

Vannilla GD (SGD): stochastic gradient descent

SGD refers to the specific case of vanilla GD when the batch size is 1.

$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w_{t-1}}$$

$$b_t = b_{t-1} - \eta \frac{\partial L}{\partial b_{t-1}}$$

SGD subtracts the gradient multiplied by the learning rate from the weights.

Relax on this “one single data point” to “a subset of data”, then batch and epochs



GD: gradient is calculated precisely from all the data points.

SGD: quick and dirty way to “approximate gradient” from one single data point

1. Last time



Optimization methods: SGD, SGD with momentum, RMSProp, Adam

SGD with momentum

The present gradient is dependent on its previous gradient and so on. It accelerates Sc

llation



Image 2: SGD without momentum

$$b_t = b_{t-1} - \eta \frac{\partial L}{\partial b_{t-1}}$$

SGD



Image 3: SGD with momentum

$$b_t = b_{t-1} - \eta V_{db_t}$$

$$\text{where } V_{db_t} = \beta V_{db_{t-1}} + (1 - \beta) \frac{\partial L}{\partial b_{t-1}}$$

SGD with momentum

$$\tau_{dw_{t-1}} + (1 - \beta) \frac{\partial L}{\partial w_{t-1}}$$

1. Last time



Optimization methods: **SGD**, **SGD with momentum**, **RMSProp**, **Adam**

RMSProp: Root mean square propagation

Uses a moving average of squared gradients to normalize the gradient. It can automatically scale the learning rate, decreasing the step for large gradients and increasing the step for small gradients.

$$\theta_i = \theta_i - \alpha \boxed{\frac{\partial L}{\partial \theta_i}}$$
$$v_i = \gamma v_i + \alpha \frac{\partial L}{\partial \theta_i} \leftarrow$$
$$\theta_i = \theta_i - \boxed{v_i} \leftarrow$$

SGD

SGD with momentum

↑ Increase

Large

$$v_i = \beta v_i + (1 - \beta) \left(\frac{\partial L}{\partial \theta_i} \right)^2 \leftarrow$$
$$\theta_i = \theta_i - \alpha \boxed{\frac{\left(\frac{\partial L}{\partial \theta_i} \right)}{\sqrt{v_i} + \epsilon}} \leftarrow$$

RMSProp

1. Last time



Optimization methods: **SGD, SGD with momentum, RMSProp, Adam**

Adam (Kingma & Ba, 2014) is one of the state-of-the-art optimization algorithms
Combines both RMSProp and momentum

$$m_i = \beta_1 m_i + (1 - \beta_1) \frac{\partial L}{\partial \theta_i} \leftarrow$$

$$v_i = \beta_2 v_i + (1 - \beta) \left(\frac{\partial L}{\partial \theta_i} \right)^2 \leftarrow$$

$$\widehat{m}_i = \frac{m_i}{1 - \beta_1}, \quad \widehat{v}_i = \frac{v_i}{1 - \beta_2} \leftarrow$$

$$\theta_i = \theta_i - \frac{\alpha}{\sqrt{\widehat{v}_i} + \epsilon} \widehat{m}_i \leftarrow$$

Increase ↗ Large ↗

$$v_i = \beta v_i + (1 - \beta) \left(\frac{\partial L}{\partial \theta_i} \right)^2 \leftarrow$$

$$\theta_i = \theta_i - \alpha \frac{\left(\frac{\partial L}{\partial \theta_i} \right)}{\sqrt{v_i} + \epsilon} \leftarrow$$

RMSProp

1. Last time



Conclusion

Adam has become a default optimization algorithms regardless of fields [1].

[9] has the following comments

- Adaptive methods (e.g., RMSProp and Adam) find solutions that **generalize worse than** those found by non-adaptive methods (SGD and momentum).
- Even when the adaptive methods achieve the same training loss or lower than non-adaptive methods, the test performance is worse.
- Adaptive methods often display **faster initial progress on the training set**, but their performance quickly plateaus on the validation set.
- Though conventional wisdom suggests that Adam does not require tuning, **we find that tuning the initial learning rate and decay scheme for Adam yields significant improvements over its default settings in all cases.**

[1] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

[2] Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., & Recht, B. (2017). The marginal value of adaptive gradient methods in machine learning. *arXiv preprint arXiv:1705.08292*.



Conclusion

A recent paper shows that hyperparameters could be the reason that adaptive methods fail to generalize [3]

- They observe that RMSProp, Adam, and NAdam **never underperformed SGD, NESTEROV, or Momentum**. Although there are some limitations that the experiments were done with some potentially confounding settings(e.g. didn't tune batch size, specific tuning protocol) the message stated was alerting and interesting.
- For now, we could say that **fine-tuned Adam is always better than SGD**, while there exists a performance gap between Adam and SGD when using default hyperparameters.



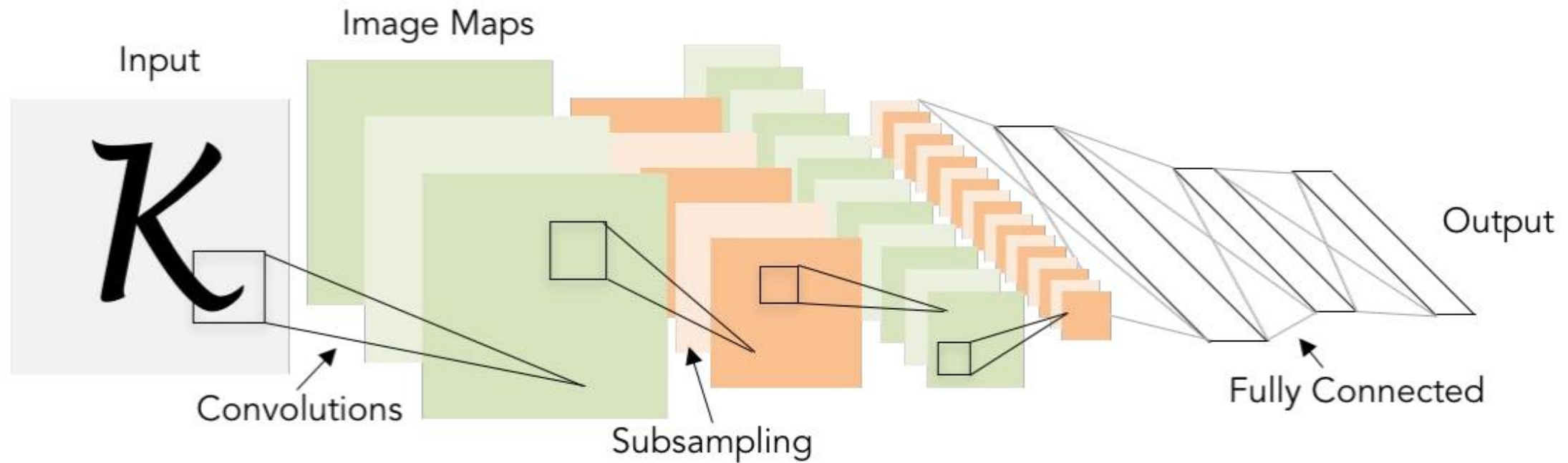
- Review what we have learned last time
- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies of CNNs for medical image classification

2. Deep learning models for image classification



LeNet-5

[LeCun et al., 1998]



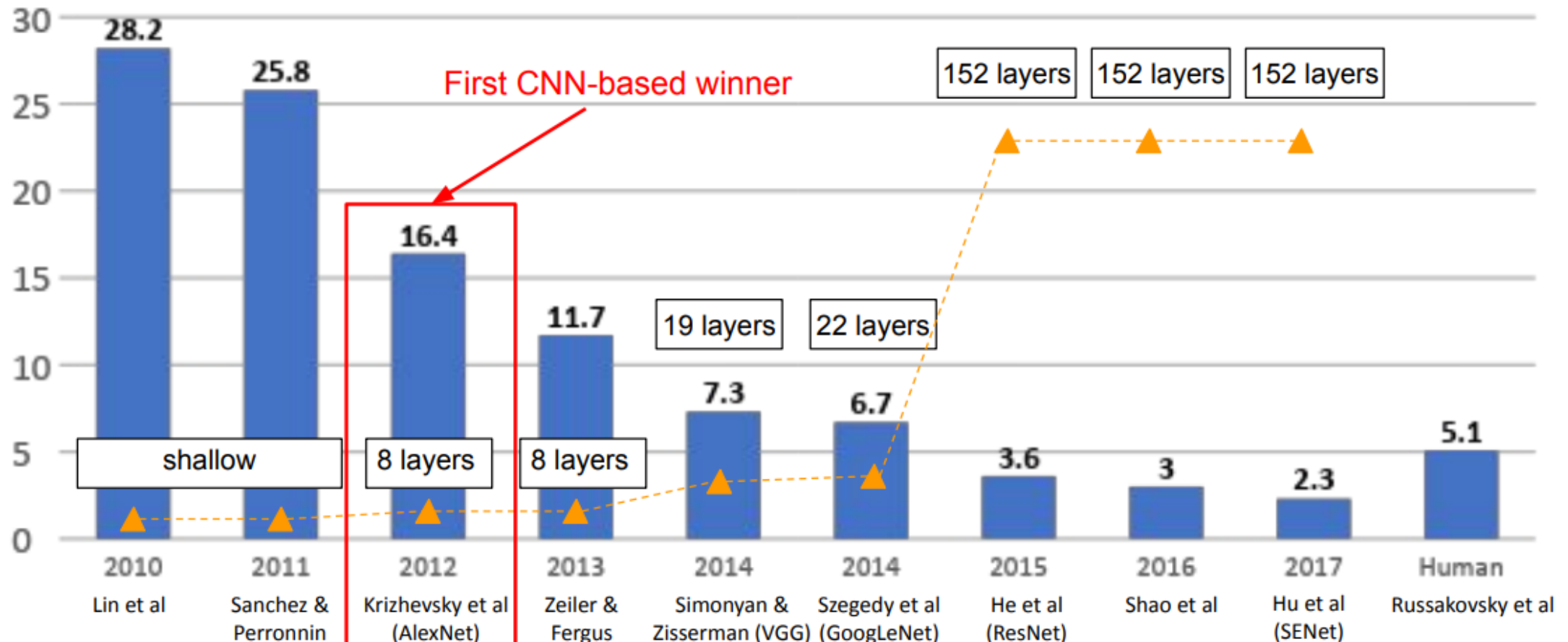
Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-FC-FC-FC]

Slide credit: CS231n

2. Deep learning models for image classification



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Slide credit: CS231n

2. Deep learning models for image classification



AlexNet

[Krizhevsky et al. 2012]

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

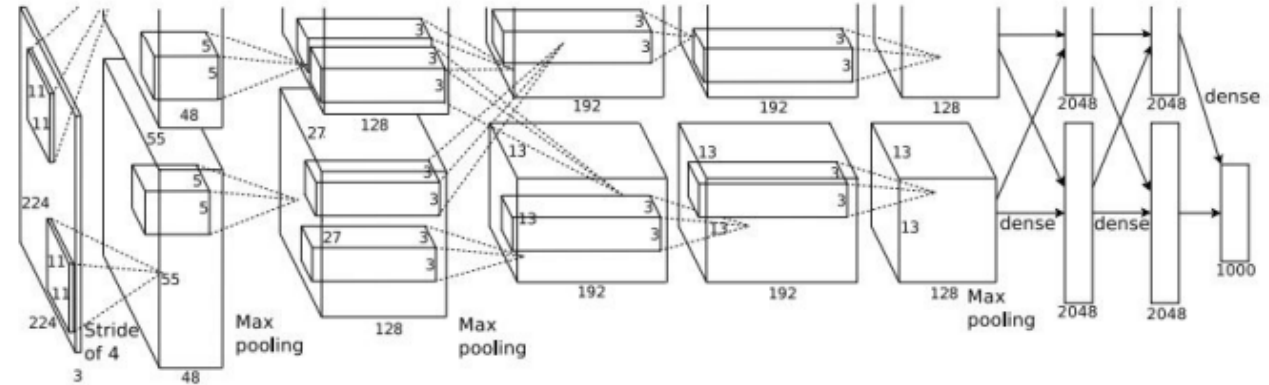
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



Details (many familiar from Lecture 2):

- first use of ReLU
- used Norm layers (not common anymore)
- heavy data augmentation
- dropout 0.5
- batch size 128
- SGD Momentum 0.9
- Learning rate 1e-2, reduced by 10 manually when val accuracy plateaus
- L2 weight decay 5e-4 (regularization)
- 7 CNN ensemble: 18.2% -> 15.4%

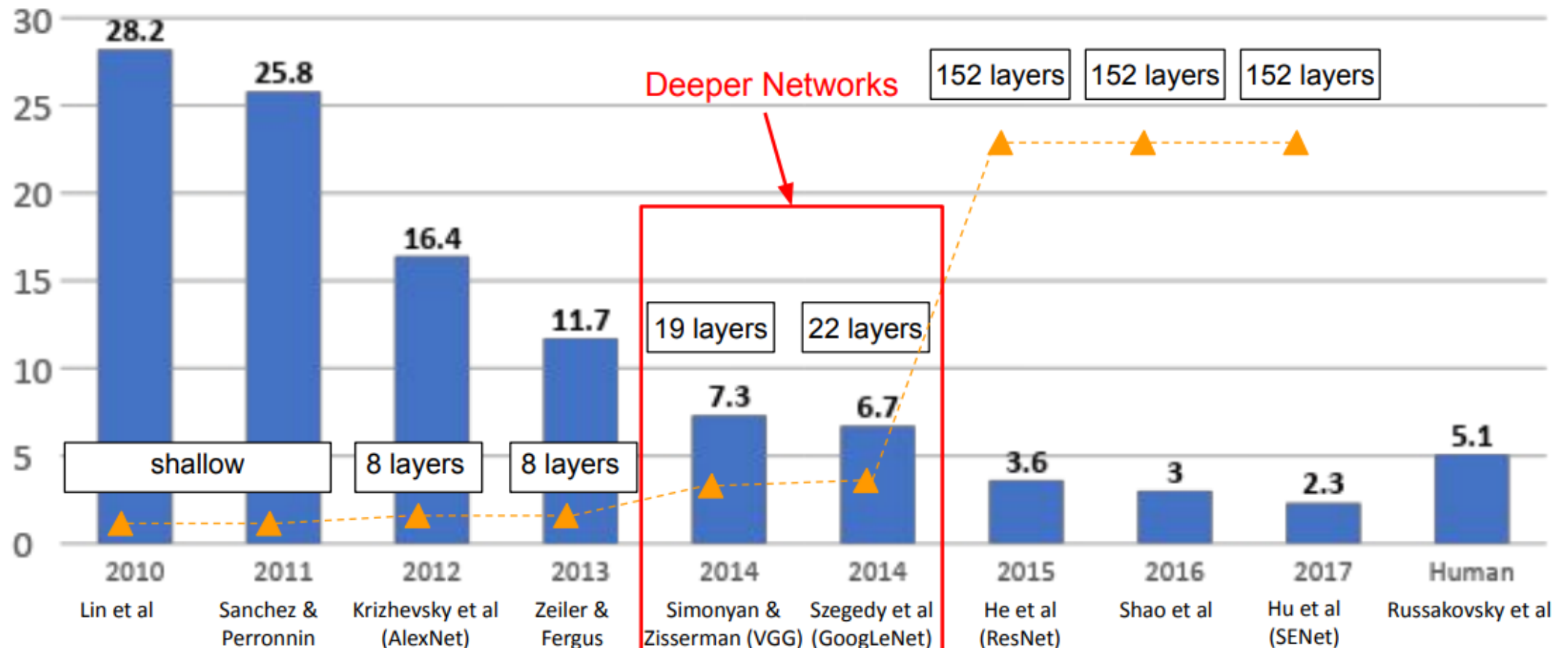
Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Slide credit: CS231n

2. Deep learning models for image classification



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Slide credit: CS231n

2. Deep learning models for image classification



VGGNet

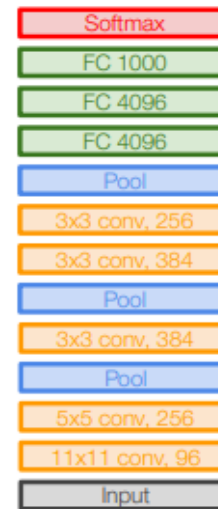
[Simonyan and Zisserman, 2014]

Small filters, Deeper networks

8 layers (AlexNet)

-> 16 - 19 layers (VGG16Net)

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2



AlexNet



VGG16

VGG19

Slide credit: CS231n

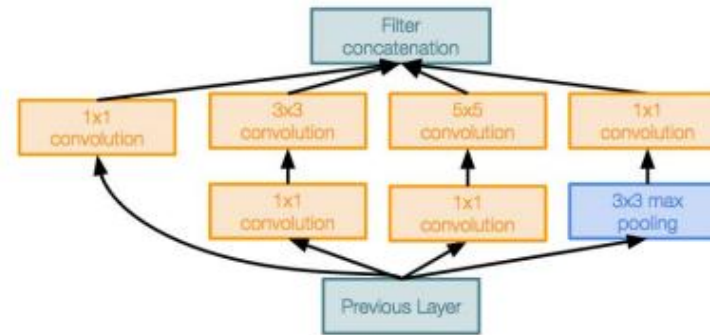
2. Deep learning models for image classification



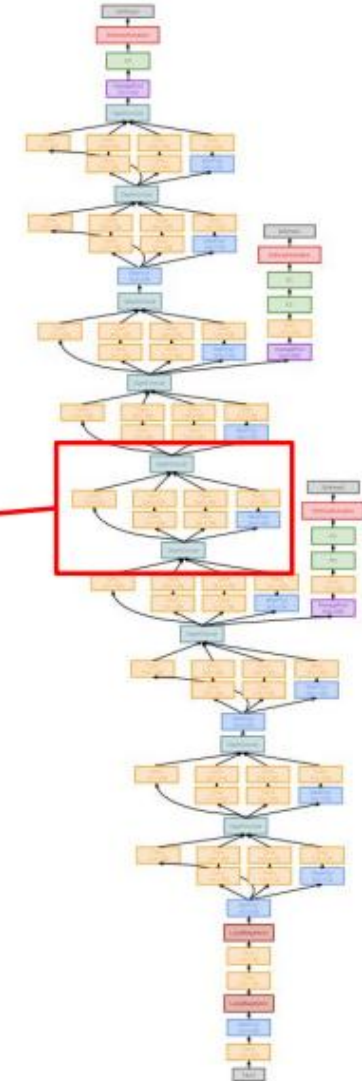
GoogLeNet

[Szegedy et al., 2014]

“Inception module”: design a good local network topology (network within a network) and then stack these modules on top of each other



Inception module



2. Deep learning models for image classification



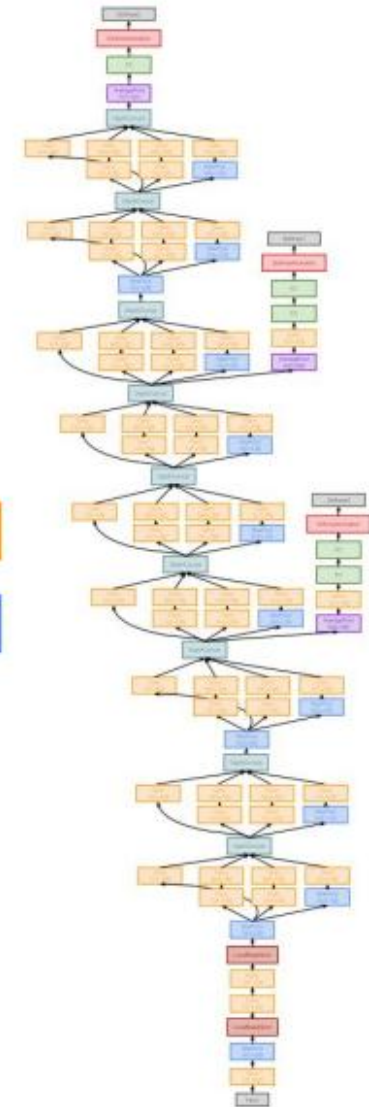
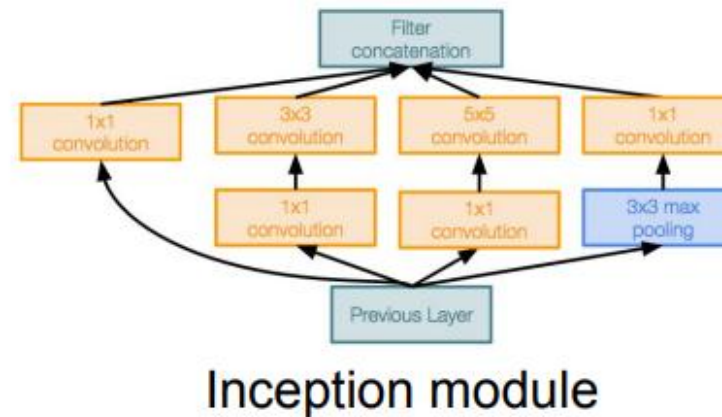
GoogLeNet

[Szegedy et al., 2014]

Deeper networks, with computational efficiency

- 22 layers
- Efficient “Inception” module
- Avoids expensive FC layers using a global averaging layer
- 12x less params than AlexNet

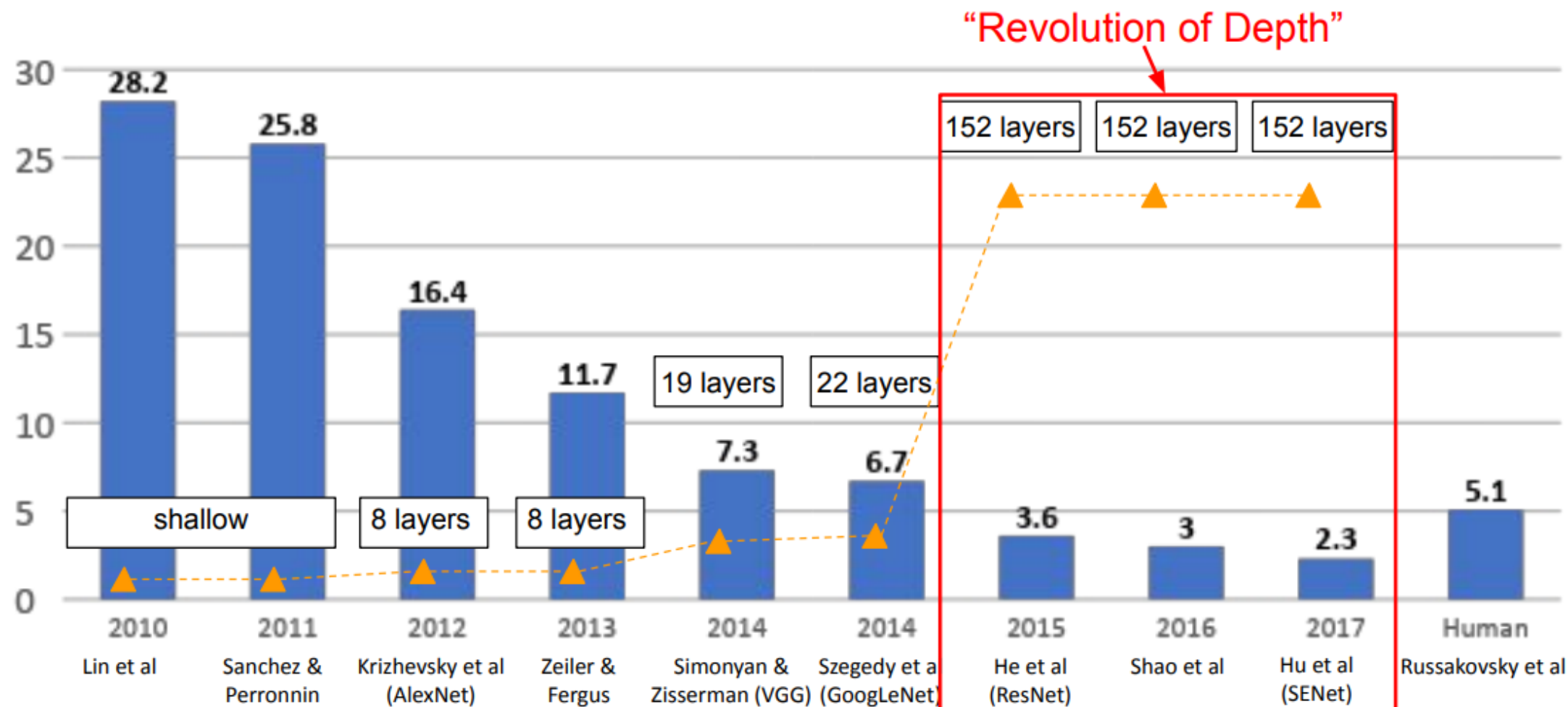
Also called “Inception Network”



2. Deep learning models for image classification



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



2. Deep learning models for image classification

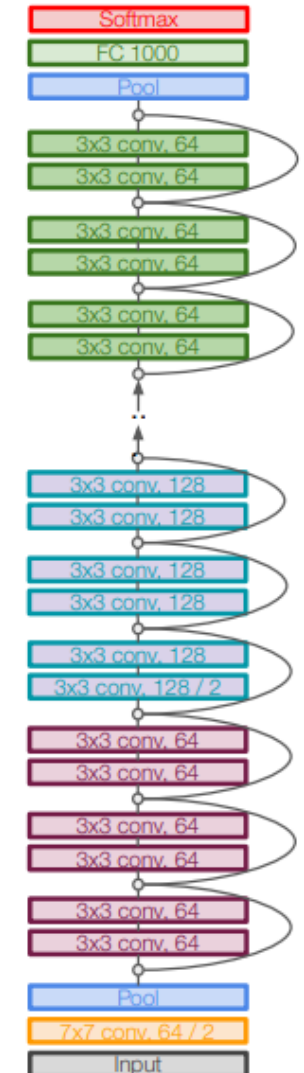
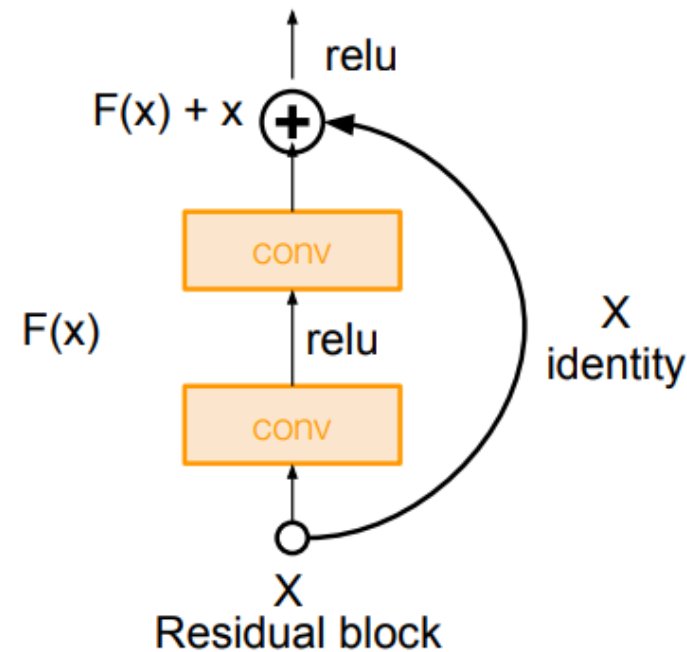


ResNet

[He et al., 2015]

Very deep networks using residual connections

- 152-layer model for ImageNet
- Won all major classification and detection benchmark challenges in 2015



2. Deep learning models for image classification



ResNet

[He et al., 2015]

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Q: What's strange about these training and test curves?
[Hint: look at the order of the curves]

2. Deep learning models for image classification



ResNet

[He et al., 2015]

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



56-layer model performs worse on both training and test error
-> The deeper model performs worse, but it's not caused by overfitting!

Slide credit: CS231n

2. Deep learning models for image classification



ResNet

[He et al., 2015]

Hypothesis: the problem is an *optimization* problem, deeper models are harder to optimize

2. Deep learning models for image classification



ResNet

[He et al., 2015]

Hypothesis: the problem is an *optimization* problem, deeper models are harder to optimize

The deeper model should be able to perform at least as well as the shallower model.

A solution by construction is copying the learned layers over from the shallower model and setting all additional layers to the **identity** function.

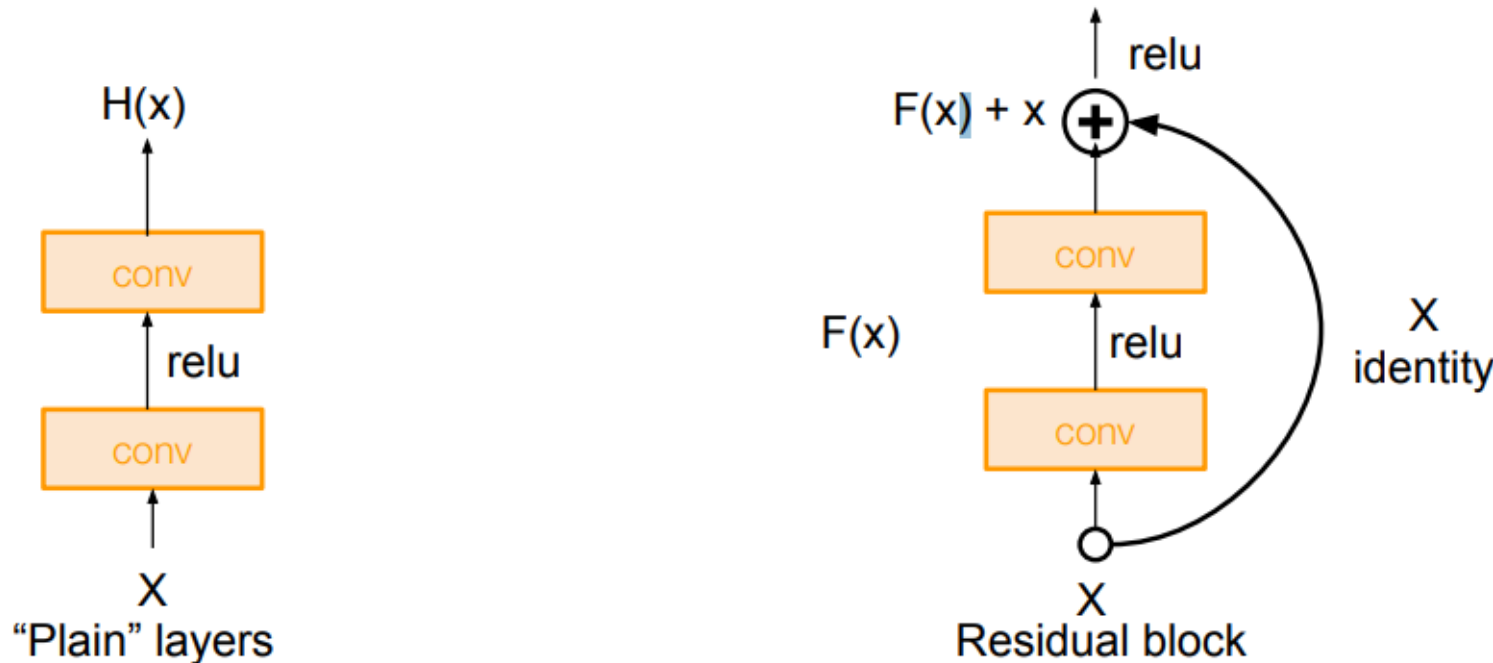
2. Deep learning models for image classification



ResNet

[He et al., 2015]

Solution: Structure each network layer to fit a “residual function” with respect to the identity function, then add the two functions together



2. Deep learning models for image classification

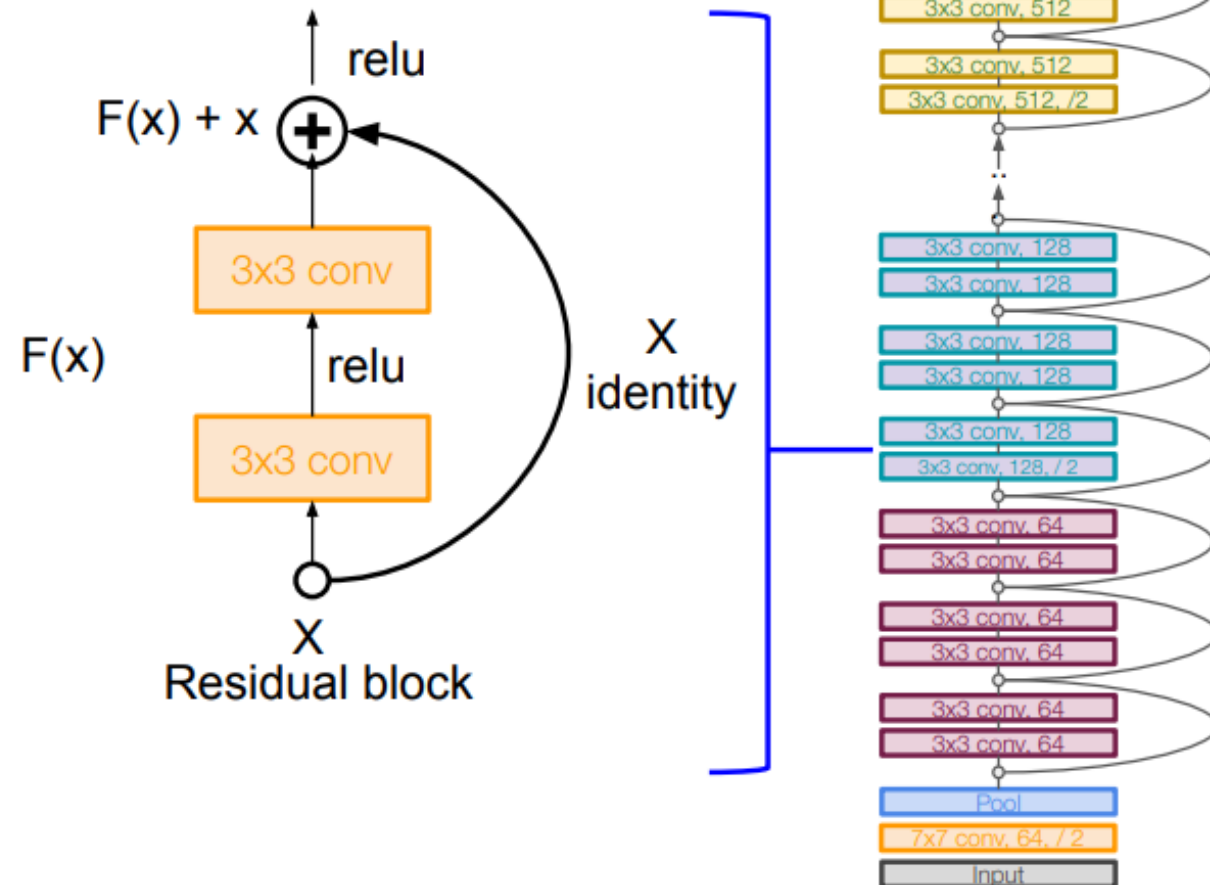


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers



2. Deep learning models for image classification

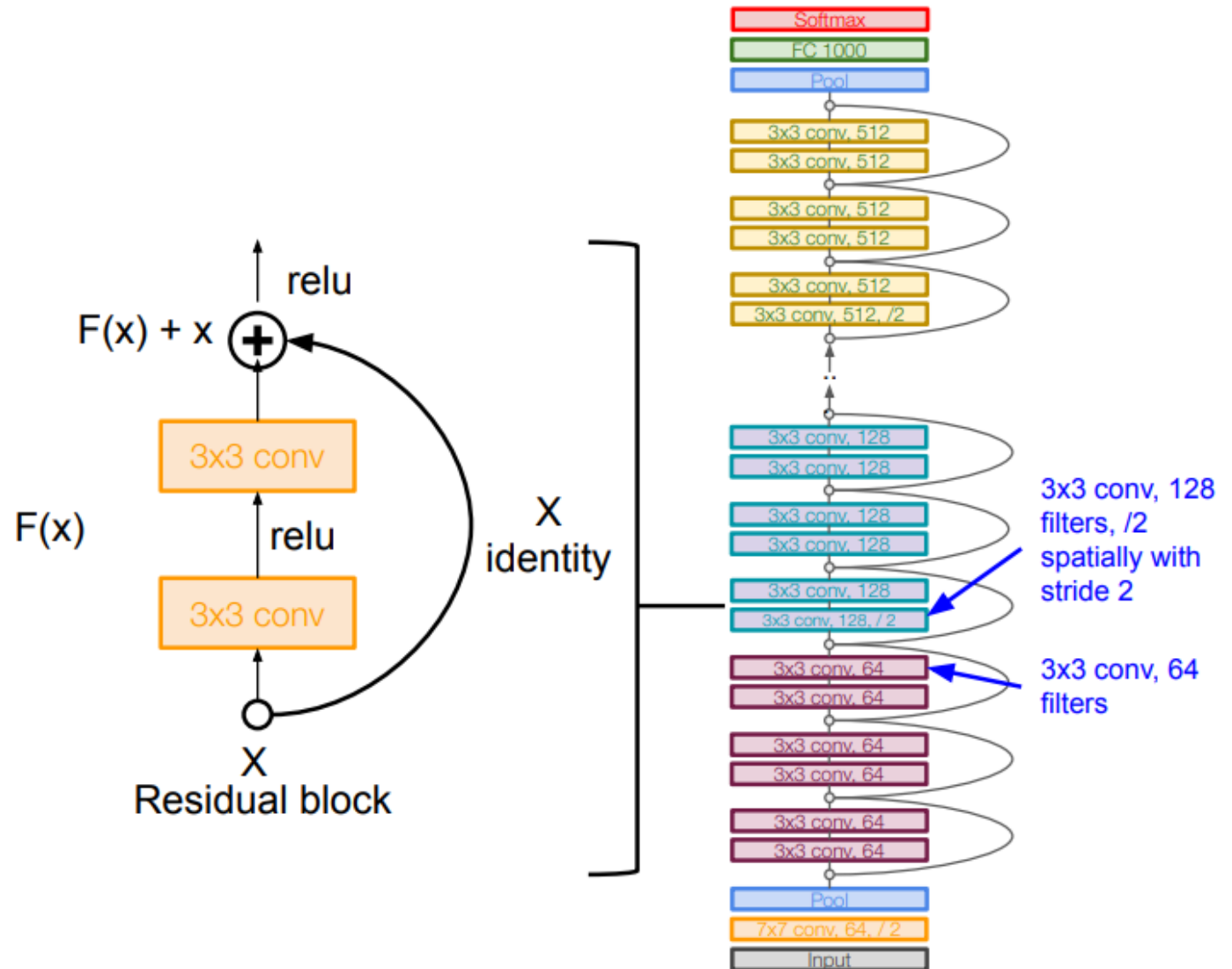


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)



2. Deep learning models for image classification

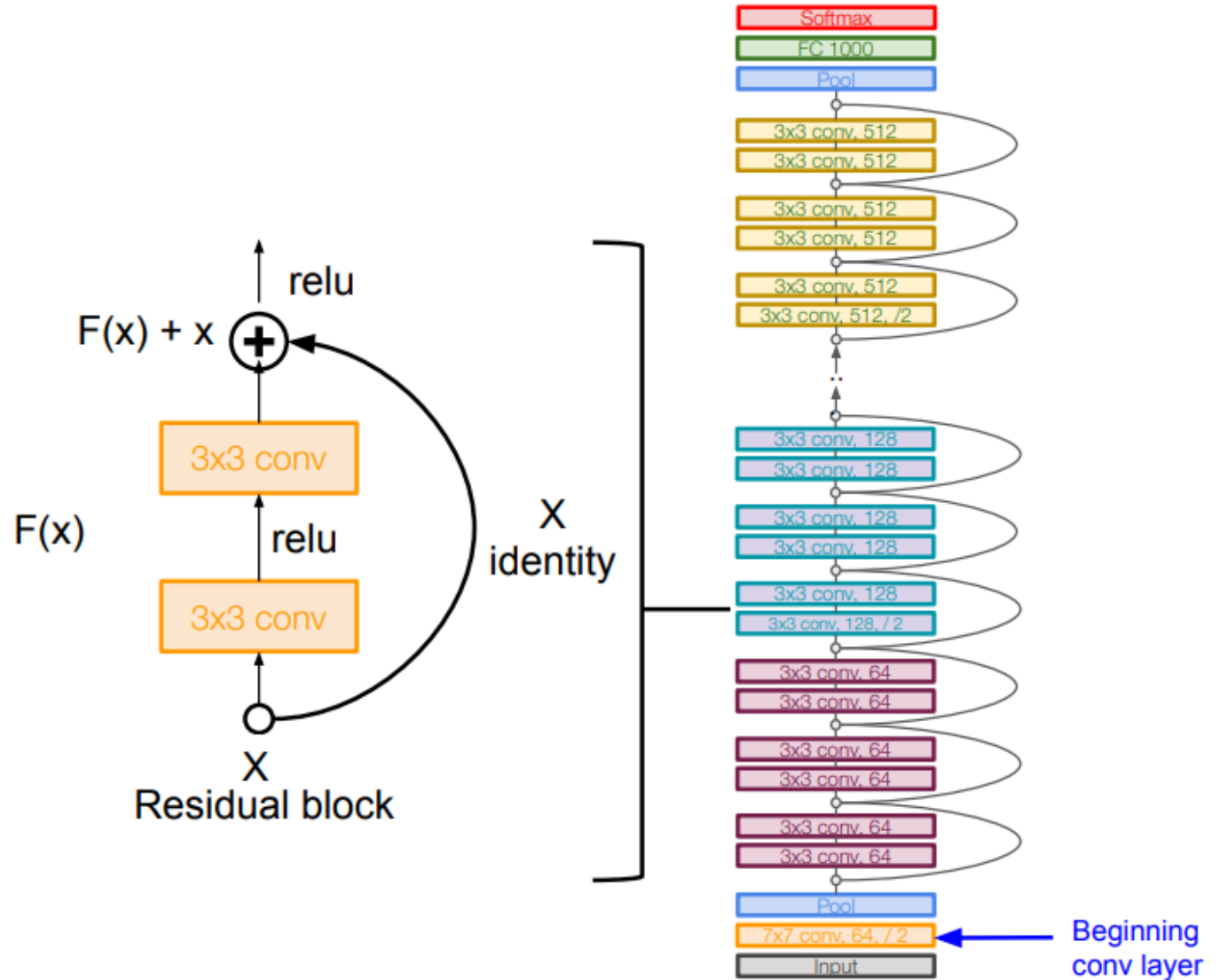


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning



2. Deep learning models for image classification

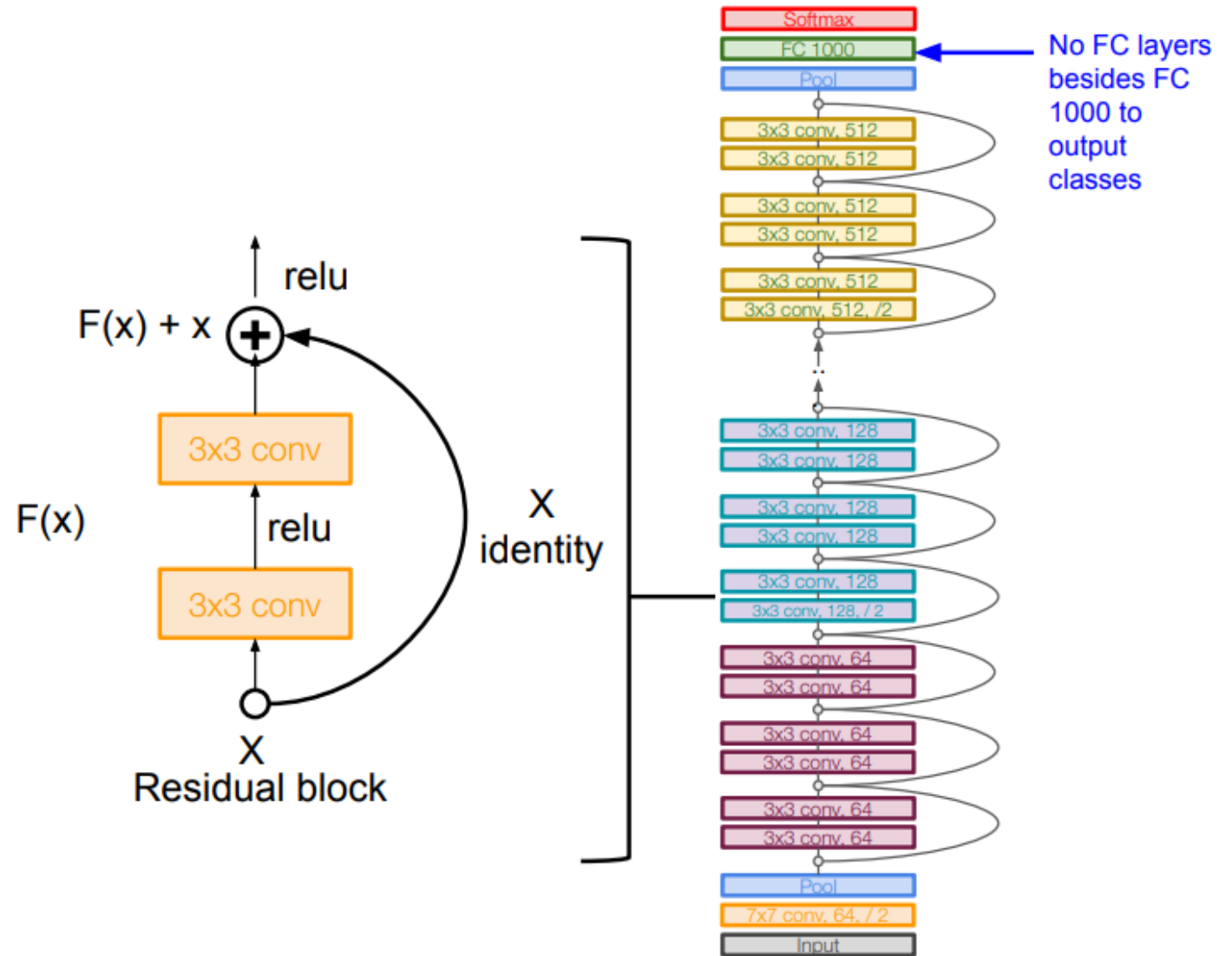


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning
- No FC layers at the end (only FC 1000 to output classes)



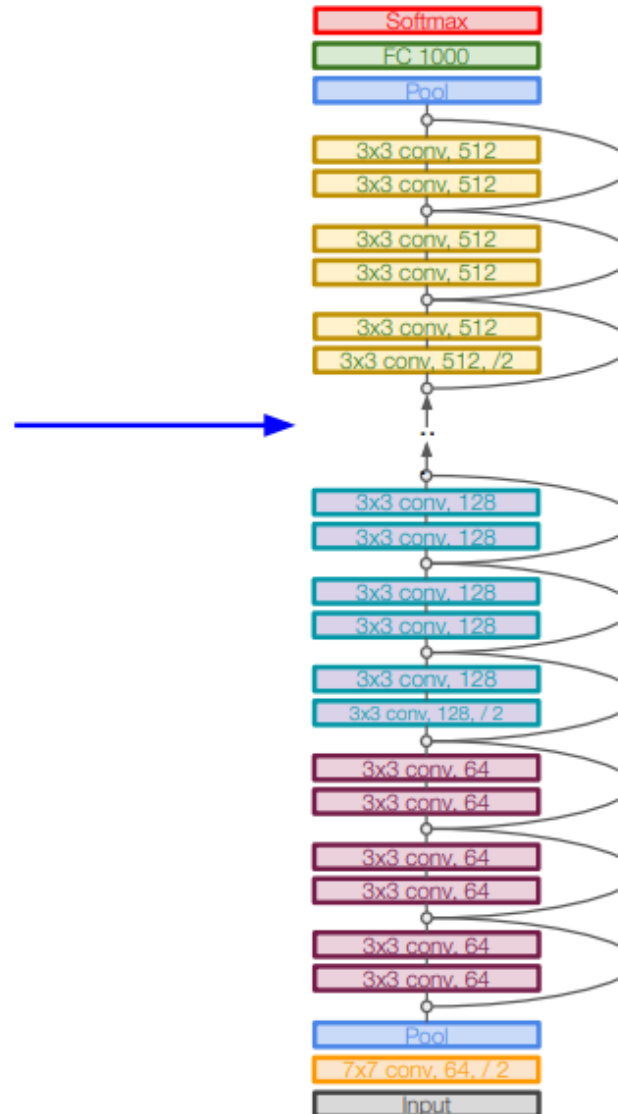
2. Deep learning models for image classification



ResNet

[He et al., 2015]

Total depths of 34, 50, 101, or 152 layers for ImageNet

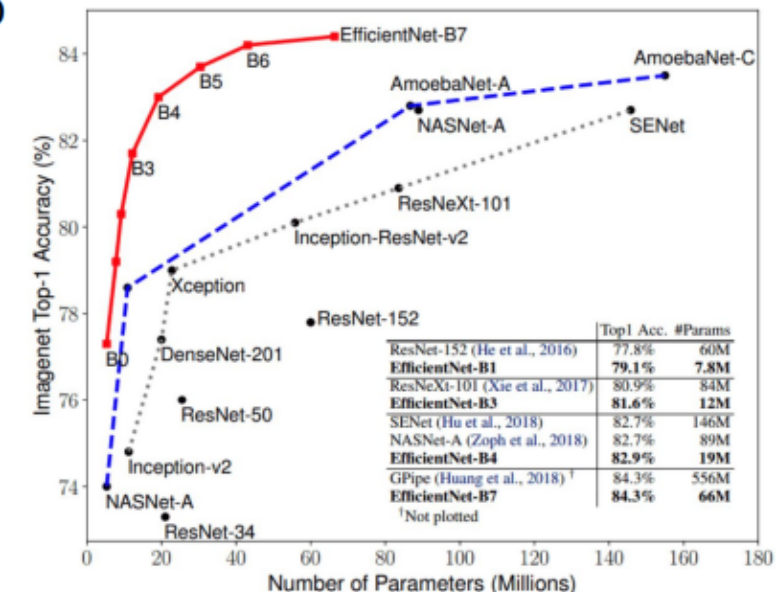


2. Deep learning models for image classification



More recent CNN architectures

- MobileNet (Sandler et al. 2018) - architecture with separable convolutions for light-weight CNNs
- NASNet (Zoph et al. 2016) and AmoebaNet (Real et al. 2019) - architectures discovered through “neural architecture search” via reinforcement learning or evolutionary algorithms
- EfficientNet (Tan et al. 2020) - family of architectures designed using “compound scaling” that simultaneously scale width, depth, and resolution of neural networks with a fixed ratio



2. Deep learning models for image classification



More recent CNN architectures

Filter: ImageNet-1k only Transformer ResNet CNN EfficientNet JFT-300M ImageNet-22k ResNeXt MLP PatchConvnet											
JFT-3B Conv+Transformer MVIT Mixer untagged Hardware Burden Operations per network pass Robustness reports											
Edit Leaderboard											
Rank	Model	Top 1 Accuracy	Top 5 Accuracy	Number of params	Extra Training Data	Paper	Code	Result	Year	Tags	
1	CoAtNet-7	90.88%		2440M	✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes			2021	Conv+Transformer	JFT-3B
2	ViT-G/14	90.45%		1843M	✓	Scaling Vision Transformers			2021	Transformer	JFT-3B
3	CoAtNet-6	90.45%		1470M	✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes			2021	Conv+Transformer	JFT-3B
4	ViT-MoE-15B (Every-2)	90.35%		14700M	✓	Scaling Vision with Sparse Mixture of Experts			2021	Transformer	JFT-3B
5	Meta Pseudo Labels (EfficientNet-L2)	90.2%	98.8%	480M	✓	Meta Pseudo Labels			2021	EfficientNet	JFT-300M
6	SwinV2-G	90.17%			✓	Swin Transformer V2: Scaling Up Capacity and Resolution			2021	Transformer	
7	Florence-CoSwin-H	90.05%	99.02%		✓	Florence: A New Foundation Model for Computer Vision			2021	Transformer	
8	Meta Pseudo Labels (EfficientNet-Rx-Wide)	90%	98.7%	390M	✓	Meta Pseudo Labels			2021	EfficientNet	JFT-300M

Check [website](#) for state-of-the-art CNN architectures for image classification

EfficientNet rank 13th in the leaderboard.

Worth exploring for class projects!

2. Deep learning models for image classification



More on loss functions

Common loss functions

Regression

Minimize squared difference between prediction output and target

$$L_{\text{regression}} = \frac{1}{M} \sum_i (\hat{y}^i - y^i)^2$$

Label is a continuous value.

Softmax

Negative log of the probability of the true class y_i , as with the BCE loss.

$$L_{\text{Softmax}} = \frac{1}{M} \sum_i -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Label is 1 of K classes in $\{0, \dots, K\}$. Extension of binary cross-entropy loss to multiple classes. s_j corresponds to the score (e.g. output of final layer) for each class; the fraction in the log provides a normalized probability for each class.

Slide credit: BIODS 220

Binary Cross-Entropy

Equivalent to the negative log of the probability of the correct ground truth class being predicted. Think about what the expression looks like when $y_i = 1$ vs. 0.

$$L_{\text{BCE}} = \frac{1}{M} \sum_i -(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Label is binary in $\{0, 1\}$. Prediction is a real number in $(0, 1)$ and is the probability of the label being 1. It is usually the output of a sigmoid operation after the final layer.

SVM

Incurs lowest loss of 0 (what we want) if the score for the true class y_i is greater than the score for each incorrect class j by a margin of 1

$$L_{\text{SVM}} = \frac{1}{M} \sum_i \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Label is 1 of K classes in $\{0, \dots, K\}$. Same use case as softmax, but different way of encouraging the model to produce outputs that we “like”. In practice, softmax is more popular and provides a nice probabilistic interpretation.

2. Deep learning models for image classification



Common loss functions

You will find these in tensorflow!

In Keras:

mean_squared_error

```
keras.losses.mean_squared_error(y_true, y_pred)
```

→ Mean squared error (MSE) is another name for regression loss

categorical_crossentropy

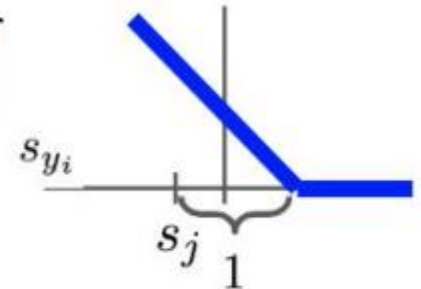
```
keras.losses.categorical_crossentropy(y_true, y_pred, from_logits=False, label_smoothing=0)
```

→ Covers both BCE and Softmax loss (remember softmax is a multiclass extension of BCE)

hinge

```
keras.losses.hinge(y_true, y_pred)
```

→ Hinge is another name for SVM loss, due to the loss function shape.



<https://keras.io/losses/>

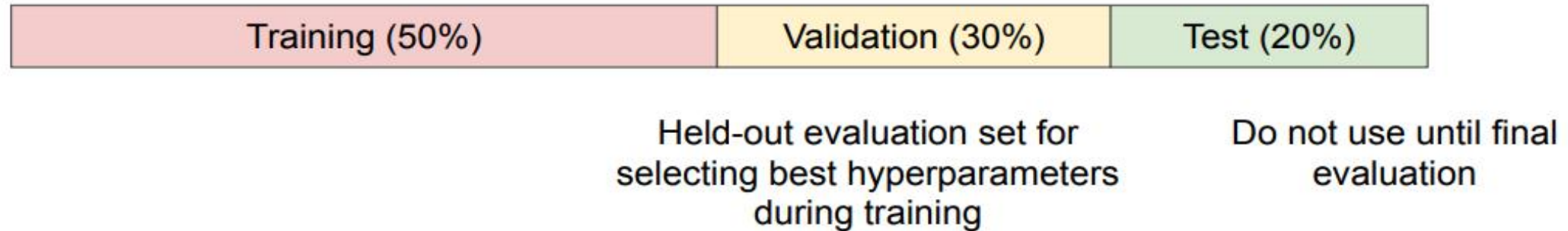


- Review what we have learned last time
- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies of CNNs for medical image classification

3. Data considerations for image classification models



Training, validation, and test sets



Other splits, e.g., 60/20/20 also popular.
Balance sufficient data for training vs. informative performance estimate on validation / testing.

3. Data considerations for image classification models



Maximizing training data for the final model



Once hyperparameters are selected using the validation set, common to merge training and validation sets into a larger "trainval" set to train a final model using the hyperparameters.

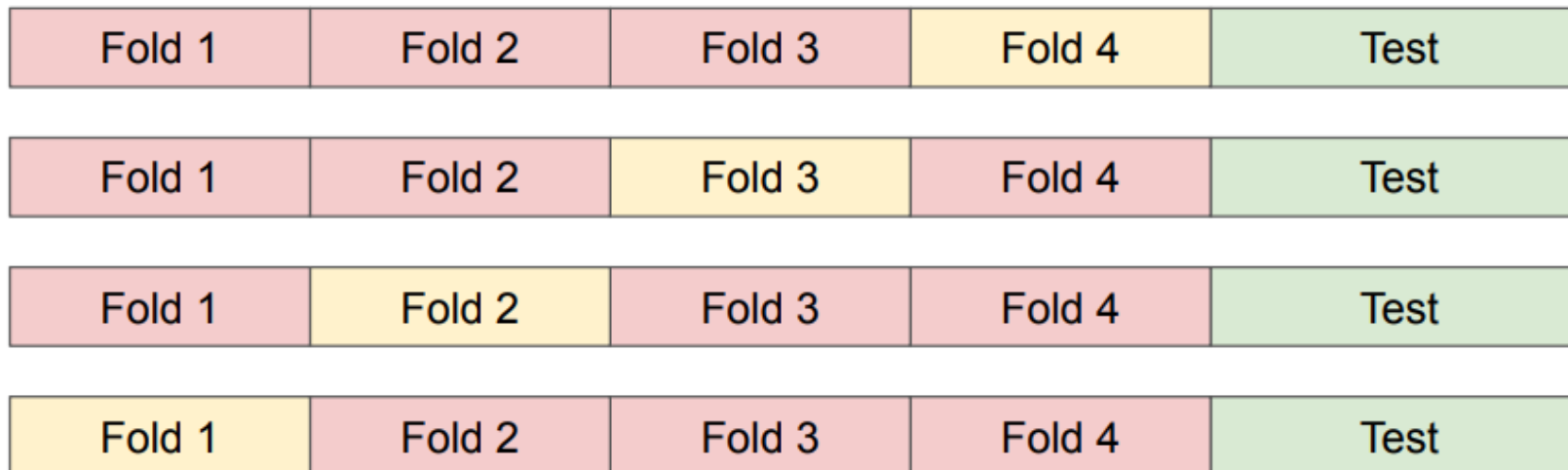
OK, since we can use non-test data however, we want during model development!

3. Data considerations for image classification models



K-fold cross validation: for small datasets

Sometimes we have small labeled datasets in healthcare... in this case K-fold cross validation (which is more computationally expensive) may be worthwhile.



Train model K times with a different fold as the validation set each time; then average the validation set results. Allows more data to be used for each training of the model, while still using enough data to get accurate validation result.

Can also apply same concept to test-time evaluation.

3. Data considerations for image classification models



Data preprocessing

Min-max scaling:

$$x_{\text{scaled}} = (x_{\text{orig}} - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

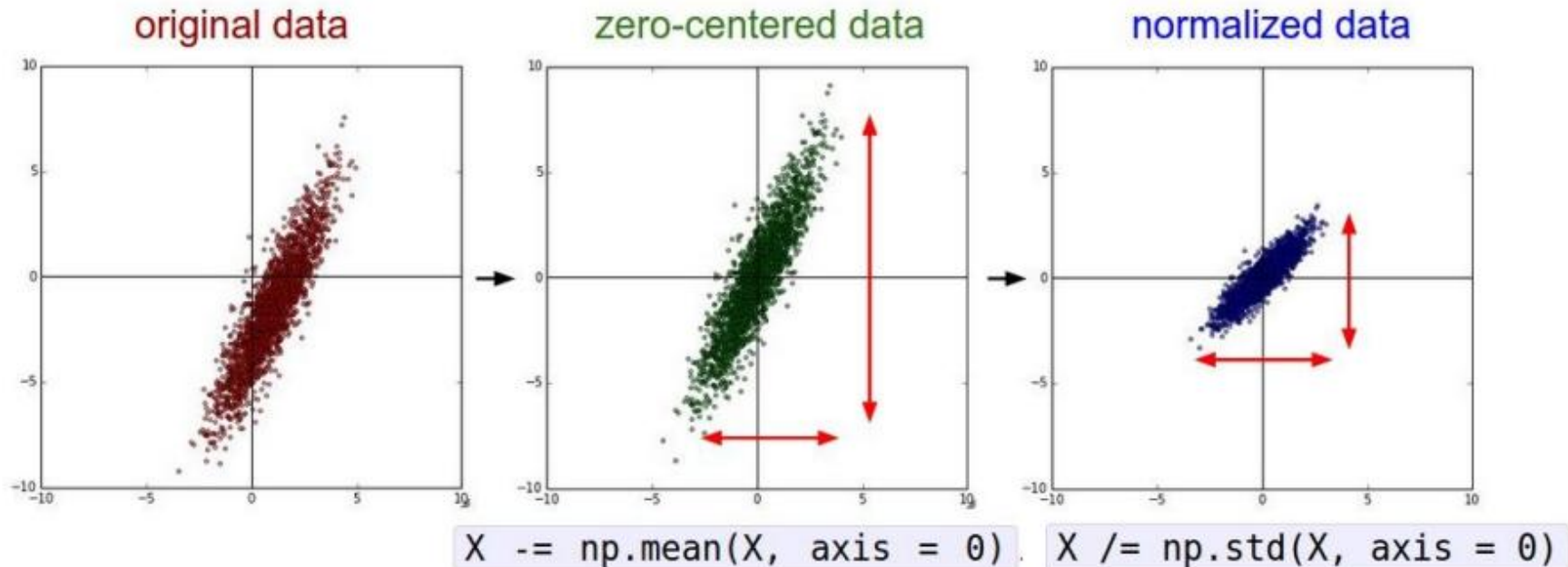
where x_{min} and x_{max} are min and max values in the original data

- Maps original range of data to [0,1] range
- Neural networks generally expect small numbers as input (not too extreme relative to scale of initialized weights)

3. Data considerations for image classification models

Data preprocessing

Common to also normalize mean and variance of features, such that features are treated equally. Most common: make all features zero-mean, unit variance.



3. Data considerations for image classification models



Data preprocessing: for images

For images, common to perform simpler normalization:

e.g. consider a dataset with [32,32,3] images

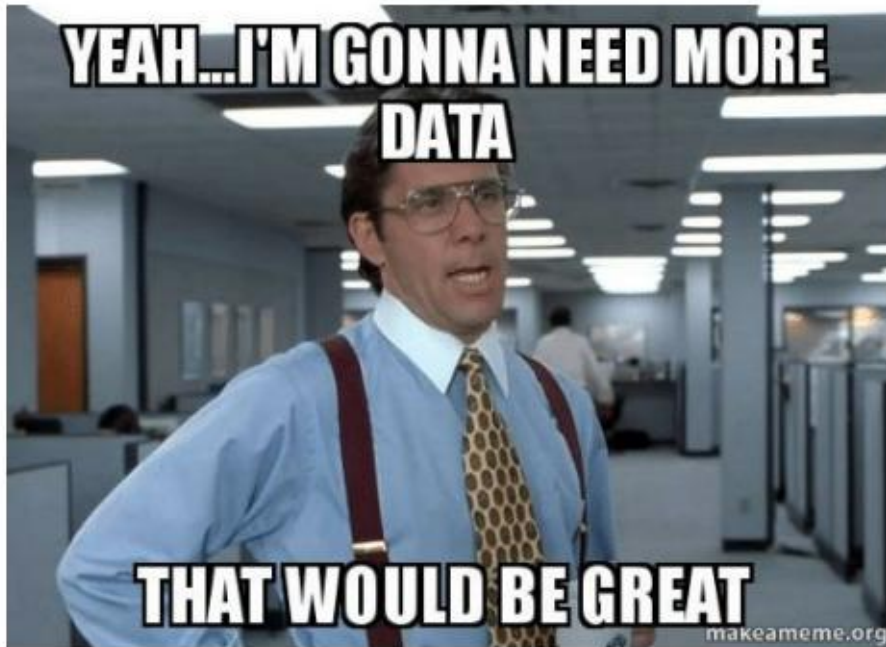
- Subtract the mean image (used in original AlexNet model)
(mean image = [32,32,3] array)
- Subtract per-channel mean (used in original VGG model)
(mean along each channel = 3 numbers)
- Subtract per-channel mean and
divide by per-channel std (used in original ResNet model)
(mean along each channel = 3 numbers)

3. Data considerations for image classification models



How much data do you need for deep learning?

A: A lot.



Premise of deep learning uses many parameters (e.g. millions) to fit complex functions

-> if the dataset is too small, easiest solution that model ends up learning can be overfitting to memorizing the labels of the training examples

ImageNet dataset consists of 1M images: 1000 classes with 1000 images each

3. Data considerations for image classification models

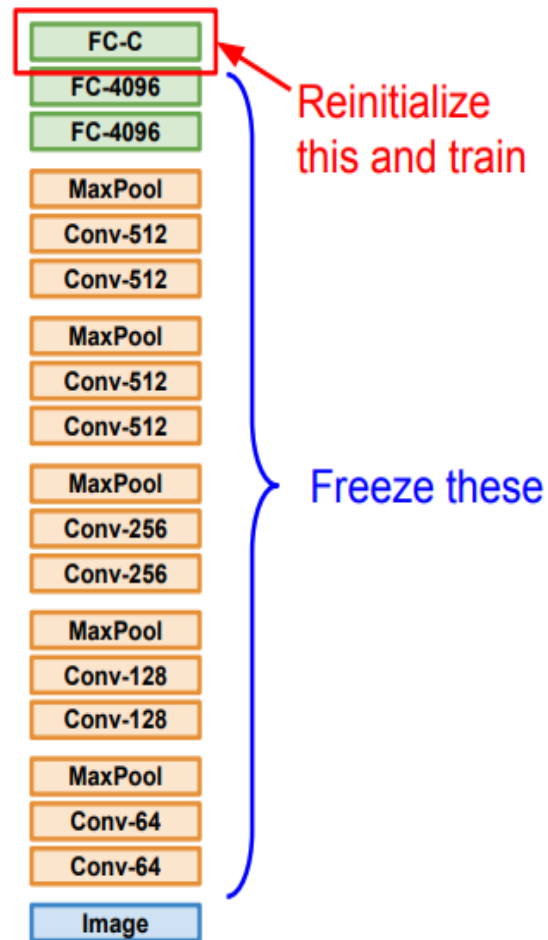


Transfer learning: amplifying training data

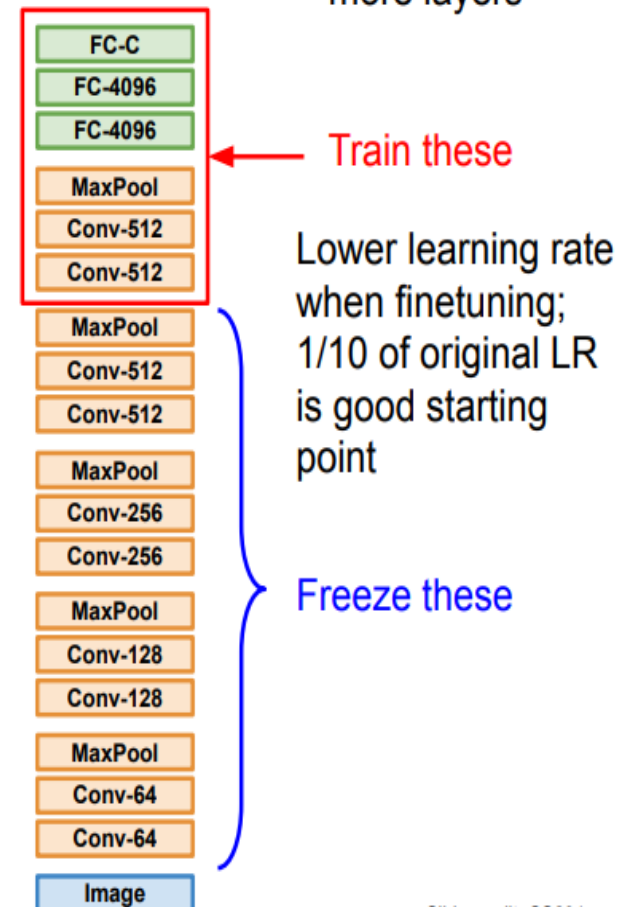
1. Train on big dataset
(e.g. ImageNet)



2. Small Dataset (C classes)

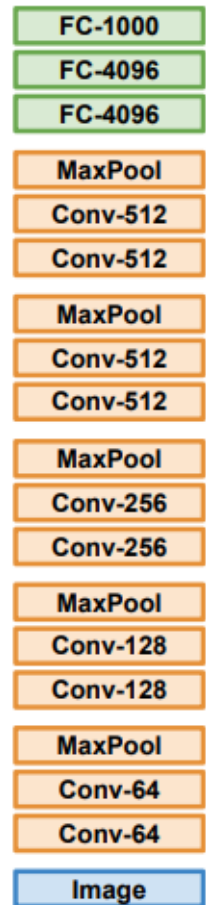


3. Bigger dataset



With bigger
dataset, train
more layers

3. Data considerations for image classification models



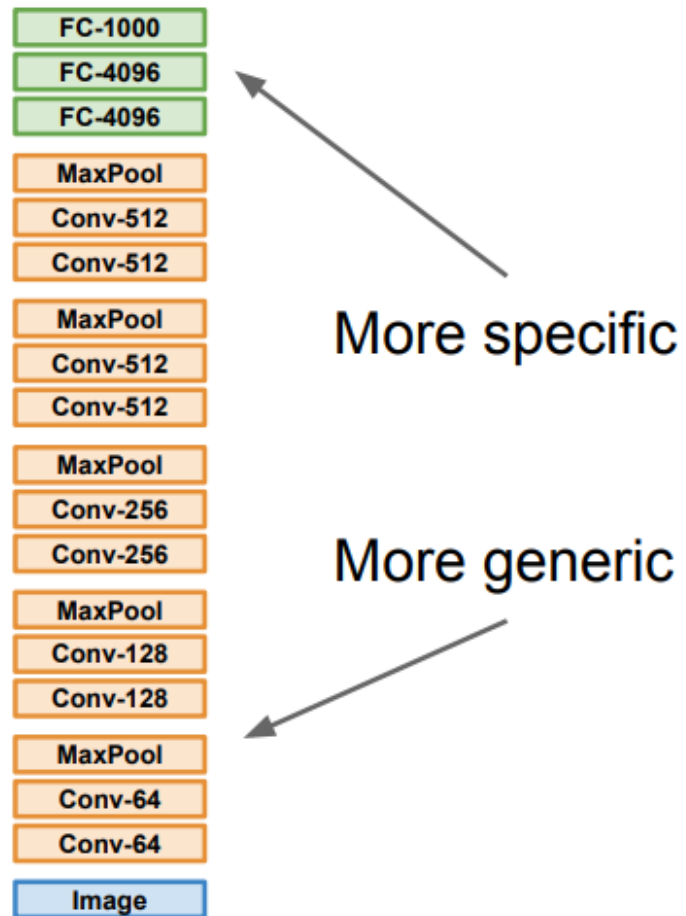
More specific

More generic

Transfer learning from a large dataset to your dataset...

	very similar dataset	very different dataset
very little data		
quite a lot of data		

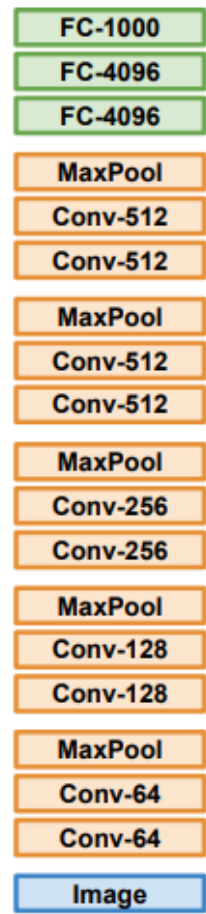
3. Data considerations for image classification models



Transfer learning from a large dataset to your dataset...

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer features	
quite a lot of data		

3. Data considerations for image classification models



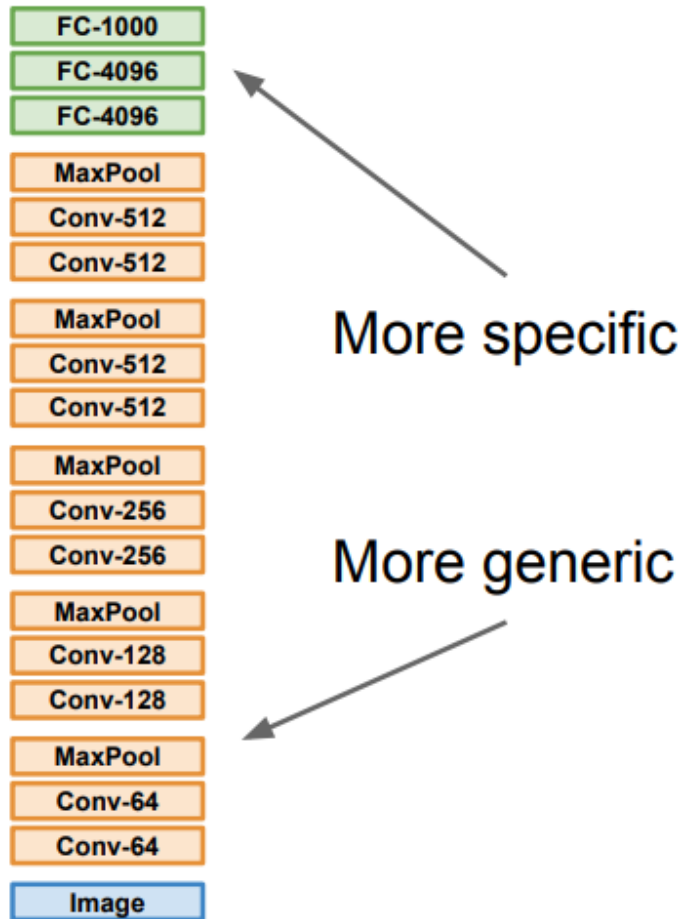
More specific

More generic

Transfer learning from a large dataset to your dataset...

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer features	You're in trouble... Try linear classifier on different layer features
quite a lot of data		

3. Data considerations for image classification models



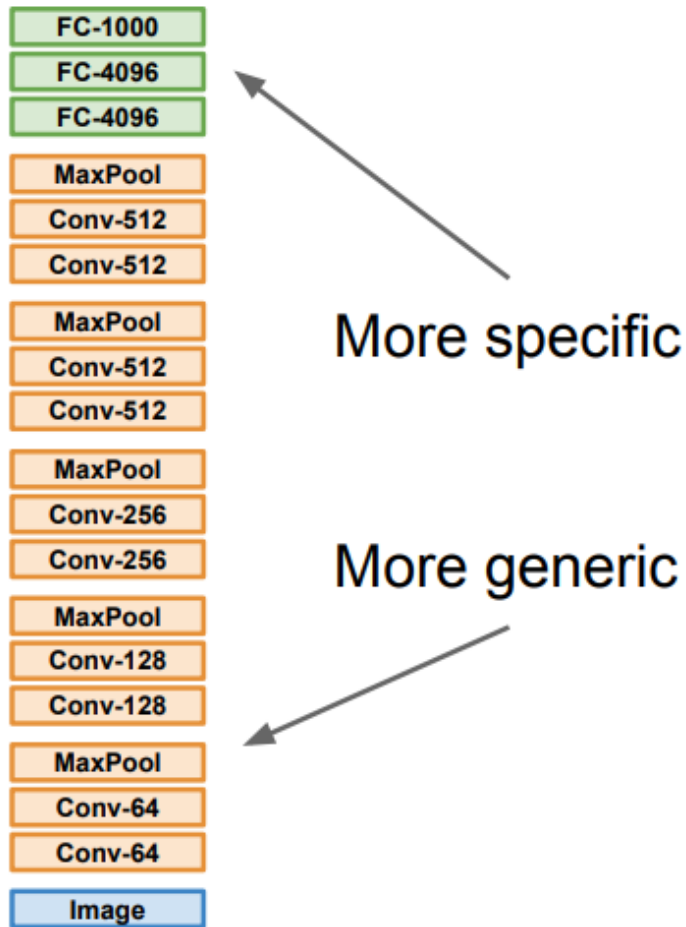
Transfer learning from a large dataset to your dataset...

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer features	You're in trouble... Try linear classifier on different layer features
quite a lot of data	Finetune a few layers	

3. Data considerations for image classification models



Transfer learning from a large dataset to your dataset...



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer features	You're in trouble... Try linear classifier on different layer features
quite a lot of data	Finetune a few layers	Finetune a large number of layers

Often good idea to try this first, try fine-tuning all layers of the network

3. Data considerations for image classification models



How much data do you need for deep learning?

Examples per class of your dataset, in addition to transfer learning (take this with grain of salt, it really depends on the problem):

- Low dozens: generally, too small to learn a meaningful model, using standard supervised deep learning
- High dozens to low hundreds: may see models with some predictive ability, unlikely to really wow or be “superhuman” though
- High hundreds to thousands: “happy regime” for deep learning

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer features	You're in trouble... Try linear classifier on different layer features
quite a lot of data	Finetune a few layers	Finetune a large number of layers

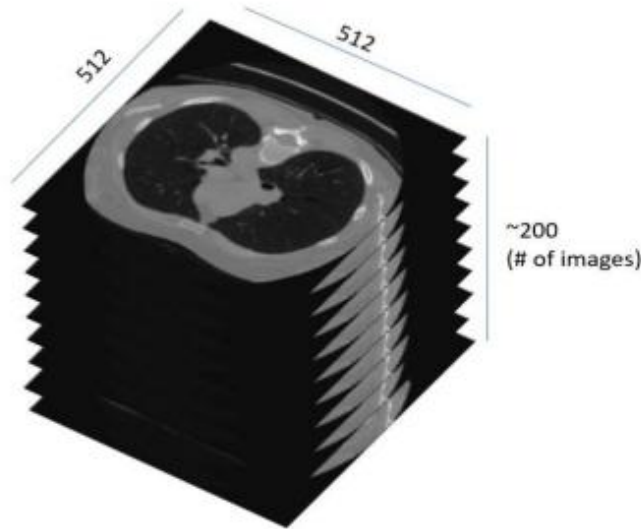
In general, deep learning is data hungry
-- the more data the better

Almost always leverage transfer learning unless you have extremely different or huge (e.g., ImageNet-scale) dataset

3. Data considerations for image classification models



What counts as a data example?



1 3D CT volume with 200 slices $\neq$ 200 data examples



5 surgery videos with thousands of frames each $\neq$ thousands of data examples

Guidelines for amount of training data refers to **# of unique instances representative** of diversity expected during testing / deployment. **E.g. # of independent CT scans or surgery videos.** Additional correlated data (e.g. different slices of the same tumor or different suturing instances within the same video) provide relatively less incremental value in comparison.

3. Data considerations for image classification models



Preview: advanced approaches for handling limited labeled data

Semi-supervised learning

Weakly-supervised learning

Domain adaptation

Will talk more about these in later lectures...

3. Data considerations for image classification models



What if there are multiple possible sources of data?

E.g., some with noisier / less accurate labels than others, from different hospital sites, etc.

- ✓ Expected diversity of data during deployment should be reflected in both training and test sets
 - ✓ Need to see these during training to learn how to handle them
 - ✓ Need to see these during testing to accurately evaluate the model

4. Evaluating image classification models



- Review what we have learned last time
- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies of CNNs for medical image classification

4. Evaluating image classification models



Evaluation metrics

Confusion matrix

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Accuracy: $(TP + TN) / \text{total}$

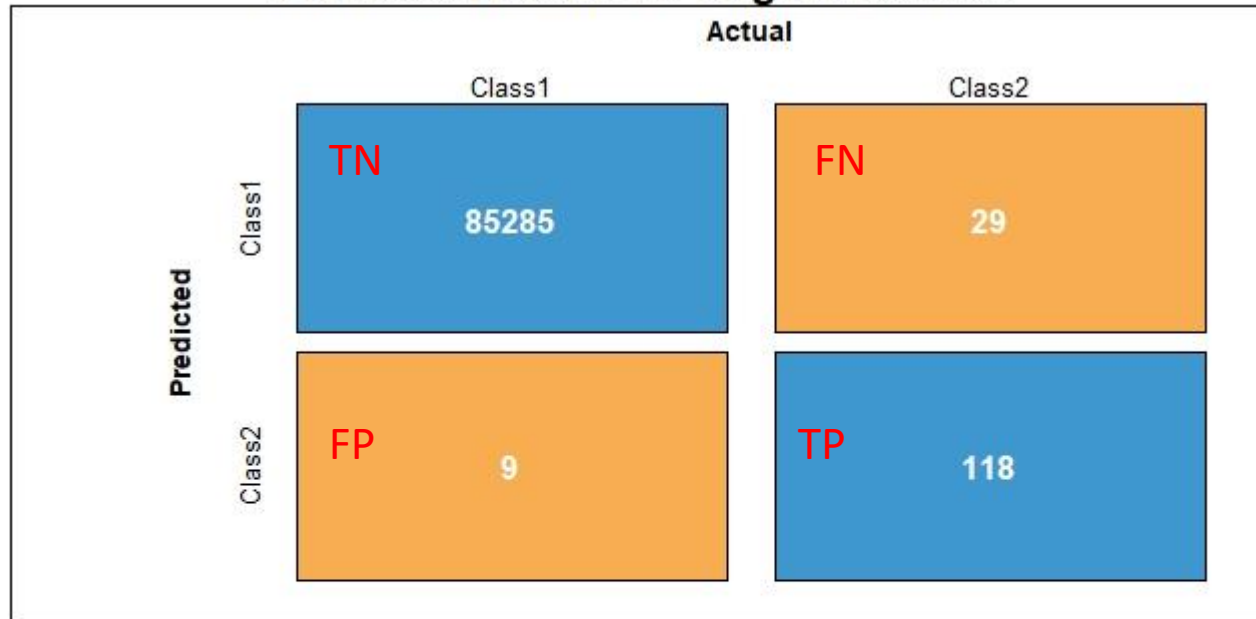
Q: When might evaluating purely accuracy be problematic?

A: Imbalanced datasets.

4. Evaluating image classification models



Confusion Matrix for original dataset



DETAILS

Sensitivity 0.803	Specificity 1	Precision 0.929	Recall 0.803	F1 0.861
Accuracy 1		Kappa 0.861		

$$\text{Accuracy} = (85285 + 118) / (85285 + 118 + 29 + 9) = 1$$

4. Evaluating image classification models



Evaluation metrics

We can trade-off different values of these metrics as we vary our classifier's score threshold to predict a positive

Confusion matrix

		Prediction	
		0	1
Ground Truth	0	TN	FP
	1	FN	TP

Accuracy: $(TP + TN) / \text{total}$

Sensitivity / Recall (true positive rate):
 $TP / \text{total positives}$

Specificity (true negative rate):
 $TN / \text{total negatives}$

Precision (positive predictive value):
 $TP / \text{total predicted positives}$

Negative predictive value:
 $TN / \text{total predicted negatives}$

4. Evaluating image classification models



Evaluation metrics

Q: As prediction threshold increases, how does that generally affect sensitivity? Specificity?

Confusion matrix

		Prediction	
		0	1
Ground Truth	0	TN	FP
	1	FN	TP

Accuracy: $(TP + TN) / \text{total}$

Sensitivity / Recall (true positive rate):
 $TP / \text{total positives}$

Specificity (true negative rate):
 $TN / \text{total negatives}$

Precision (positive predictive value):
 $TP / \text{total predicted positives}$

Negative predictive value:
 $TN / \text{total predicted negatives}$

4. Evaluating image classification models



Evaluation metrics

Confusion matrix

		Prediction	
		0	1
Ground Truth	0	TN	FP
	1	FN	TP

Q: As prediction threshold increases, how does that generally affect sensitivity? Specificity?

A: Sensitivity goes down, specificity up

Accuracy: $(TP + TN) / \text{total}$

Sensitivity / Recall (true positive rate):
 $TP / \text{total positives}$

Specificity (true negative rate):
 $TN / \text{total negatives}$

Precision (positive predictive value):
 $TP / \text{total predicted positives}$

Negative predictive value:
 $TN / \text{total predicted negatives}$

4. Evaluating image classification models



Confusion Matrix for original dataset

		Actual	
		Class1	Class2
Predicted	Class1	TN 85285	FN 29
	Class2	FP 9	TP 118

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{118}{118 + 9} = 0.929$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{118}{118 + 29} = 0.803 \quad \text{aka Recall}$$

$$\text{F1 score} = \frac{2 * (\text{Recall} * \text{Precision})}{(\text{Recall} + \text{Precision})} = 0.861$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{85285}{85285 + 9} = 1$$

DETAILS				
Sensitivity 0.803	Specificity 1	Precision 0.929	Recall 0.803	F1 0.861
Accuracy 1		Kappa 0.861		

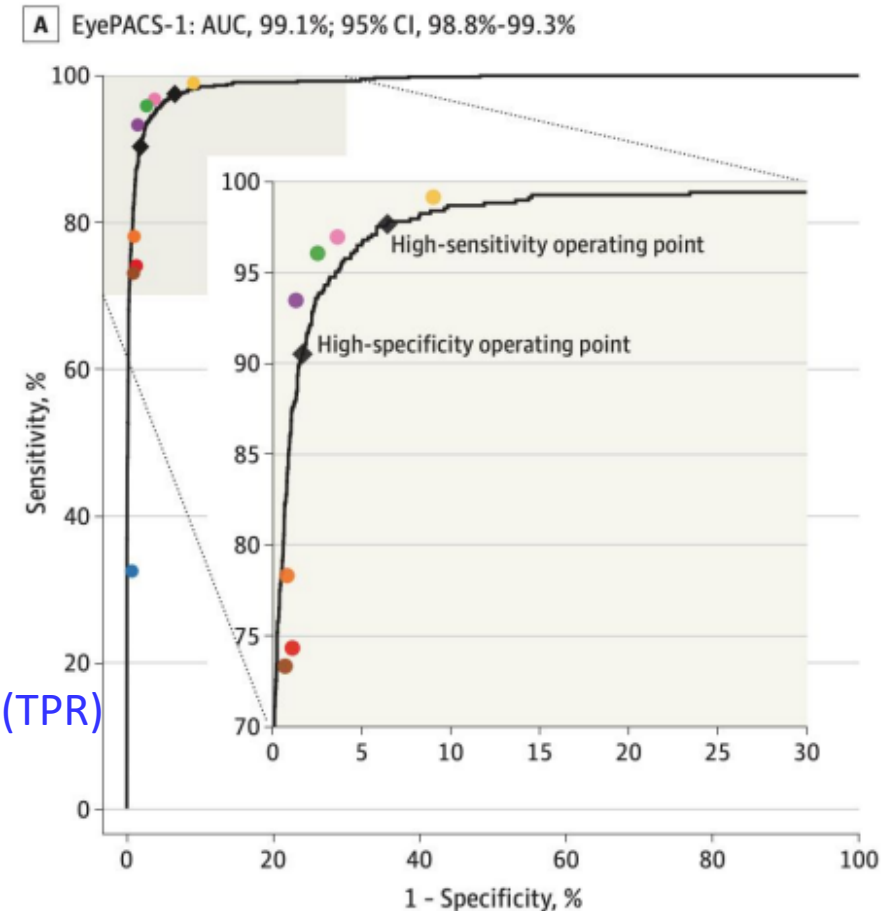
4. Evaluating image classification models



Evaluation metrics

- **Receiver Operating Characteristic (ROC) curve:**
 - Plots sensitivity and specificity (specifically, 1 - specificity) as prediction threshold is varied
 - Gives trade-off between sensitivity and specificity
 - Also report summary statistic AUC (area under the curve)

True Positive Rate (TPR)



False Positive Rate (FPR)

知乎 @visionshao



4. Evaluating image classification models



Evaluation metrics

- Sometimes also see **precision recall curve**
 - More informative when dataset is heavily imbalanced (sensitivity = true negative rate less meaningful in this case)

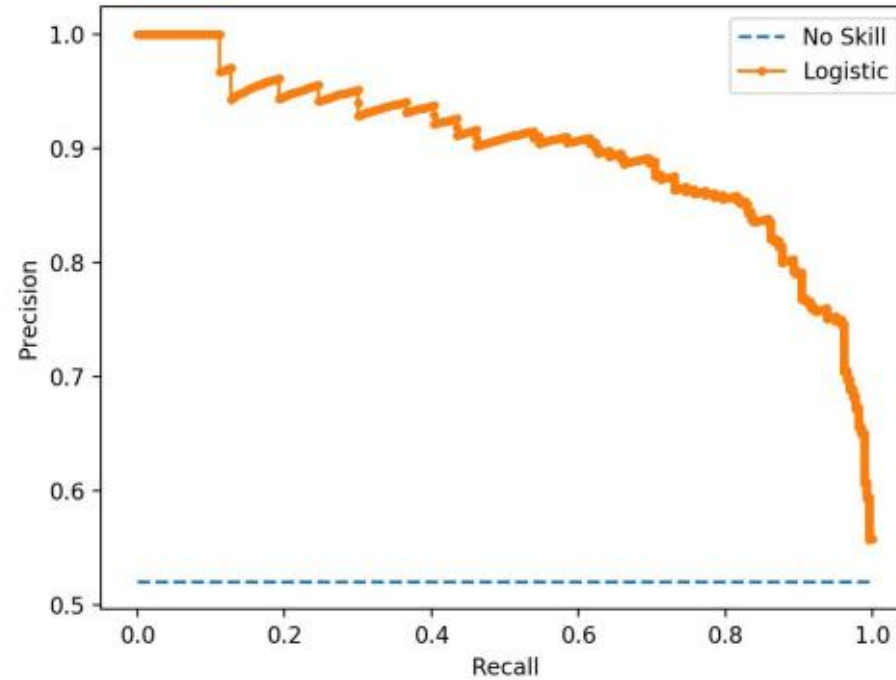


Figure credit: <https://3qepr26caki16dnhd19sv6by6v-wpengine.netdna-ssl.com/wp-content/uploads/2018/08/Precision-Recall-Plot-for-a-No-Skill-Classifer-and-a-Logistic-Regression-Model4.png>

4. Evaluating image classification models



Evaluation metrics

- Selecting optimal trade-off points
 - Maximize **Youden's Index**
 - $J = \text{sensitivity} + \text{specificity} - 1$
 - Gives equal weight to optimizing true positives and true negatives

Also equal to distance above chance line for a balanced dataset: $\text{sensitivity} - (1 - \text{specificity}) = \text{sensitivity} + \text{specificity} - 1$

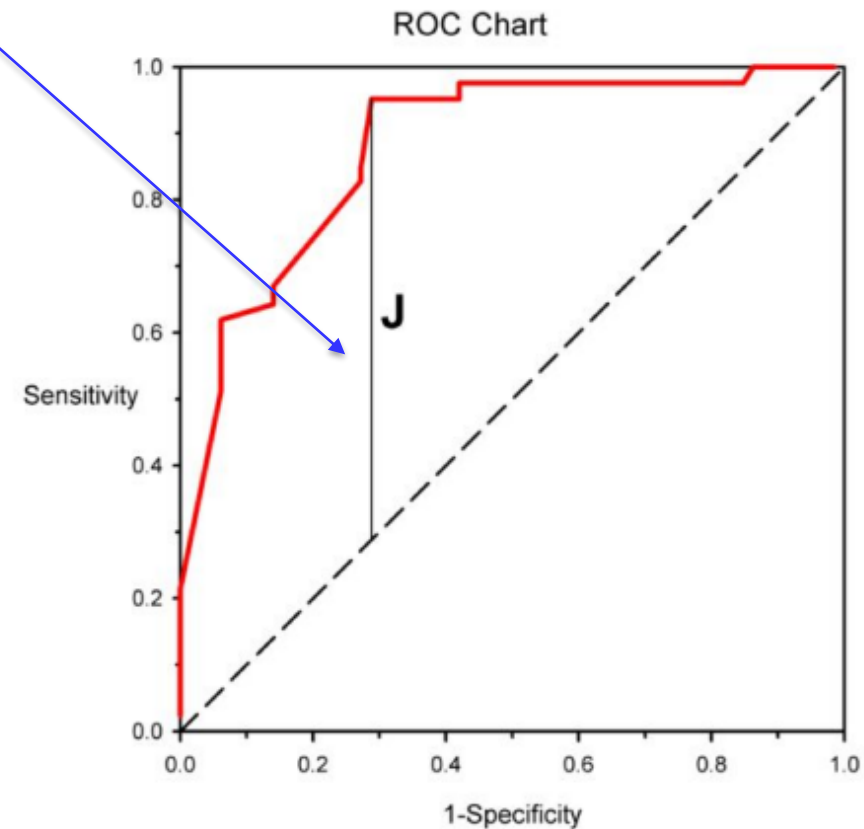


Figure credit: https://en.wikipedia.org/wiki/File:ROC_Curve_Youden_J.png

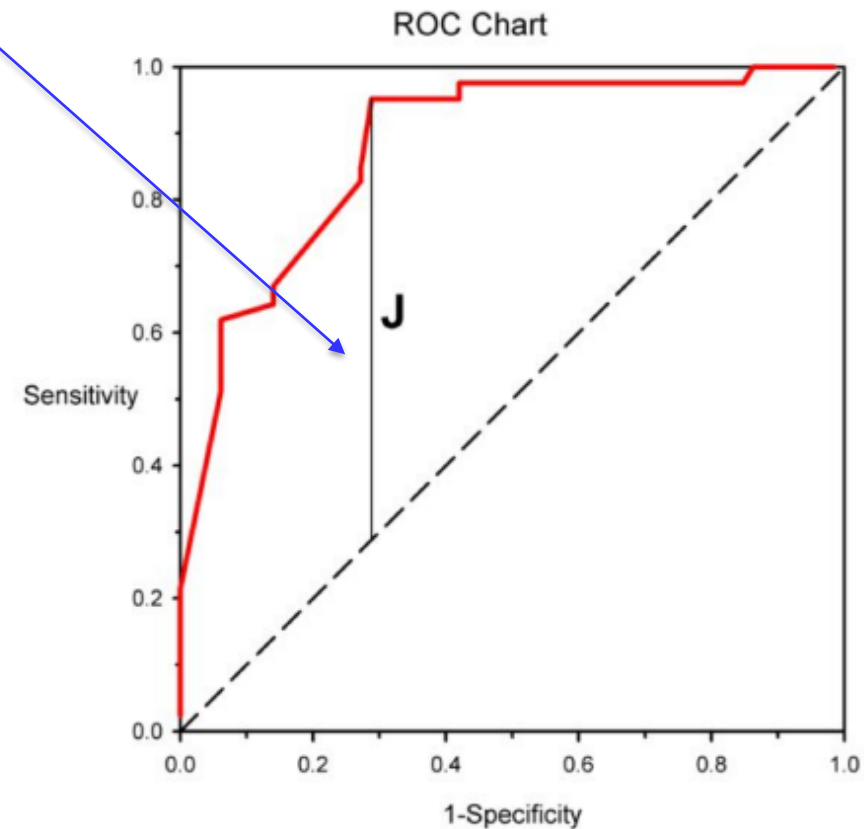
4. Evaluating image classification models



Evaluation metrics

- Selecting optimal trade-off points
 - Maximize **Youden's Index**
 - $J = \text{sensitivity} + \text{specificity} - 1$
 - Gives equal weight to optimizing true positives and true negatives
- Sometimes also see F-measure (or F1 score)
 - $F1 = \frac{2 * (\text{precision} * \text{recall})}{(\text{precision} + \text{recall})}$
 - Harmonic mean of precision and recall

Also equal to distance above chance line for a balanced dataset: $\text{sensitivity} - (1 - \text{specificity}) = \text{sensitivity} + \text{specificity} - 1$



But selected trade-off points
could also depend on application

Figure credit: https://en.wikipedia.org/wiki/File:ROC_Curve_Youden_J.png



- Review what we have learned last time
- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies of CNNs for medical image classification

5. Case studies



Joint Diabetic Retinopathy (DR) and Diabetic Macular Edema (DME) Grading

Diabetic Retinopathy (DR)

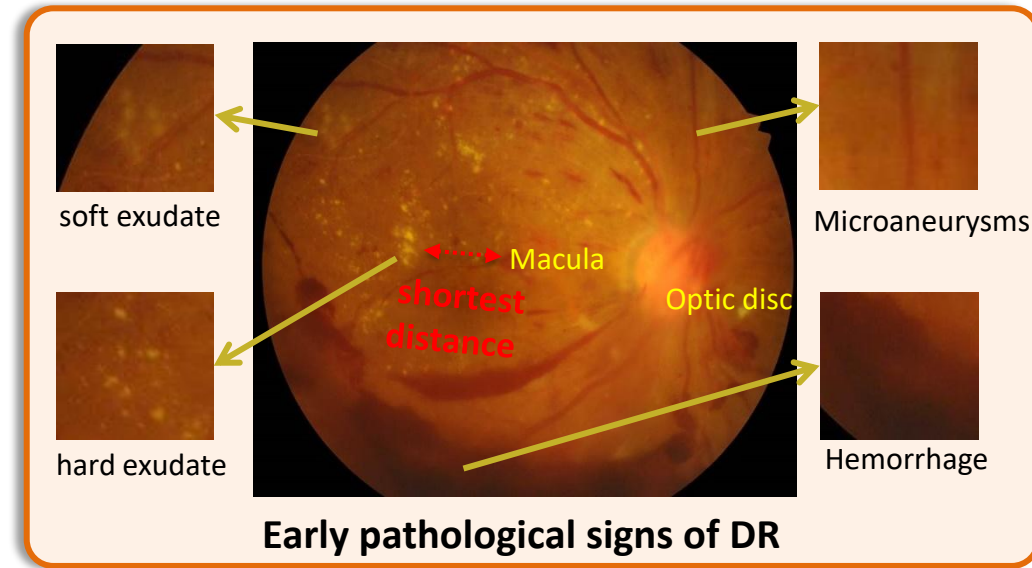
- a consequence of microvascular changes triggered by diabetes
- **leading cause** of blindness

Diabetic Macular Edema (DME)

- a **complication** of DR
- retinal thickening of fluid
- occur at any stage of DR

Grading:

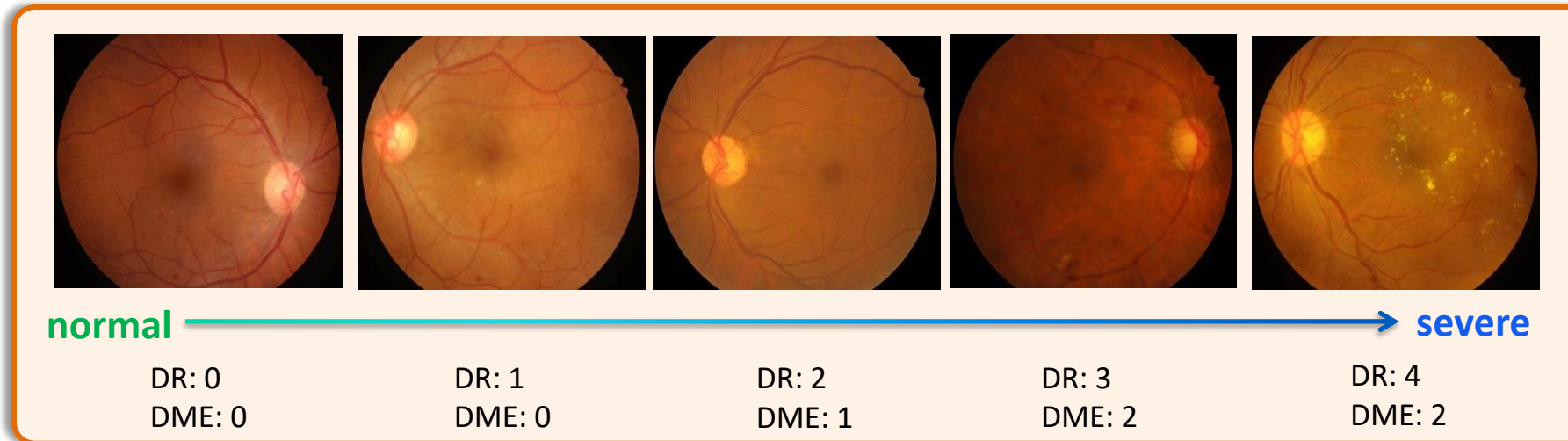
- DR: **the severity**
- DME: **shortest distance** between macula and hard exudates (0: no risk; 1: $d < 1$, 2: $d > 1$)



5. Case studies



Joint Diabetic Retinopathy (DR) and Diabetic Macular Edema (DME) Grading



Grading

- DR: **the severity**.
- DME: **shortest distance** between macula and hard exudates (0: no risk; 1: $d < 1$, 2: $d > 1$).

Clinical Importance of Grading

- DR/DME patients can receive tailored treatments.

5. Case studies



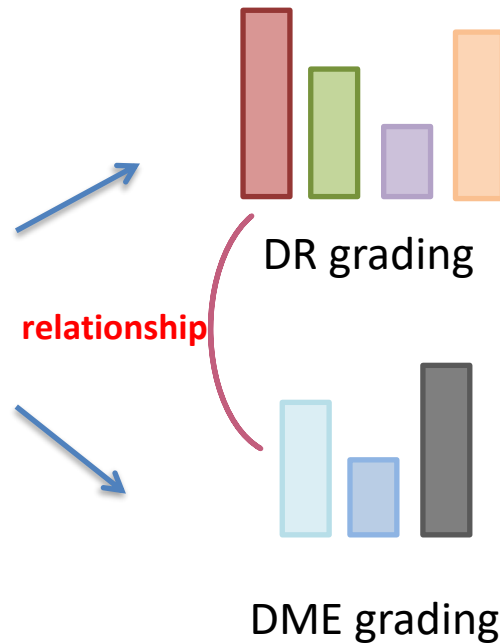
Automatically learned features for DR and DME grading



Fundus Image



Neural Network



no published works for joint DR and DME grading.

Multi-task learning

- the information among different tasks is shared
- promote the performance of each individual task

It also requires

- an understanding of each disease
- the internal relationship between the two diseases.

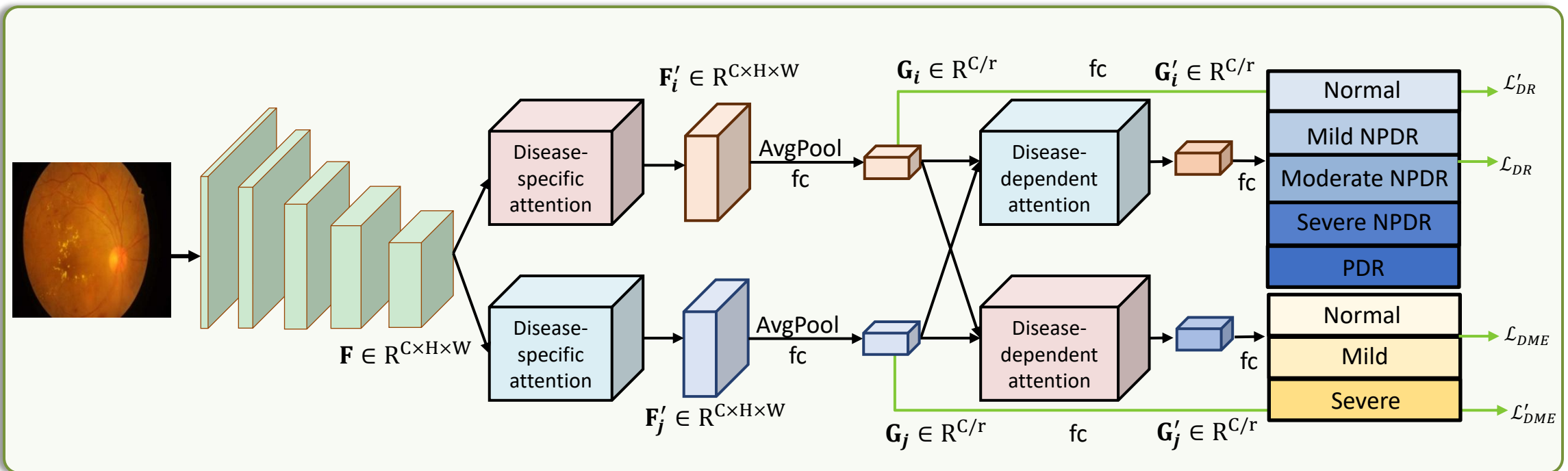
5. Case studies



Cross-disease Attention Network (CANet)

disease-specific attention block (*disease-specific features*) deep understanding of each disease

disease-dependent attention block (*disease-dependent features*) internal relationship between diseases



The overall architecture of cross-disease attention network (CANet).

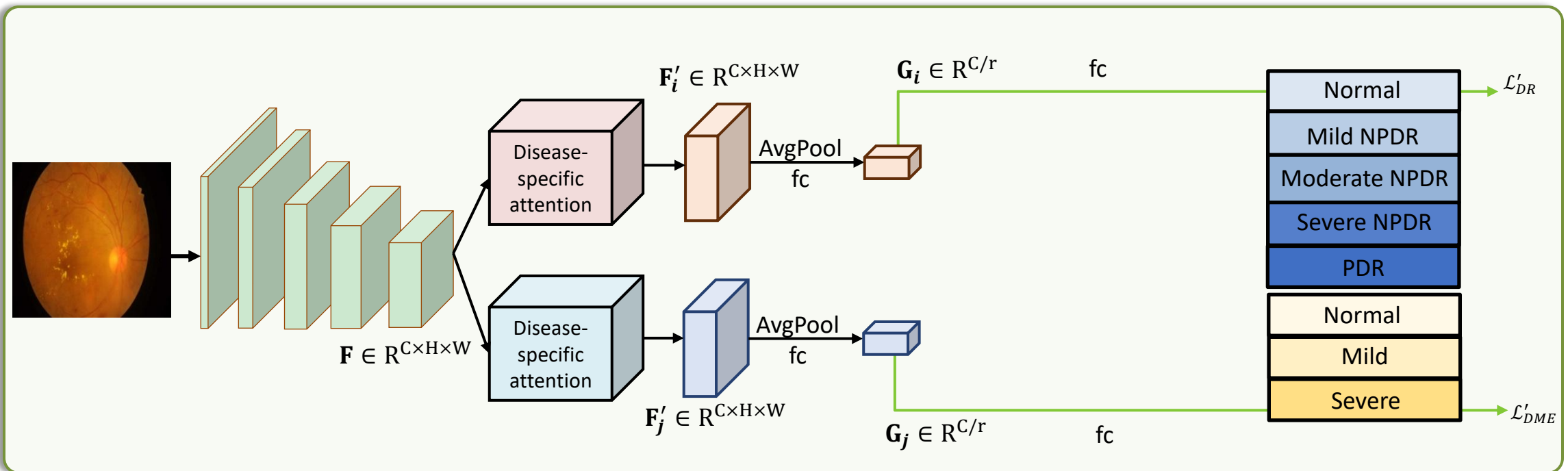
5. Case studies



Cross-disease Attention Network (CANet)

disease-specific attention block (*disease-specific features*) deep understanding of each disease

disease-dependent attention block (*disease-dependent features*) internal relationship between diseases



The overall architecture of cross-disease attention network (CANet).

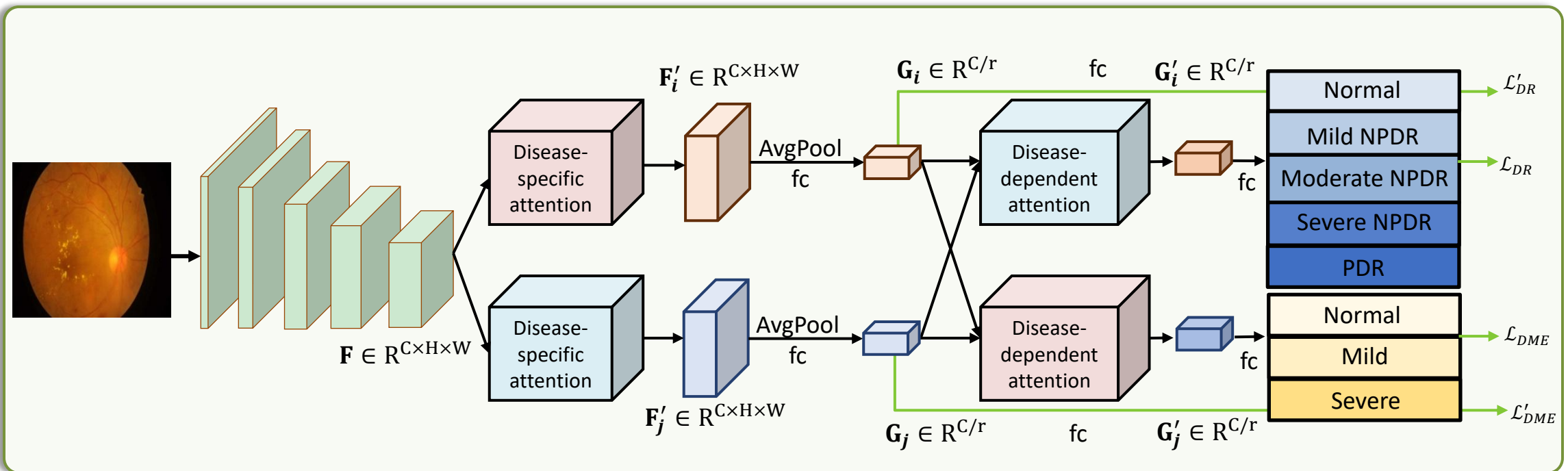
5. Case studies



Cross-disease Attention Network (CANet)

disease-specific attention block (*disease-specific features*) deep understanding of each disease

disease-dependent attention block (*disease-dependent features*) internal relationship between diseases



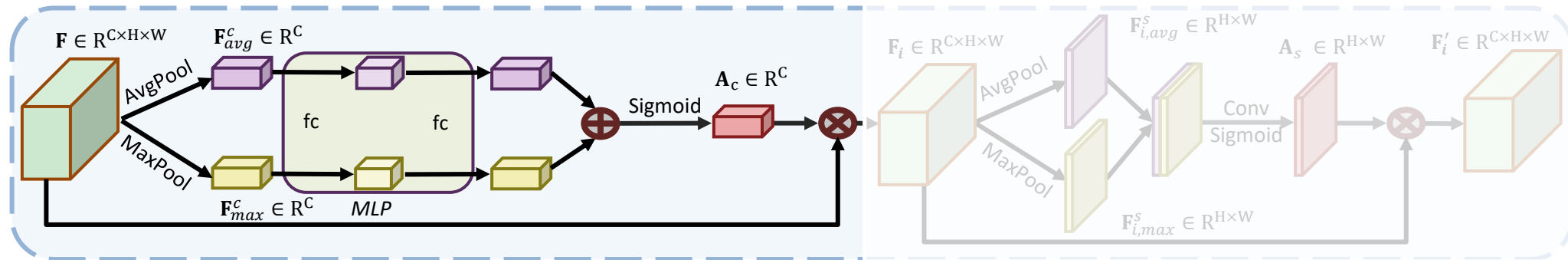
The overall architecture of cross-disease attention network (CANet).

5. Case studies

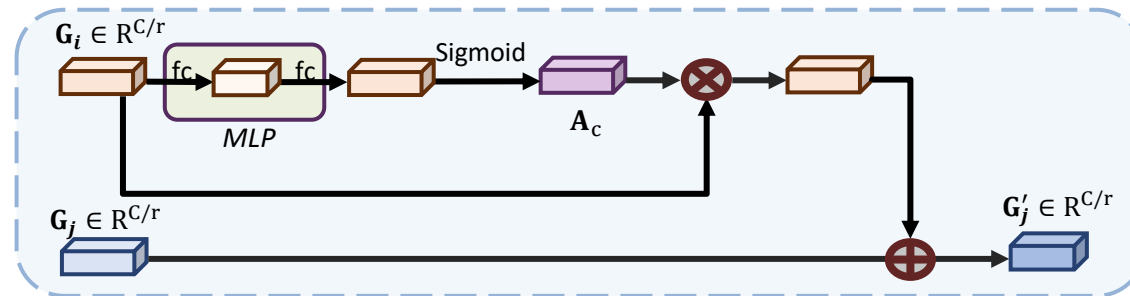


Cross-disease Attention Network (CANet)

Channel-wise attention



(a) Disease-specific attention module.

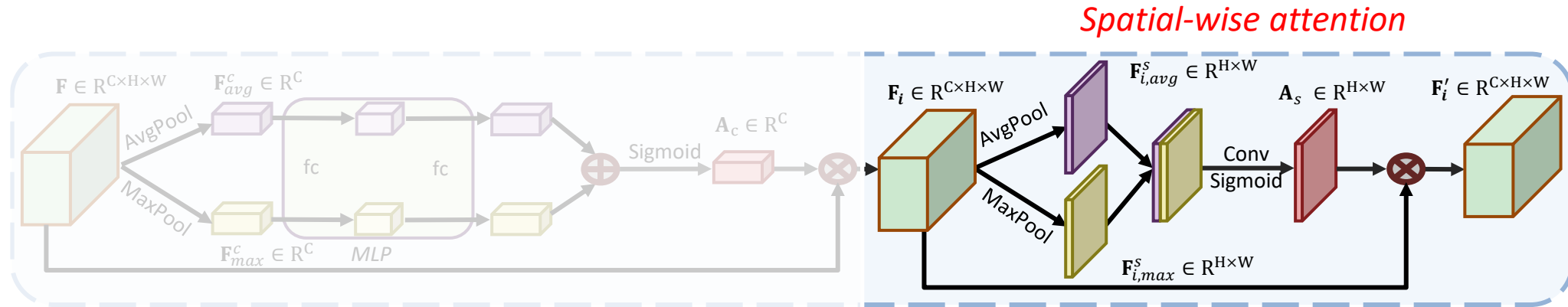


(b) Disease-dependent attention module.

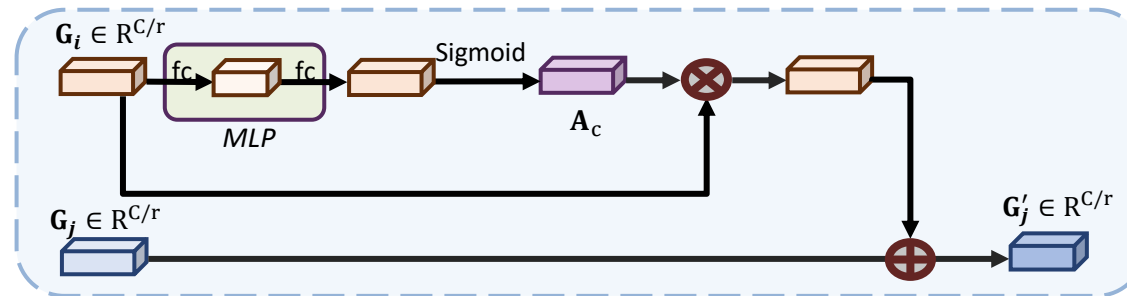
5. Case studies



Cross-disease Attention Network (CANet)



(a) Disease-specific attention module.

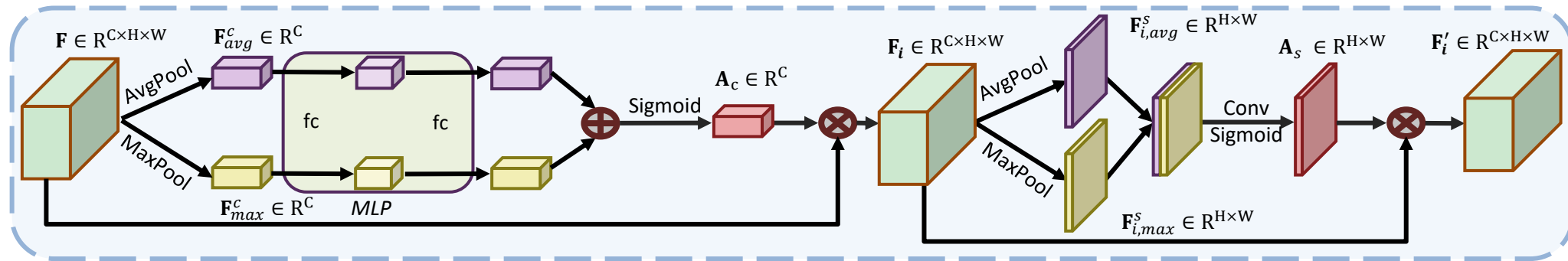


(b) Disease-dependent attention module.

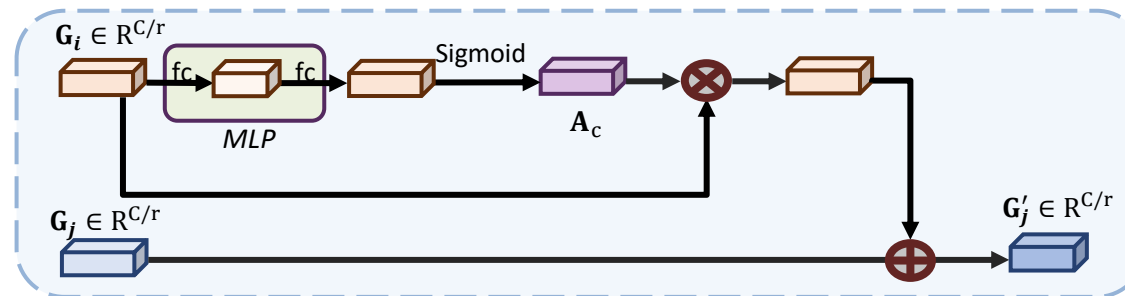
5. Case studies



Cross-disease Attention Network (CANet)



(a) Disease-specific attention module.



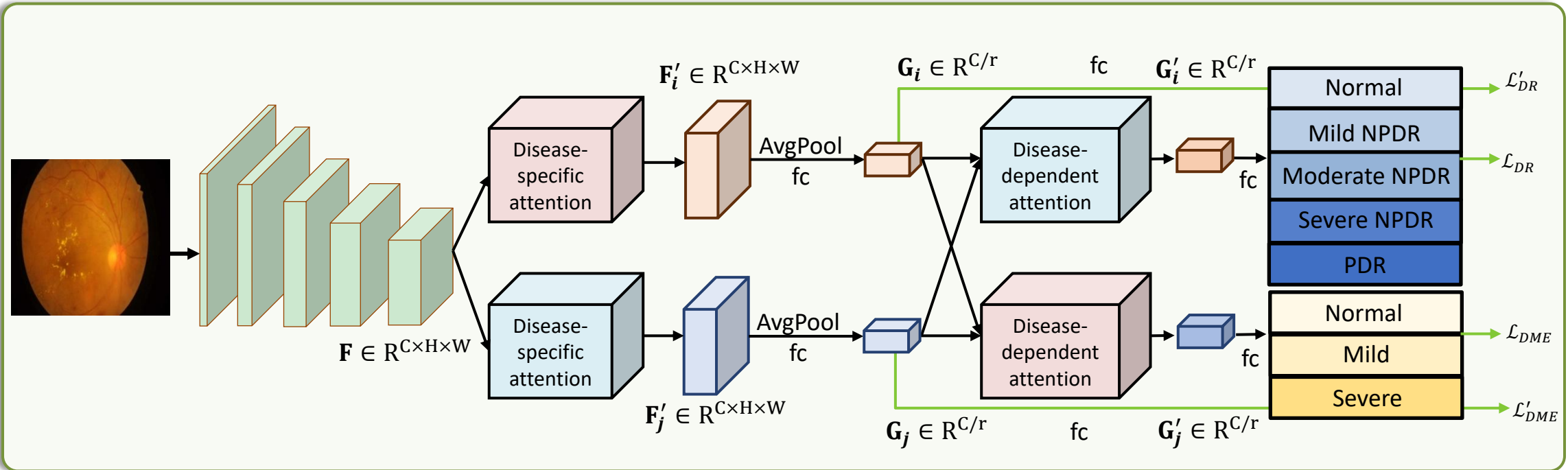
(b) Disease-dependent attention module.

Channel-wise attention

5. Case studies



Cross-disease Attention Network (CANet)



The overall architecture of cross-disease attention network (CANet).

Loss function $\mathcal{L} = \mathcal{L}_{DR} + \mathcal{L}_{DME} + \lambda(\mathcal{L}'_{DR} + \mathcal{L}'_{DME}),$

$$\mathcal{L}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^M y_i^c \log \hat{y}_i^c$$

Weighting factor

5. Case studies



Joint DR and DME grading results on the public Messidor dataset.

Methods	Parameters	Joint Ac	DR					DME				
			AUC	Ac	Pre	Rec	F1	AUC	Ac	Pre	Rec	F1
Individual training (DR)	23.52 M	-	89.5	81.0	76.2	78.9	77.3	-	-	-	-	-
Individual training (DME)	23.52 M	-	-	-	-	-	-	89.1	86.8	62.8	65.2	61.9
Joint training	23.52 M	82.0	94.2	89.1	86.5	88.2	87.2	90.5	90.4	78.7	73.9	75.3
Joint training (complex)	29.04 M	82.8	95.2	91.0	90.5	88.6	89.5	90.3	91.7	73.6	71.2	71.1
Joint training (complex_v2)	29.08 M	82.5	95.4	90.3	87.5	89.8	88.5	89.4	91.0	78.3	72.0	73.4
CANet (d-S only)	29.03 M	84.1	95.8	91.7	91.5	88.8	89.9	90.3	91.0	79.6	71.0	72.5
CANet (d-S; d-D _{DR⇒DME})		84.5	96.0	91.7	91.5	88.7	90.0	89.4	91.9	82.3	72.0	74.8
CANet (d-S; d-D _{DR⇐DME})		84.9	96.3	91.9	90.8	90.1	90.3	91.0	91.9	79.6	73.2	74.6
CANet ($\lambda=0.25$; final model)		85.1	96.3	92.6	90.6	92.0	91.2	92.4	91.2	76.3	70.8	72.4
CANet ($\lambda=0.00$)	29.03 M	84.8	96.3	92.1	92.2	88.6	90.3	91.6	91.5	82.4	73.7	75.3
CANet ($\lambda=0.50$)		84.7	96.1	91.4	89.9	89.7	89.6	92.2	92.0	78.6	74.9	75.3
CANet ($\lambda=0.75$)		84.8	96.5	92.2	92.3	88.8	90.3	91.8	92.0	79.5	71.4	73.7
CANet ($\lambda=1.00$)		84.9	96.3	92.7	91.6	90.6	91.0	90.7	91.7	77.4	69.4	71.0

- Joint training outperforms individual training

5. Case studies



Joint DR and DME grading results on the public Messidor dataset.

Methods	Parameters	Joint Ac	DR					DME				
			AUC	Ac	Pre	Rec	F1	AUC	Ac	Pre	Rec	F1
Individual training (DR)	23.52 M	-	89.5	81.0	76.2	78.9	77.3	-	-	-	-	-
Individual training (DME)	23.52 M	-	-	-	-	-	-	89.1	86.8	62.8	65.2	61.9
Joint training	23.52 M	82.0	94.2	89.1	86.5	88.2	87.2	90.5	90.4	78.7	73.9	75.3
Joint training (complex)	29.04 M	82.8	95.2	91.0	90.5	88.6	89.5	90.3	91.7	73.6	71.2	71.1
Joint training (complex_v2)	29.08 M	82.5	95.4	90.3	87.5	89.8	88.5	89.4	91.0	78.3	72.0	73.4
CANet (d-S only)	29.03 M	84.1	95.8	91.7	91.5	88.8	89.9	90.3	91.0	79.6	71.0	72.5
CANet (d-S; d-D _{DR⇒DME})		84.5	96.0	91.7	91.5	88.7	90.0	89.4	91.9	82.3	72.0	74.8
CANet (d-S; d-D _{DR⇐DME})		84.9	96.3	91.9	90.8	90.1	90.3	91.0	91.9	79.6	73.2	74.6
CANet ($\lambda=0.25$; final model)		85.1	96.3	92.6	90.6	92.0	91.2	92.4	91.2	76.3	70.8	72.4
CANet ($\lambda=0.00$)	29.03 M	84.8	96.3	92.1	92.2	88.6	90.3	91.6	91.5	82.4	73.7	75.3
CANet ($\lambda=0.50$)		84.7	96.1	91.4	89.9	89.7	89.6	92.2	92.0	78.6	74.9	75.3
CANet ($\lambda=0.75$)		84.8	96.5	92.2	92.3	88.8	90.3	91.8	92.0	79.5	71.4	73.7
CANet ($\lambda=1.00$)		84.9	96.3	92.7	91.6	90.6	91.0	90.7	91.7	77.4	69.4	71.0

- Our method outperforms Joint training under the same level of model parameters (*effectiveness of our network design*)

5. Case studies



Joint DR and DME grading results on the public Messidor dataset.

Methods	Parameters	Joint Ac	DR					DME				
			AUC	Ac	Pre	Rec	F1	AUC	Ac	Pre	Rec	F1
Individual training (DR)	23.52 M	-	89.5	81.0	76.2	78.9	77.3	-	-	-	-	-
Individual training (DME)	23.52 M	-	-	-	-	-	-	89.1	86.8	62.8	65.2	61.9
Joint training	23.52 M	82.0	94.2	89.1	86.5	88.2	87.2	90.5	90.4	78.7	73.9	75.3
Joint training (complex)	29.04 M	82.8	95.2	91.0	90.5	88.6	89.5	90.3	91.7	73.6	71.2	71.1
Joint training (complex v2)	29.08 M	82.5	95.4	90.3	87.5	89.8	88.5	89.4	91.0	78.3	72.0	73.4
CANet (d-S only)	29.03 M	84.1	95.8	91.7	91.5	88.8	89.9	90.3	91.0	79.6	71.0	72.5
CANet (d-S; d-D _{DR⇒DME})		84.5	96.0	91.7	91.5	88.7	90.0	89.4	91.9	82.3	72.0	74.8
CANet (d-S; d-D _{DR⇐DME})		84.9	96.3	91.9	90.8	90.1	90.3	91.0	91.9	79.6	73.2	74.6
CANet ($\lambda=0.25$; final model)		85.1	96.3	92.6	90.6	92.0	91.2	92.4	91.2	76.3	70.8	72.4
CANet ($\lambda=0.00$)	29.03 M	84.8	96.3	92.1	92.2	88.6	90.3	91.6	91.5	82.4	73.7	75.3
CANet ($\lambda=0.50$)		84.7	96.1	91.4	89.9	89.7	89.6	92.2	92.0	78.6	74.9	75.3
CANet ($\lambda=0.75$)		84.8	96.5	92.2	92.3	88.8	90.3	91.8	92.0	79.5	71.4	73.7
CANet ($\lambda=1.00$)		84.9	96.3	92.7	91.6	90.6	91.0	90.7	91.7	77.4	69.4	71.0

- Both disease-specific and disease-dependent attentions can contribute to the performance

5. Case studies



Joint DR and DME grading results on the public Messidor dataset.

Methods	Parameters	Joint Ac	DR					DME				
			AUC	Ac	Pre	Rec	F1	AUC	Ac	Pre	Rec	F1
Individual training (DR)	23.52 M	-	89.5	81.0	76.2	78.9	77.3	-	-	-	-	-
Individual training (DME)	23.52 M	-	-	-	-	-	-	89.1	86.8	62.8	65.2	61.9
Joint training	23.52 M	82.0	94.2	89.1	86.5	88.2	87.2	90.5	90.4	78.7	73.9	75.3
Joint training (complex)	29.04 M	82.8	95.2	91.0	90.5	88.6	89.5	90.3	91.7	73.6	71.2	71.1
Joint training (complex_v2)	29.08 M	82.5	95.4	90.3	87.5	89.8	88.5	89.4	91.0	78.3	72.0	73.4
CANet (d-S only)	29.03 M	84.1	95.8	91.7	91.5	88.8	89.9	90.3	91.0	79.6	71.0	72.5
CANet (d-S; d-D _{DR⇒DME})		84.5	96.0	91.7	91.5	88.7	90.0	89.4	91.9	82.3	72.0	74.8
CANet (d-S; d-D _{DR⇐DME})		84.9	96.3	91.9	90.8	90.1	90.3	91.0	91.9	79.6	73.2	74.6
CANet ($\lambda=0.25$; final model)		85.1	96.3	92.6	90.6	92.0	91.2	92.4	91.2	76.3	70.8	72.4
CANet ($\lambda=0.00$)	29.03 M	84.8	96.3	92.1	92.2	88.6	90.3	91.6	91.5	82.4	73.7	75.3
CANet ($\lambda=0.50$)		84.7	96.1	91.4	89.9	89.7	89.6	92.2	92.0	78.6	74.9	75.3
CANet ($\lambda=0.75$)		84.8	96.5	92.2	92.3	88.8	90.3	91.8	92.0	79.5	71.4	73.7
CANet ($\lambda=1.00$)		84.9	96.3	92.7	91.6	90.6	91.0	90.7	91.7	77.4	69.4	71.0

- Our method achieves **best** when $\lambda = 0.25$

5. Case studies



Comparisons with other multi-task learning methods.

Methods	Joint Ac	DR					DME				
		AUC	Ac	Pre	Rec	F1	AUC	Ac	Pre	Rec	F1
Multi-task net Chen et al.	82.4	94.8	89.9	89.7	85.7	87.5	90.5	90.5	79.1	70.8	72.2
MTMR-Net Liu et al.	83.1	94.9	90.3	90.0	86.7	88.1	90.6	90.4	79.8	73.2	75.4
CANet (ours)	85.1	96.3	92.6	90.6	92.0	91.2	92.4	91.2	76.3	70.8	72.4

Comparisons with state-of-the-art methods on the Messidor dataset.

Methods	DR		DME	
	AUC	Ac	AUC	Ac
Lesion-based Pires et al.	76.0	-	-	-
Fisher Vector Pires et al.	86.3	-	-	-
VNXX/LGI Vo et al.	88.7	89.3	-	-
CKML Net/LGI Vo et al.	89.1	89.7	-	-
Comprehensive CAD Sanchez et al.	91.0	-	-	-
DSF-RFcara Seoud et al.	91.6	-	-	-
Clinical B Sanchez et al.	92.0	-	-	-
Clinical A Sanchez et al.	94.0	-	-	-
Zoom-in-net Wang et al. †	95.7	91.1	-	-
DME classifier Al et al.	-	-	-	88.8
CANet (ours)	96.3	92.6	92.4	91.2

- **2% higher** than other multi-task learning methods.
- **Clearly outperforms** than others.

“†”: denotes using additional dataset EyePACS (Cuadros et al.) as the pretrain.

5. Case studies



Results on the IDRiD challenge leaderboard.

Methods	Joint Ac	Rank
CANet (ours)	65.1	1
lzyuncc	63.1	2
VRT	55.3	3
Mammoth	51.5	4
HarangiM1	47.6	5
AVSASVA	47.6	5
HarangiM2	40.8	6

Results are from the IDRiD 2018 challenge website.

- **Clearly outperforms** than other results.

Ablation Study on the IDRiD challenge leaderboard.

Methods	Joint Ac
Joint training	59.2
CANet (d-S only)	60.2
CANet (d-S, d-D $_{DR \Rightarrow DME}$)	62.1
CANet (d-S, d-D $_{DR \Leftarrow DME}$)	63.1
CANet (final model)	65.1

- Both disease-specific and disease-dependent attentions are useful.
- Results **keep consistent** with those in the Messidor dataset.

5. Case studies



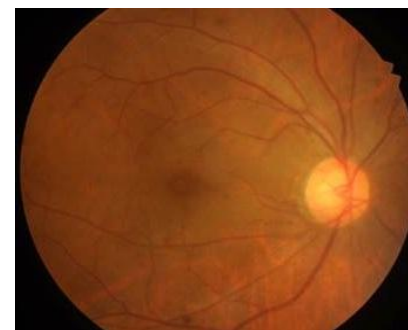
Joint DR and DME grading results on fundus photography on the Messidor dataset.



DR 3: 0.00 0.00 0.06 **0.73** 0.20
DME 2: 0.00 0.00 **0.99**



DR 2: 0.00 0.00 **0.92** 0.07 0.00
DME 2: 0.00 0.00 **1.00**



DR 0: **0.69** 0.23 0.07 0.00 0.00
DME 0: **0.85** 0.10 0.04



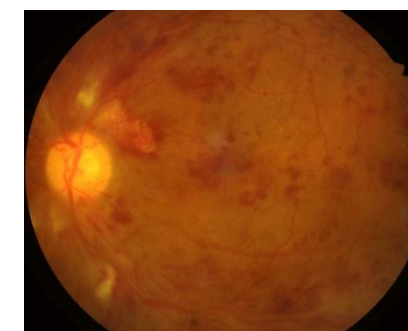
DR 2: 0.00 0.00 **0.99** 0.01 0.00
DME 2: 0.00 0.08 **0.92**



DR 2: 0.07 0.00 **0.86** 0.00 0.07
DME 1: 0.05 **0.95** 0.00



DR 3: 0.00 0.00 0.00 **0.64** 0.35
DME 2: 0.00 0.00 **0.99**



DR 4: 0.00 0.00 0.00 0.05 **0.95**
DME 2: 0.00 0.10 **0.90**



DR 2: 0.00 0.00 **0.99** 0.00 0.00
DME 2: 0.00 0.14 **0.86**

Ground-truth
provided by doctors

The output probabilities for DME,
belonging to grade 0,1,2 respectively.

The output probabilities for DR,
belonging to grade 0,1,2,3,4 respectively.

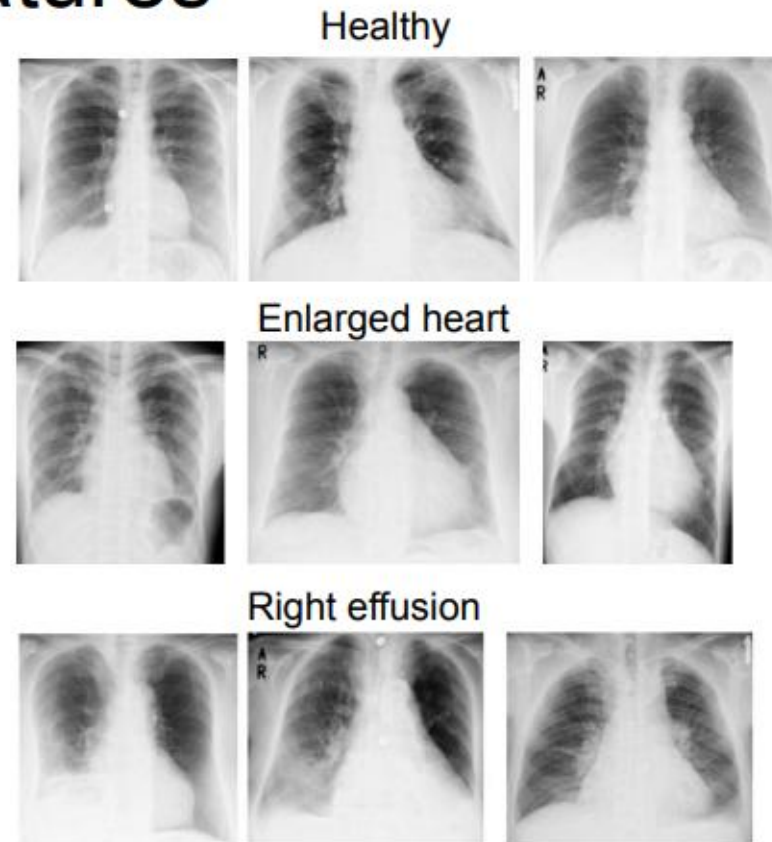
5. Case studies



Early steps of deep learning in medical imaging: using ImageNet CNN features

Bar et al. 2015

- Input: Chest **x-ray images**
- Output: Several binary classification tasks
 - Right pleural effusion or not
 - Enlarged heart or not
 - Healthy or abnormal
- Very small dataset: 93 frontal chest x-ray images



Q: How might we approach this problem?

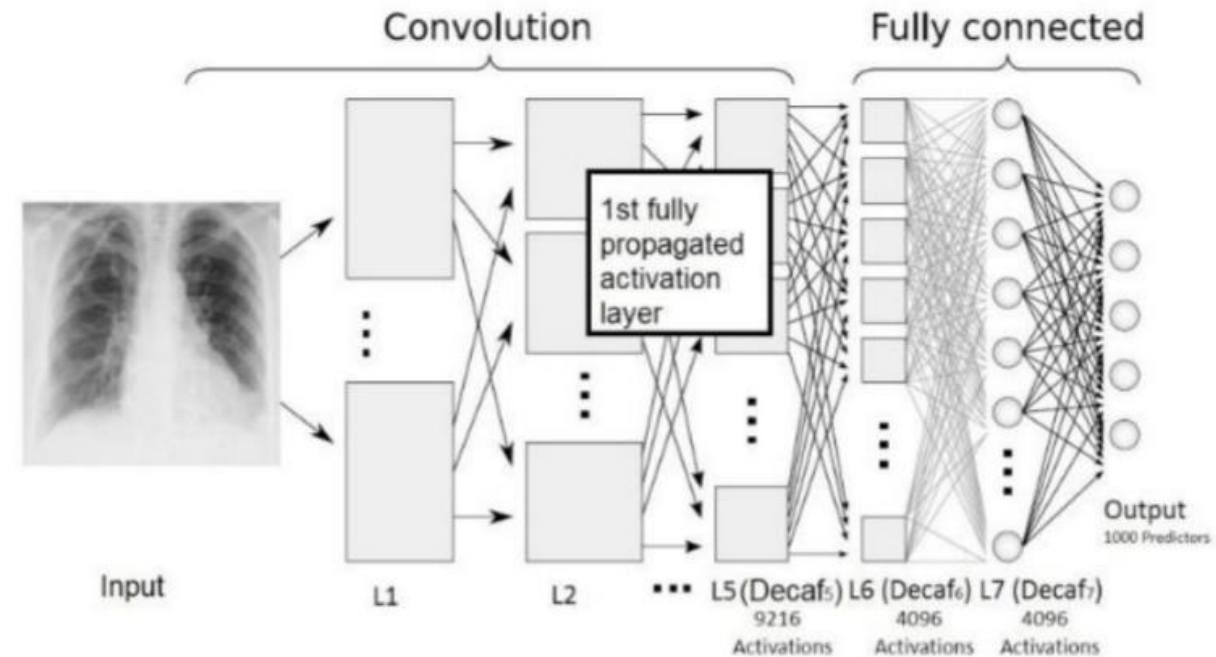
Bar et al. Deep learning with non-medical training used for chest pathology identification. SPIE, 2015.

5. Case studies



Bar et al. 2015

- Did not train a deep learning model on the medical data
- Instead, extracted features from an AlexNet trained on ImageNet
 - 5th, 6th, and 7th layers
- Used extracted features with an SVM classifier
- Performed zero-mean unit-variance normalization of all features
- Evaluated combination with other hand-crafted image features



5. Case studies



Bar et al. 2015

Table 1. Right Pleural Effusion Condition.

	Low Level		High Level	Deep			Fusion
	LBP	GIST	PiCoDes	Decaf L5	Decaf L6	Decaf L7	PiCoDes+Decaf L5
Sensitivity	0.71	0.79	0.79	0.93	0.86	0.86	0.93
Specificity	0.77	0.92	0.91	0.84	0.86	0.80	0.84
AUC	0.75	0.93	0.91	0.92	0.91	0.84	0.93

Table 2. Healthy vs. Pathology.

	Low Level		High Level	Deep			Fusion
	LBP	GIST	PiCoDes	Decaf L5	Decaf L6	Decaf L7	PiCoDes+Decaf L5
Sensitivity	0.65	0.68	0.59	0.73	0.89	0.76	0.81
Specificity	0.61	0.66	0.79	0.80	0.64	0.64	0.79
AUC	0.63	0.72	0.72	0.78	0.79	0.72	0.79

Table 3. Enlarged Heart Condition.

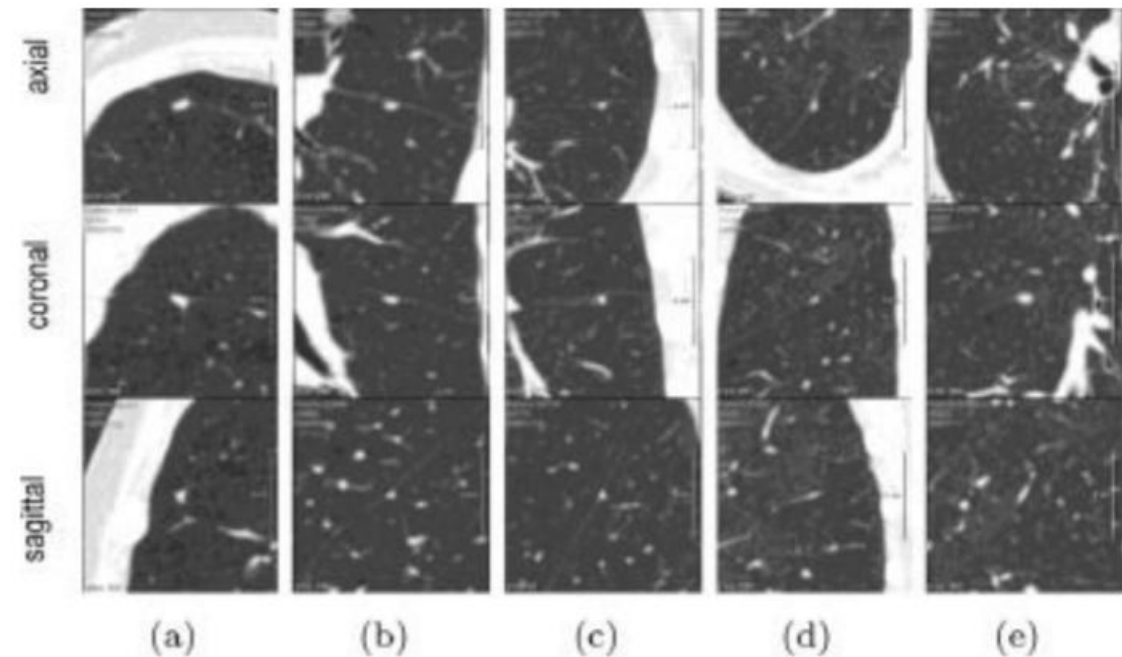
	Low Level		High Level	Deep			Fusion
	LBP	GIST	PiCoDes	Decaf L5	Decaf L6	Decaf L7	PiCoDes+Decaf L5
Sensitivity	0.75	0.79	0.79	0.88	0.79	0.79	0.83
Specificity	0.78	0.81	0.84	0.78	0.88	0.77	0.84
AUC	0.80	0.82	0.87	0.87	0.84	0.79	0.89

5. Case studies



Ciampi et al. 2015

- Task: classification of lung nodules in **3D CT scans** as peri-fissural nodules (PFN, likely to be benign) or not
- Dataset: 568 nodules from 1729 scans at a single institution. (65 typical PFNs, 19 atypical PFNs, 484 non-PFNs).
- Data pre-processing: prescaling from CT hounsfield units (HU) into [0,255]. Replicate 3x across R,G,B channels to match input dimensions of ImageNet-trained CNNs.



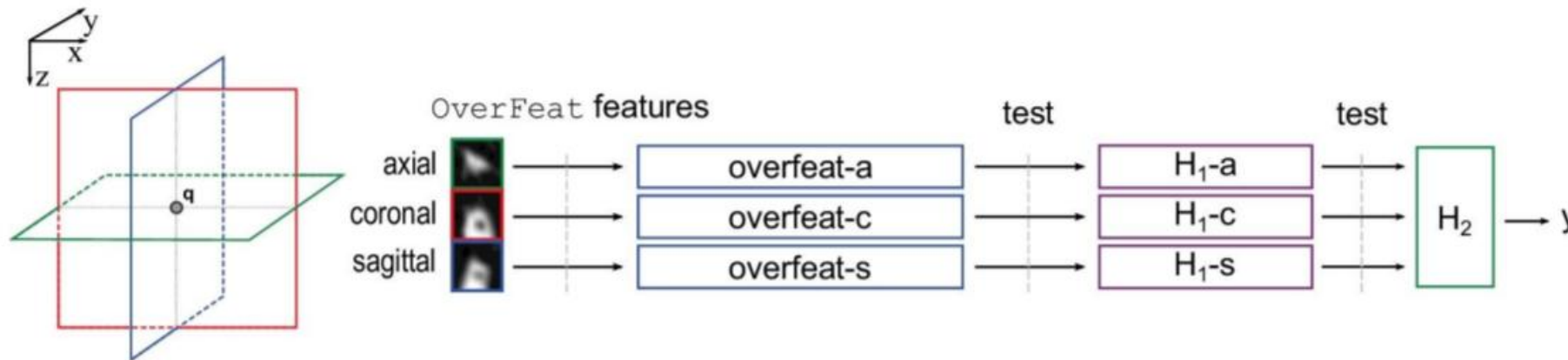
Ciampi et al. Automatic classification of pulmonary peri-fissural nodules in computed tomography using an ensemble of 2D views and a convolutional neural network out-of-the-box. Medical Image Analysis, 2015.

5. Case studies



Ciampi et al. 2015

- Also extracted features from a deep learning model trained on ImageNet
 - Overfeat feature extractor (similar to AlexNet, but trained using additional losses for localization and detection)
 - To capture 3D information, extracted features from 3 different 2D views of each nodule, then input into 2-stage classifier (independent predictions on each view first, then outputs combined into second classifier).



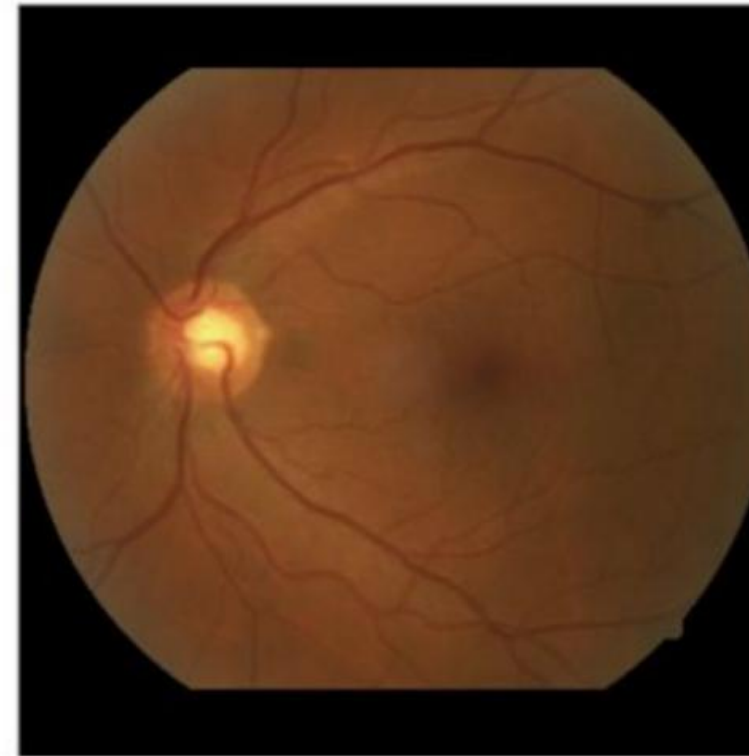
Ciampi et al. Automatic classification of pulmonary peri-fissural nodules in computed tomography using an ensemble of 2D views and a convolutional neural network out-of-the-box. Medical Image Analysis, 2015.

5. Case studies



Gulshan et al. 2016

- **Task:** Binary classification of referable diabetic retinopathy from **retinal fundus photographs**
- **Input:** Retinal fundus photographs
- **Output:** Binary classification of referable diabetic retinopathy (y in $\{0,1\}$)
 - Defined as moderate and worse diabetic retinopathy, referable diabetic macular edema, or both



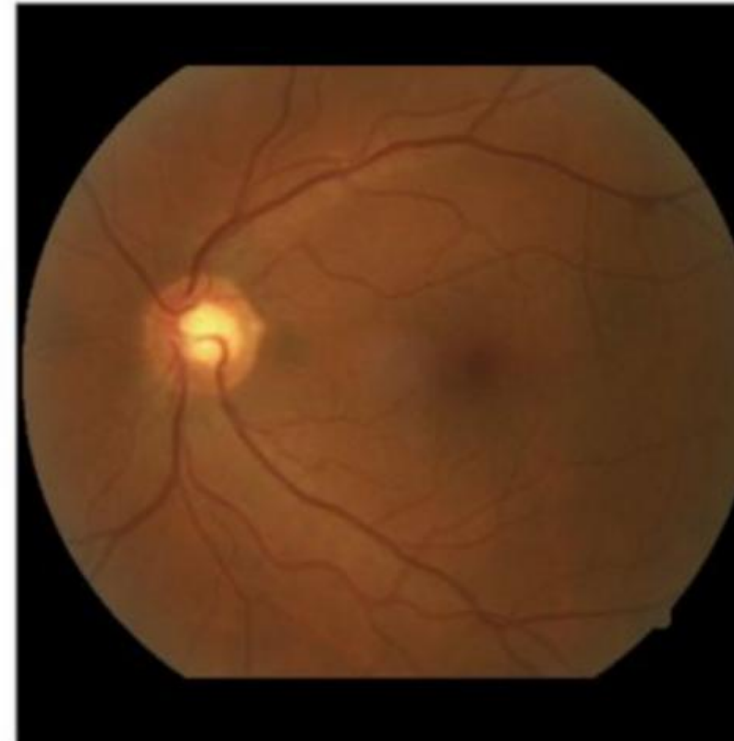
Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016. 106

5. Case studies



Gulshan et al. 2016

- **Dataset:**
 - 128,175 images, each graded by 3-7 ophthalmologists.
 - 54 total graders, each paid to grade between 20 to 62508 images.
- **Data preprocessing:**
 - Circular mask of each image was detected and rescaled to be 299 pixels wide
- **Model:**
 - Inception-v3 CNN, with ImageNet pre-training
 - Multiple BCE losses corresponding to different binary prediction problems, which were then used for final determination of referable diabetic retinopathy



Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016.

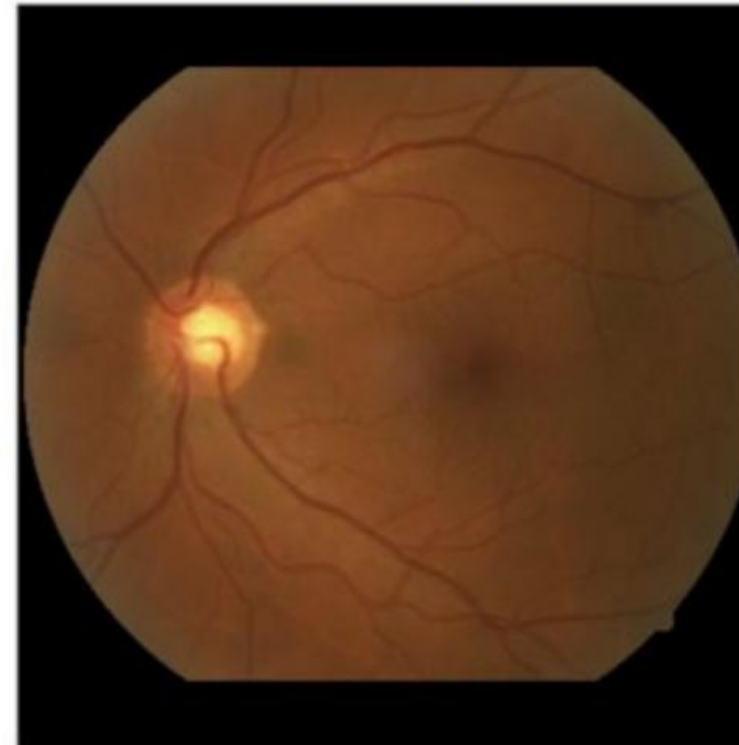
5. Case studies



Gulshan et al. 2016

- **Dataset:**
 - 128,175 images, each graded by 3-7 ophthalmologists.
 - 54 total graders, each paid to grade between 20 to 62508 images.
- **Data preprocessing:**
 - Circular mask of each image was detected and rescaled to be 299 pixels wide
- **Model:**
 - Inception-v3 CNN, with ImageNet pre-training
 - Multiple BCE losses corresponding to different binary prediction problems, which were then used for final determination of referable diabetic retinopathy

Graders provided finer-grained labels which were then consolidated into (easier) binary prediction problems



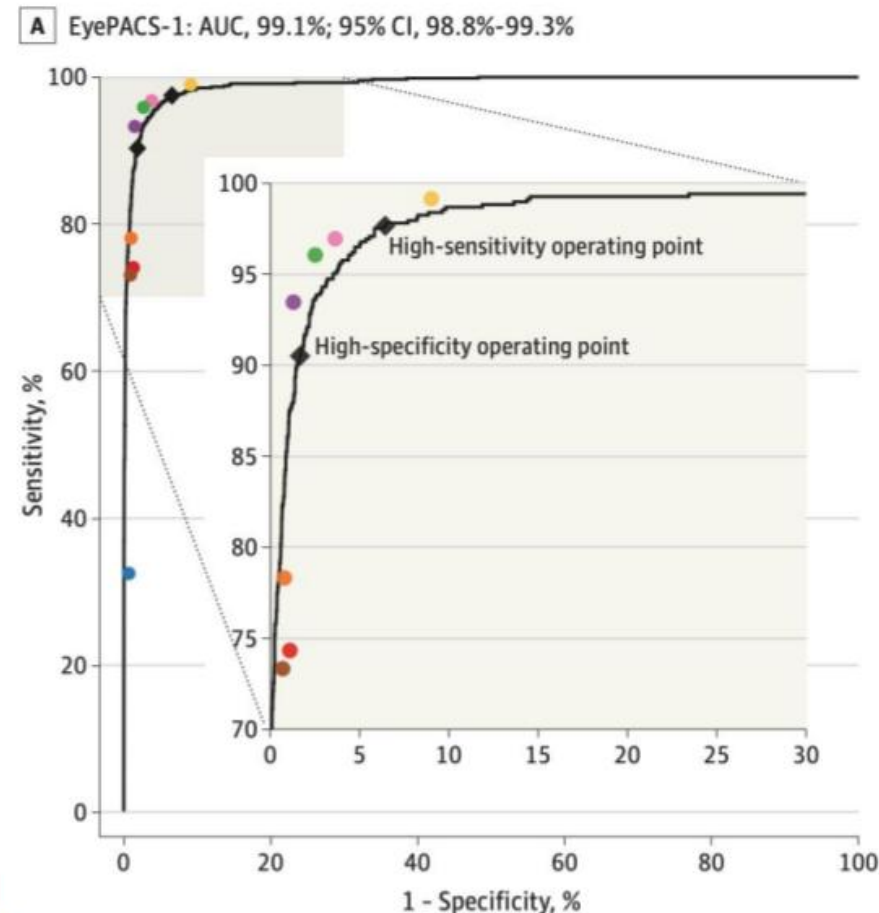
Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016.

5. Case studies



Gulshan et al. 2016

- **Results:**
 - Evaluated using ROC curves, AUC, sensitivity and specificity analysis

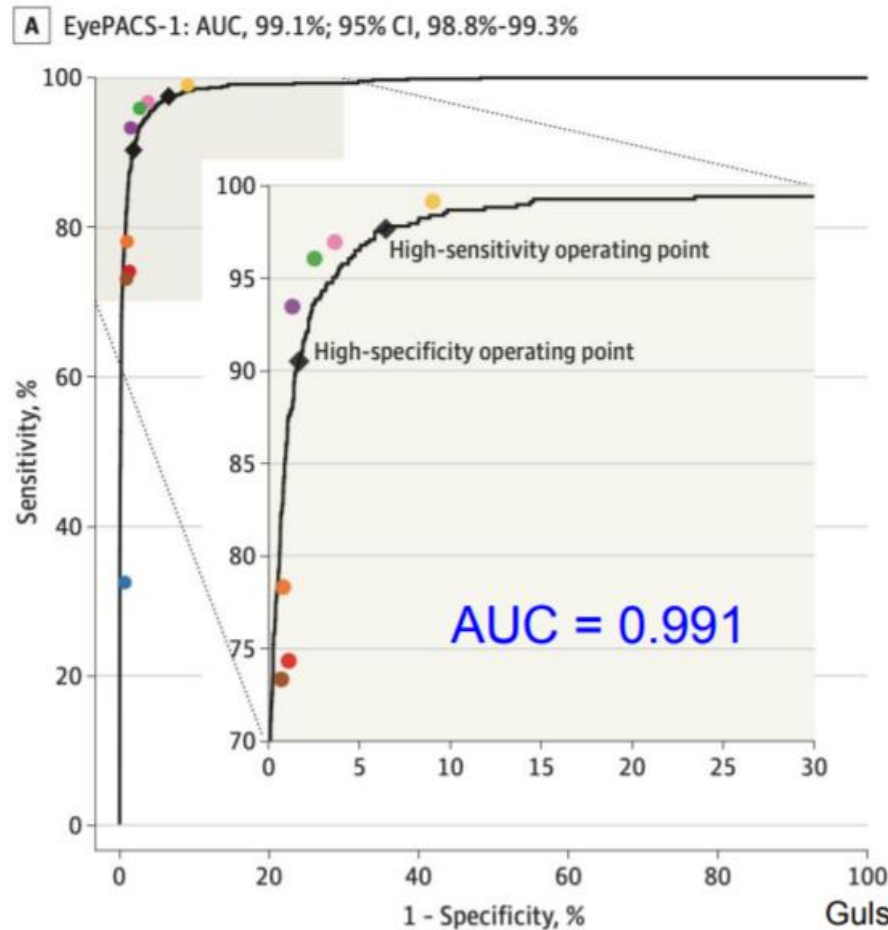


Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016.

5. Case studies



Gulshan et al. 2016



Looked at different operating points

- High-specificity point approximated ophthalmologist specificity for comparison. Should also use high-specificity to make decisions about high-risk actions.
- High-sensitivity point should be used for screening applications.

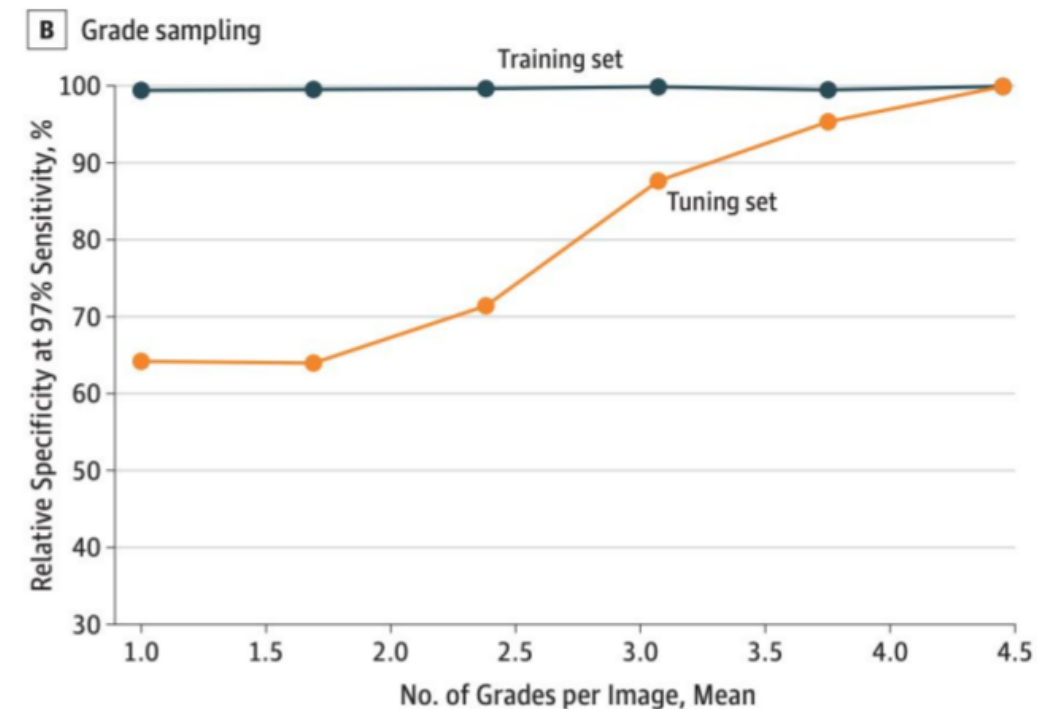
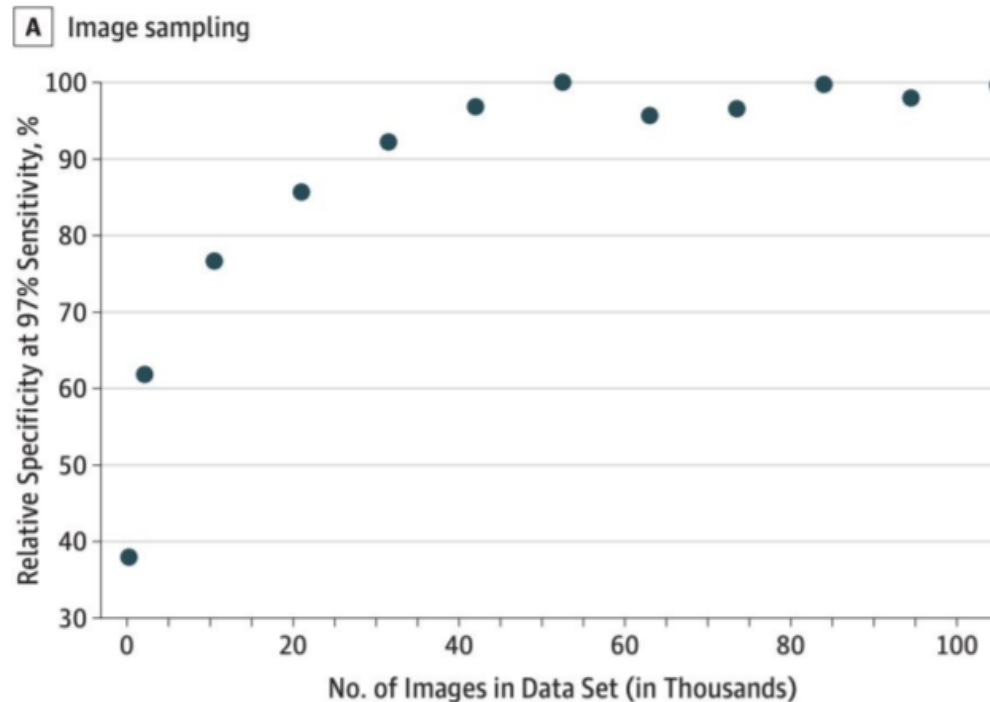
Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016. 110

5. Case studies



Gulshan et al. 2016

Q: What could explain the difference in trends for reducing # grades / image on training set vs. tuning set, on tuning set performance?



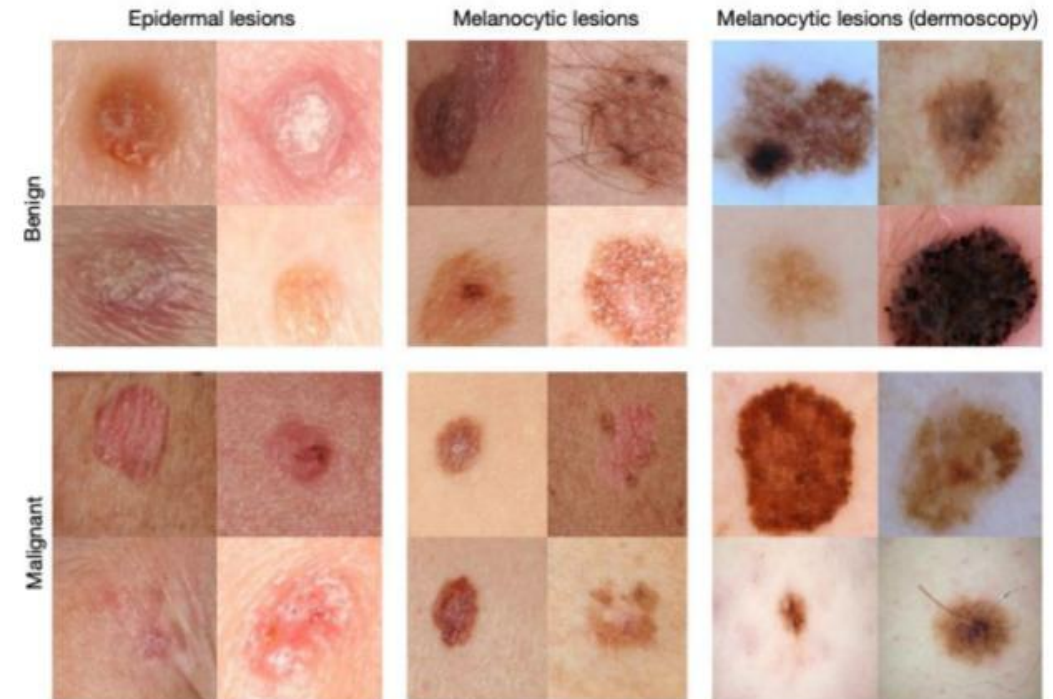
Gulshan, et al. Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs. JAMA, 2016.

5. Case studies



Esteva et al. 2017

- Two binary classification tasks: malignant vs. benign lesions of epidermal or melanocytic origin
- Inception-v3 (GoogLeNet) CNN with ImageNet pre-training
- Fine-tuned on dataset of 129,450 lesions (from several sources) comprising 2,032 diseases
- Evaluated model vs. 21 or more dermatologists in various settings



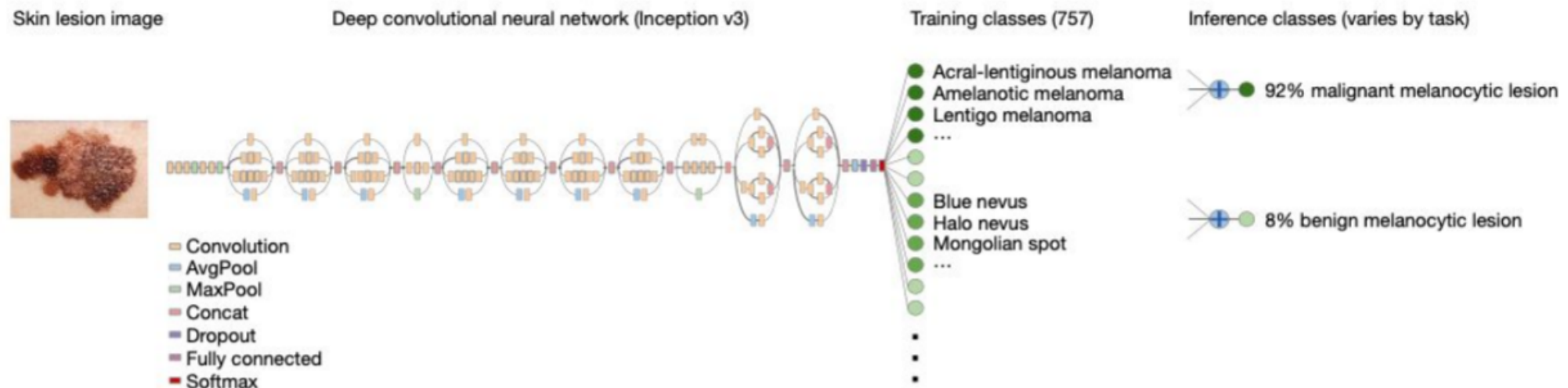
Esteva*, Kuprel*, et al. Dermatologist-level classification of skin cancer with deep neural networks. Nature, 2017.

5. Case studies



Esteva et al. 2017

- Train on finer-grained classification (757 classes) but perform binary classification at inference time by summing probabilities of fine-grained sub-classes
- The stronger fine-grained supervision during the training stage improves inference performance!



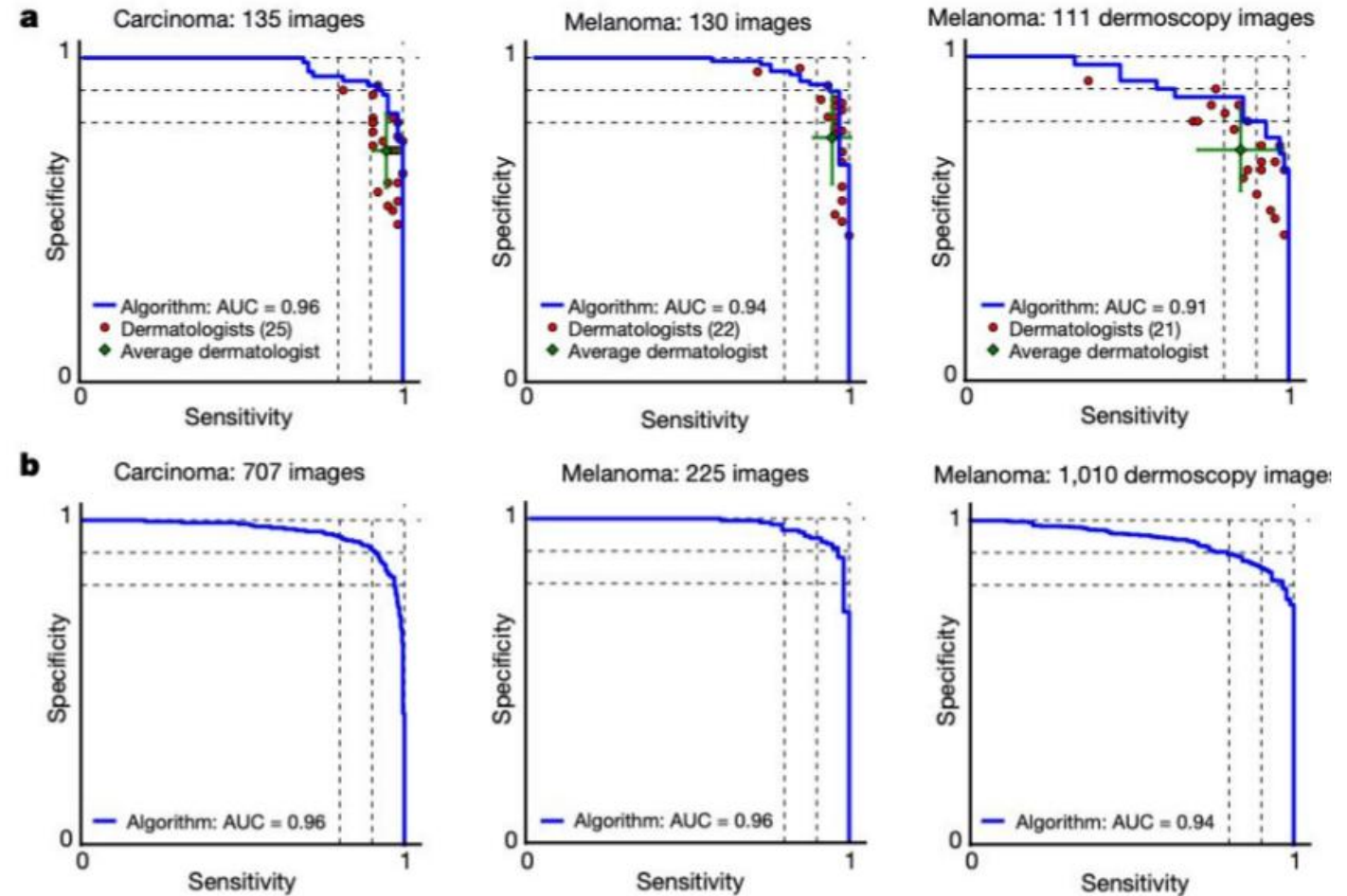
Esteva*, Kuprel*, et al. Dermatologist-level classification of skin cancer with deep neural networks. Nature, 2017.

5. Case studies



Esteva et al. 2017

- Evaluation of algorithm vs. dermatologists



Esteva*, Kuprel*, et al. Dermatologist-level classification of skin cancer with deep neural networks. Nature, 2017.

5. Case studies



Lakhani and Sundaram 2017

- Binary classification of pulmonary tuberculosis from x-rays
- Four de-identified datasets
- 1007 chest x-rays (68% train, 17.1% validation, 14.9% test)
- Tried training CNNs from scratch as well as fine-tuning from ImageNet

AUC Test Dataset				
Parameter	Untrained	Pretrained	Untrained with Augmentation*	Pretrained with Augmentation*
AlexNet	0.90 (0.84, 0.95)	0.98 (0.95, 1.00)	0.95 (0.90, 0.98)	0.98 (0.94, 0.99)
GoogLeNet	0.88 (0.81, 0.92)	0.97 (0.93, 0.99)	0.94 (0.89, 0.97)	0.98 (0.94, 1.00)
Ensemble				0.99 (0.96, 1.00)

Note.—Data in parentheses are 95% confidence interval.

* Additional augmentation of 90, 180, 270 rotations, and Contrast Limited Adaptive Histogram Equalization processing.

All training images were resized to 256x256 and underwent base data augmentation of random 227x227 cropping and mirror images. Additional data augmentation experiments in results table.

5. Case studies



Lakhani and Sundaram 2017

- Binary classification of pulmonary tuberculosis from x-rays
- Four de-identified datasets
- 1007 chest x-rays (68% train, 17.1% validation, 14.9% test)
- Tried training CNNs from scratch as well as fine-tuning from ImageNet

AUC Test Dataset				
Parameter	Untrained	Pretrained	Untrained with Augmentation*	Pretrained with Augmentation*
AlexNet	0.90 (0.84, 0.95)	0.98 (0.95, 1.00)	0.95 (0.90, 0.98)	0.98 (0.94, 0.99)
GoogLeNet	0.88 (0.81, 0.92)	0.97 (0.93, 0.99)	0.94 (0.89, 0.97)	0.98 (0.94, 1.00)
Ensemble				0.99 (0.96, 1.00)

Note.—Data in parentheses are 95% confidence interval.

* Additional augmentation of 90, 180, 270 rotations, and Contrast Limited Adaptive Histogram Equalization processing.

All training images were resized to 256x256 and underwent base data augmentation of random 227x227 cropping and mirror images. Additional data augmentation experiments in results table.

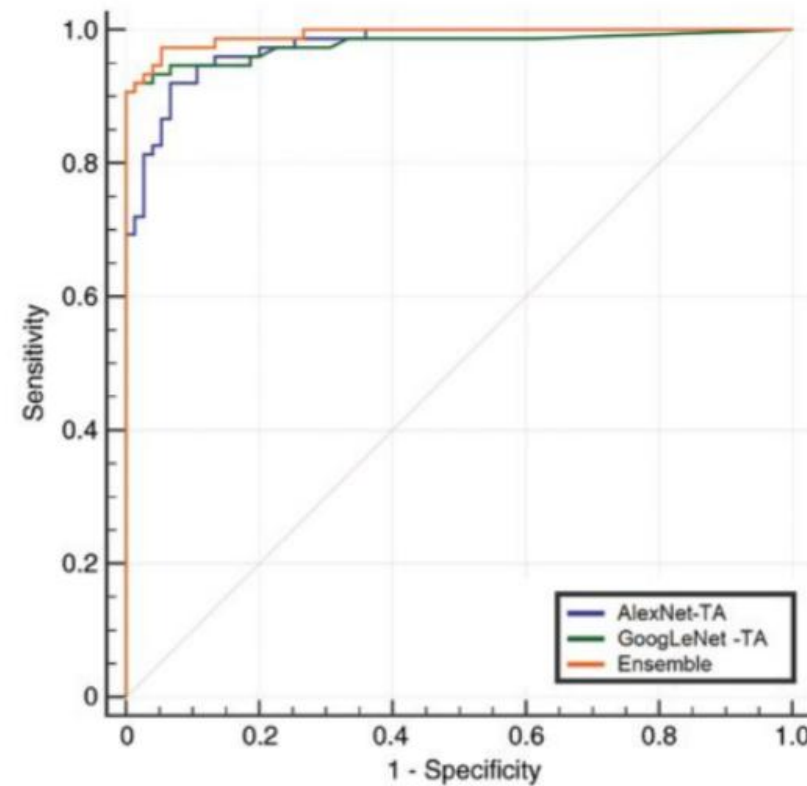
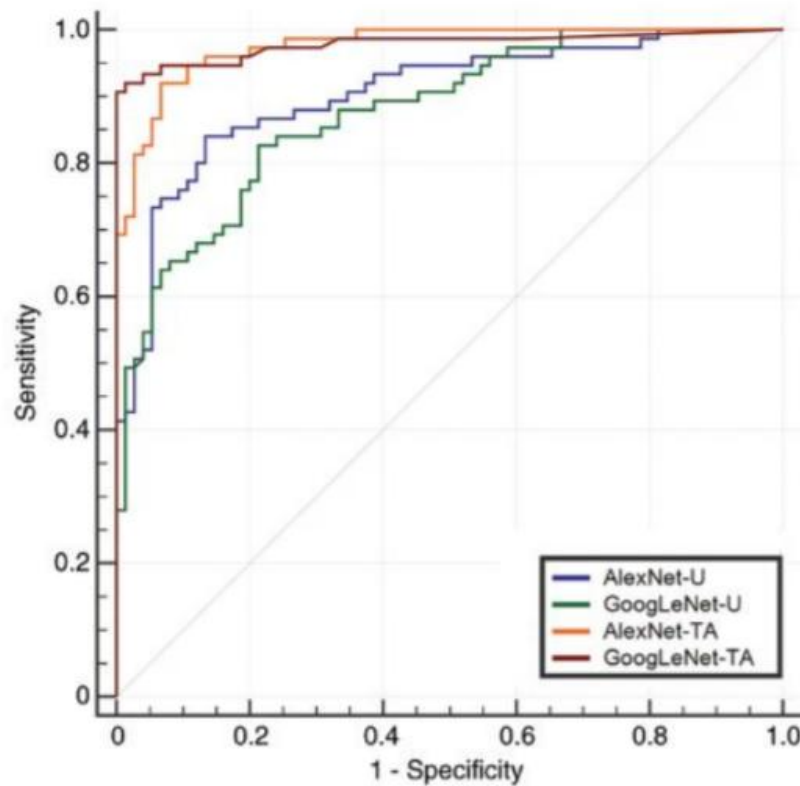
Often resize to match input size of pre-trained networks. Also fine approach to making high-res dataset easier to work with!

5. Case studies



Lakhani and Sundaram 2017

Performed further analysis at optimal threshold determined by the Youden Index.

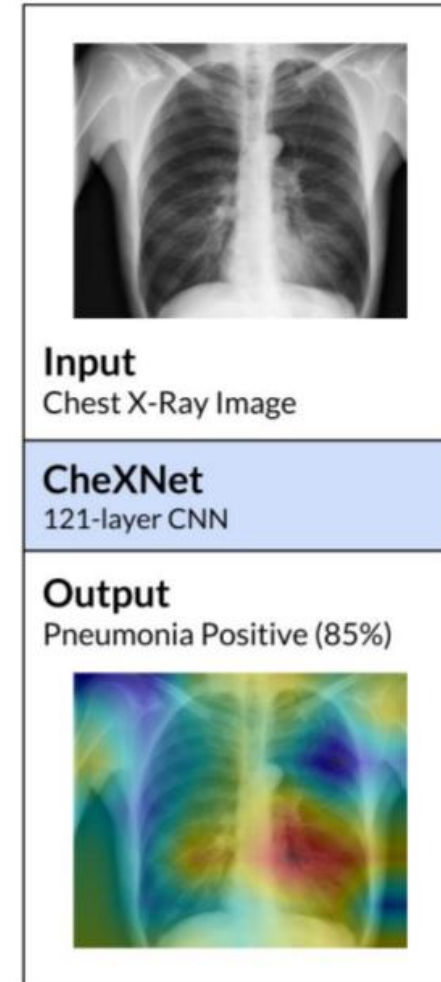


5. Case studies



Rajpurkar et al. 2017

- Binary classification of pneumonia presence in chest X-rays
- Used ChestX-ray14 dataset with over 100,000 frontal X-ray images with 14 diseases
- 121-layer DenseNet CNN
- Compared algorithm performance with 4 radiologists
- Also applied algorithm to other diseases to surpass previous state-of-the-art on ChestX-ray14



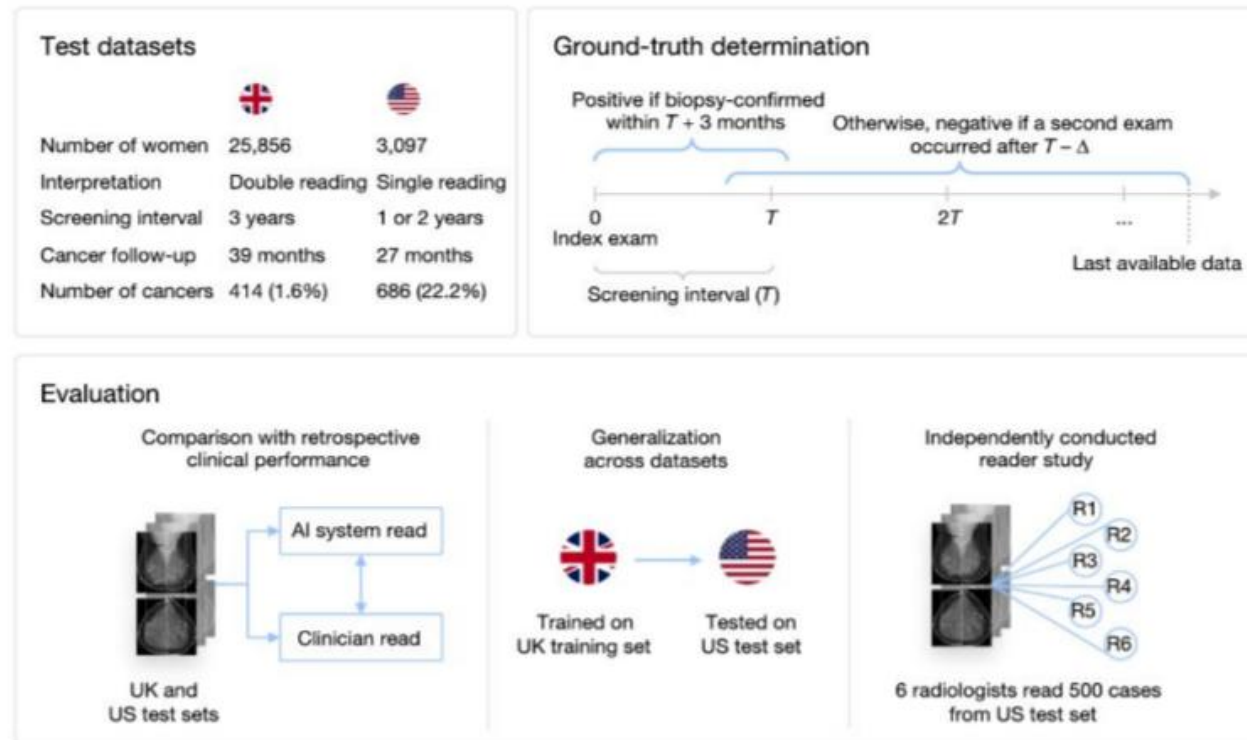
Rajpurkar et al. CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. 2017.

5. Case studies



McKinney et al. 2020

- Binary classification of breast cancer in mammograms
- International dataset and evaluation, across UK and US



McKinney et al. International evaluation of an AI system for breast cancer screening. Nature, 2020.



Today we saw:

- Deep learning models for image classification
- Data considerations for image classification models
- Evaluating image classification models
- Case studies

Next time: Advanced Vision Models (Segmentation)